

UNIVERSITY OF CAMBRIDGE

LECTURE NOTES AND RESEARCH GUIDE

Neural Samplers

Diffusion models, stochastic control and Schrödinger bridges for sampling unnormalised densities in machine learning and generative modelling

A concise course for doctoral research

About this book

Neural samplers are generative models trained from an energy instead of a data set: they learn to sample a density known only up to a constant, $\pi(x) \propto e^{-\mathcal{E}(x)}$. Such targets are everywhere in machine learning, from reward-tilted and guided diffusion models to Bayesian posteriors and energy-based models. This book develops the subject from first principles (measure transport, stochastic calculus, stochastic optimal control, Schrödinger bridges), with proofs and intuitions, then covers Boltzmann generators, diffusion samplers, annealed transport, energy matching, Adjoint Sampling and Schrödinger-bridge samplers, with pseudo-code and tested Python. It ends with geometry, machine-learning applications, molecules and open research questions.

Date: 3 November 2026
Keywords: neural samplers, generative modelling, diffusion models, stochastic optimal control, adjoint sampling, Schrödinger bridges

Contents

Preface **1**

Notation and conventions 2

1 Densities, transport and divergences **4**

Background I: what sampling means, change of variables, continuity equation, KL, importance sampling

1.1 What does it mean to sample? 4
1.2 Moving densities with maps 4
1.3 Moving densities with velocity fields 5
1.4 Measuring the distance between distributions 6
1.5 Importance sampling and the normalising constant 7
1.6 Forward versus reverse KL 8

2 Diffusions: from random walks to SDEs **11**

Background II: random walks and path densities, Fokker–Planck, Girsanov, time reversal, bridges

2.1 A Gaussian random walk 11
2.2 Brownian motion and stochastic differential equations 12
2.3 How densities evolve 12
2.4 Langevin dynamics, Ornstein–Uhlenbeck and metastability 13
2.5 Comparing two diffusions: the Girsanov theorem 14
2.6 The Feynman–Kac formula 16
2.7 Time reversal 16
2.8 Building SDEs that carry one distribution to another 17

3 Reweighting paths: optimal control and Schrödinger bridges **20**

Background III: reweighting a random walk, the Gibbs principle of control, sampling as control, adjoints, Schrödinger bridges

3.1 Warm-up: reweighting a random walk 20
3.2 The control problem and its Gibbs solution 21
3.3 Sampling as a control problem 22
3.4 The adjoint representation of the optimal control 24
3.5 Schrödinger bridges 25

4 The sampling problem: baselines, metrics and benchmarks **30**

What a sampler must deliver, MCMC, annealed importance sampling, SMC, a map of neural samplers, evaluation

4.1 What does it mean to solve the problem? 30
4.2 Monte Carlo baselines 30
4.3 A map of neural samplers 31
4.4 Evaluation 31
4.5 Benchmarks 32

5 Boltzmann generators: normalising flows as samplers . . . **34**
Flows with exact densities, reverse and forward KL, reweighting, FAB, symmetry, flow matching, transferable and sequential BGs
5.1 The idea . . . 34
5.2 Architectures and their densities . . . 35
5.3 Training objectives . . . 35
5.4 Flow Annealed importance sampling Bootstrap (FAB) . . . 36
5.5 Symmetry . . . 37
5.6 Flow matching for Boltzmann generators . . . 37
5.7 Transferable and sequential Boltzmann generators . . . 38

6 Diffusion samplers and path-space divergences . . . **40**
Comparing path densities; PIS, DDS and DIS; reverse KL, log-variance and trajectory balance; off-policy training
6.1 The discrete-time framework . . . 40
6.2 Three classical instances . . . 41
6.3 Losses on path space . . . 42
6.4 Beyond first-order dynamics . . . 44

7 Annealed transport, Jarzynski weights and SMC hybrids . . . **46**
Annealing paths, the escorted Jarzynski equality, zero-variance transport, NETS and CMCD, SMC with learned transitions
7.1 Annealing paths . . . 46
7.2 The escorted Jarzynski equality . . . 46
7.3 Learning the transport: NETS and CMCD . . . 48
7.4 Sequential Monte Carlo with learned transitions . . . 48

8 Energy- and score-matching samplers . . . **51**
Noised densities and their scores, iDEM, NEM and BNEM, why high noise is hard, temperature and other remedies
8.1 Setting . . . 51
8.2 Three expressions for the noised score . . . 51
8.3 Where the estimates fail . . . 52
8.4 Iterated Denoising Energy Matching (iDEM) . . . 53
8.5 Noised energy matching and bootstrapping . . . 54
8.6 Temperature, interpolants and other remedies . . . 54

9 Adjoint Sampling . . . **56**
A control problem whose optimal drift is an average endpoint force; exact bridges; reciprocal adjoint matching; buffers; scalability
9.1 Why another diffusion sampler? . . . 56
9.2 The control problem . . . 56
9.3 Reciprocal Adjoint Matching . . . 57
9.4 The algorithm . . . 58
9.5 Symmetries and molecular systems . . . 59
9.6 Relations to other methods . . . 60

10 Schrödinger-bridge samplers . . . **62**
ASBS: adjoint and corrector matching as IPF; annealed references (NAAS); metadynamics biases (WT-ASBS)
10.1 The Adjoint Schrödinger Bridge Sampler . . . 62
10.2 Annealed references: NAAS . . . 65
10.3 Enhanced sampling with collective variables: WT-ASBS . . . 66

11 Sampling on manifolds and with symmetries . . . **68**
Diffusions on curved spaces, what transfers and what does not, symmetries, torsion tori, exact Adjoint Sampling on tori, Riemannian samplers
11.1 A minimal geometric toolkit . . . 68
11.2 Diffusions on manifolds . . . 68
11.3 Heat kernels and bridges . . . 70
11.4 Symmetry: quotients and equivariance . . . 70
11.5 Internal coordinates and torsion tori . . . 71
11.6 Adjoint Sampling on tori . . . 72
11.7 Riemannian samplers . . . 73

12 Neural samplers in machine learning and generative modelling . . . **76**
Reward fine-tuning as tilting, guidance as an h-transform, Bayesian inference, energy-based models, variational inference, discrete spaces
12.1 Where unnormalised densities appear in ML . . . 76
12.2 Reward fine-tuning is sampling from a tilted density . . . 77
12.3 Guidance and inverse problems . . . 78
12.4 Bayesian inference . . . 79
12.5 Training energy-based models . . . 80
12.6 Variational inference and latent-variable models . . . 80
12.7 Discrete and structured spaces . . . 81

13 Application: molecular generation and equilibrium sampling . . . **82**
Molecular Boltzmann densities, representations, conformers, peptides, free energies, transition paths, proteins and materials
13.1 The target . . . 82
13.2 Choosing a representation . . . 82
13.3 Tasks . . . 82
13.4 A reference pipeline . . . 85

14 Open problems and a research roadmap . . . **87**
A unified view, the main open questions, a reading plan and starter projects
14.1 One template for all samplers . . . 87
14.2 The big open questions . . . 87
14.3 A reading plan . . . 89
14.4 Starter projects . . . 89

A Gaussian toolbox . . . **91**
Gaussian conditioning, Brownian and OU bridges

B Code companion . . . **93**
The scripts used in the book, how to run them, and what they output

Bibliography . . . **94**

Preface

Why this book. Diffusion and flow models have made us very good at learning a distribution from *samples*. Many of the next problems in machine learning are the reverse: the distribution is given by a *formula*, and there are no samples. A pretrained image or protein model should be steered towards high reward without collapsing; a diffusion prior should be conditioned on a measurement; a Bayesian posterior, an energy-based model or a molecule's Boltzmann distribution should be sampled. In each case we can evaluate an *unnormalised density*, and we want a generative model of it. Neural samplers are exactly this: *generative models trained from an energy instead of a data set*.

The book is written for readers who know basic probability and machine learning but are new to the field. It starts from simple pictures and discrete-time calculations, and only then moves to stochastic calculus. Every result comes with an explanation of what it means, and with its form in terms of probability densities.

The problem. We can evaluate an *energy* $\mathcal{E} : \mathbb{R}^d \to \mathbb{R}$ and its gradient $\nabla\mathcal{E}$, and we want samples from the density

$$\pi(x) = \frac{\tilde{\pi}(x)}{Z}, \qquad \tilde{\pi}(x) = e^{-\mathcal{E}(x)}, \qquad Z = \int_{\mathbb{R}^d} e^{-\mathcal{E}(x)}\,\mathrm{d}x, \tag{1}$$

together with an estimate of $\log Z$. We know $\tilde{\pi}$ (the *unnormalised* density) but not Z, and we have no samples from π. Low energy means high probability. In machine learning, $\mathcal{E}$ may be minus a log-posterior, or minus the log-density of a pretrained model plus a reward; in molecular science it is a potential energy divided by $k_B T$. In all cases π tends to concentrate on a few narrow, well-separated regions (*modes*), which is what makes the problem hard.

Table 1 translates between generative modelling and neural sampling; most ideas of the book are one row of it.

How the book is organised. Figure 1 shows the four parts. Read them in order; within a part, chapters build on each other.

- **Part I, foundations (Chapters 1 to 3).** Densities and how maps move them; diffusions, introduced first as Gaussian random walks; reweighting paths, optimal control and Schrödinger bridges. Three results carry the rest of the book: the KL divergence between two diffusions (Corollary 2.10), the fact that the optimal controlled process is a reweighted reference process (Theorem 3.2), and time reversal (Theorem 2.12).
- **Part II, the classical methods (Chapters 4 to 8).** Baselines and metrics; normalising flows (Boltzmann generators); diffusion samplers; annealing with importance weights; samplers that learn scores from energies.
- **Part III, control-based samplers (Chapters 9 and 10).** Adjoint Sampling and Schrödinger-bridge samplers, the current state of the art for scalable sampling.
- **Part IV, geometry and applications (Chapters 11 to 14).** Manifolds and symmetries; machine learning (reward fine-tuning, guidance, Bayesian inference, energy-based models); molecules; open problems.

Table 1 Generative modelling (samples given, density unknown) versus neural sampling (density given up to Z, no samples).

	Generative modelling	Neural sampling
Training signal	samples $x \sim p_{\text{data}}$	energy $\mathcal{E}$ and forces $-\nabla\mathcal{E}$
Natural objective	forward KL, i.e. maximum likelihood (covers all modes)	reverse KL, i.e. variational inference (tends to pick modes), Section 1.6
Lower bound	ELBO on $\log p_\theta(x)$ (VAEs)	ELBO on $\log Z$ (Proposition 1.10 and Theorem 6.1)
Diffusion training	denoising score matching	score estimated from forces (Proposition 8.1)
Flow matching	regress velocities on data pairs	regress endpoint forces on bridges (Chapter 9)
Schrödinger bridge	data at both ends	data at neither end (Chapter 10)
Guidance, fine-tuning	h-transforms, reward tilting	the same, as control problems (Chapter 12)
Typical failure	poor coverage of rare data	missing or misweighted modes

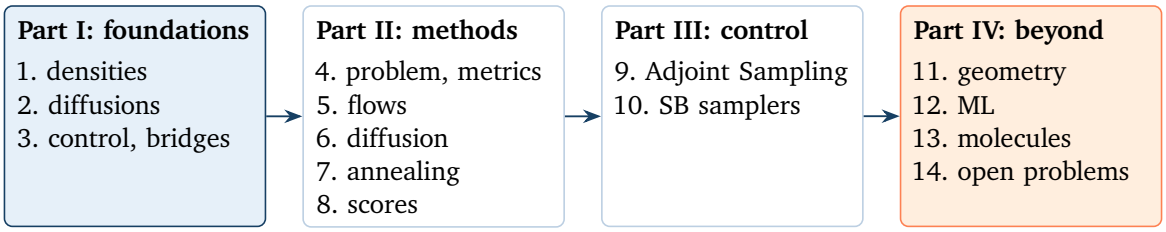


Figure 1 The four parts of the book. Each part uses the previous ones.

How each chapter is written. Each chapter opens with a card giving its goal, prerequisites and key papers, and an overview in plain words. Results are stated as lemmas, propositions or theorems. Each comes with an *Intuition* box and a proof; when a result is stated for general probability measures, its version for densities is given next to it. Boxes titled *Generative-modelling view* connect the chapter to diffusion models, flow matching, VAEs and reinforcement learning. Methods come with pseudo-code and, when useful, short PyTorch programs that have been run (Chapter B). Most chapters end with open problems.

Notation and conventions

Much confusion in this field comes from mixing up different objects attached to a random process $X = (X_t)_{t\in[0,1]}$. We keep them strictly apart (Figure 2).

Object	Notation	Meaning
Path law	$\mathbb{P}$, $\mathbb{Q}$, $\mathbb{P}^u$	a probability distribution over *whole trajectories* $X = (X_t)_{t\in[0,1]}$; blackboard letters are always path laws
Marginal density	$p_t(x)$, $q_t(x)$, $p_t^u(x)$	the density of the single random vector X_t under $\mathbb{P}$, $\mathbb{Q}$, $\mathbb{P}^u$
Joint density	$p_{s,t}(x,y)$	density of the pair (X_s, X_t); $p_{0,1}$ couples start and end
Transition density	$p_{t\mid s}(y \mid x)$	density of X_t at y given $X_s = x$
Conditional path law	$\mathbb{P}(\cdot \mid X_1 = y)$	law of the whole path given its endpoint (a bridge)

So $\mathbb{P}$ answers questions about whole paths ("how likely is this trajectory?"), and p_t answers questions about one time ("where is the particle at time t?"). They are related by $\mathbb{P}(X_t \in A) = \int_A p_t(x)\,\mathrm{d}x$ and $p_{s,t}(x,y) = p_s(x)\,p_{t|s}(y \mid x)$. In continuous time a path law has no density in the usual sense (there is no "Lebesgue measure on paths"), so two path laws are compared through their ratio $\frac{\mathrm{d}\mathbb{P}}{\mathrm{d}\mathbb{Q}}$ (Section 2.5). After discretising time, a path becomes a vector $x_{0:N} = (x_0, \dots, x_N)$ with an ordinary density $p(x_{0:N})$, and $\frac{\mathrm{d}\mathbb{P}}{\mathrm{d}\mathbb{Q}}$ is the limit of the density ratio $\frac{p(x_{0:N})}{q(x_{0:N})}$. We always give this discrete form too.

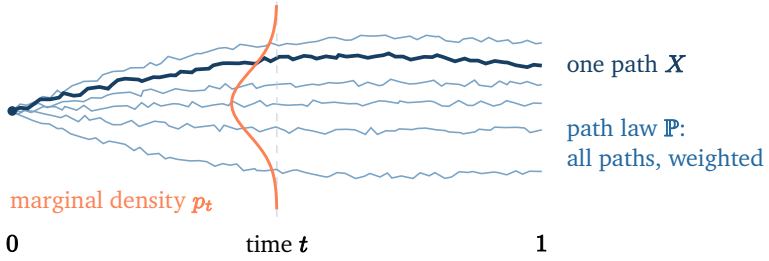


Figure 2 Path law versus marginal. $\mathbb{P}$ is a distribution over whole curves; p_t is the density of where the curves are at one time t (the slice).

Other conventions.

- The target density is $\pi = \frac{\tilde{\pi}}{Z}$ with $\tilde{\pi} = e^{-\mathcal{E}}$; the source (prior) density is μ, or the point mass δ_{x_0}.
- Time runs from $t = 0$ (source) to $t = 1$ (target). B_t is a standard Brownian motion (Wiener process) in $\mathbb{R}^d$.
- A sampler is the solution of $\mathrm{d}X_t = \big(b_t(X_t) + \sigma_t u_t(X_t)\big)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t$: b is a fixed base drift, u the learned *control*, $\sigma_t > 0$ a scalar noise level. Its path law is $\mathbb{P}^u$ and its marginal densities are p_t^u; $\mathbb{P}^0$ (with marginals p_t^0) is the base process. We write $\bar{\sigma}_{s,t}^2 = \int_s^t \sigma_r^2\,\mathrm{d}r$ and $\bar{\sigma}_t^2 = \bar{\sigma}_{0,t}^2$.
- $\mathbb{P}^\star$ and $u^\star$ denote the optimal path law and control. $\mathrm{KL}(\mathbb{P}\|\mathbb{Q})$ compares path laws; $\mathrm{KL}(p_1\|q_1)$ compares densities at one time.
- $\mathbb{E}_{\mathbb{P}}[F(X)]$ is an expectation over paths; $\mathbb{E}_{x\sim p}[f(x)] = \int f(x)p(x)\,\mathrm{d}x$ is an expectation over one random vector.
- $\nabla\cdot v$ is the divergence, Δ the Laplacian, and $\mathcal{L}_t f = b_t \cdot \nabla f + \frac{\sigma_t^2}{2}\Delta f$ the generator of the base process; $\mathrm{sg}(\cdot)$ is stop-gradient.
- All fractions are written $\frac{a}{b}$; $\mathcal{N}(x; m, s^2\mathrm{I})$ is the Gaussian density with mean m and covariance $s^2\mathrm{I}$ evaluated at x.

Assumption 0.1 (Standing regularity). Unless stated otherwise: drifts are continuously differentiable and globally Lipschitz in x; σ_t is continuous and bounded away from 0; marginal densities are smooth, strictly positive and decay fast enough at infinity for integration by parts; changes of measure (Girsanov) are valid; and $Z < \infty$. Under these conditions SDEs have unique solutions and the Fokker–Planck equation has a unique solution among probability densities (Øksendal, 2003; Karatzas and Shreve, 1991). These conditions hold in all our examples; a first reading can ignore them.

CHAPTER 1

Densities, transport and divergences

Goal	Understand what it means to sample from a density, how maps and velocity fields move densities around, how to measure the distance between two distributions, and why the direction of the KL divergence matters so much for samplers.
Prerequisites	Calculus in several variables; random variables and densities.
Key papers	Papamakarios et al. (2021); Owen (2013); Villani (2009).

Overview. Every sampler in this book takes easy random numbers (Gaussian noise) and transforms them so that the result is distributed according to π. This chapter answers three questions about such transformations: how the density changes under a map (Section 1.2) or a flow (Section 1.3); how far the result is from π (Section 1.4); and how to correct a sampler that is not exactly right (Section 1.5).

1.1. What does it mean to sample?

A random vector $X \in \mathbb{R}^d$ *has density* p if $\Pr(X \in A) = \int_A p(x)\,\mathrm{d}x$ for every region A. We write $X \sim p$. To *sample* from p is to produce realisations of such an X, independently, as many as we like.

In one dimension this is easy if we can compute the cumulative distribution function $F(x) = \int_{-\infty}^{x} p(y)\,\mathrm{d}y$ and its inverse: if U is uniform on $[0,1]$, then $X = F^{-1}(U)$ has density p. Two things break in the setting of this book. First, we only know $\tilde{\pi} = Z\pi$, so even F is unknown up to the constant Z; computing Z is itself an integral over $\mathbb{R}^d$. Second, in high dimension there is no analogue of F^{-1} that we can compute, and grids are useless: a grid with 10 points per coordinate has 10^d points. Any practical sampler must therefore *learn* a transformation from noise to π. The rest of this chapter is about how transformations act on densities.

Intuition

Think of a density as sand spread over $\mathbb{R}^d$, total mass 1. Sampling means dropping grains one at a time so that, in the long run, they pile up according to the density. A sampler is a machine that moves grains of a simple heap (a Gaussian) to the right places.

1.2. Moving densities with maps

If $Z \sim \mu$ and $T : \mathbb{R}^d \to \mathbb{R}^d$ is a map, the law of $X = T(Z)$ is called the *pushforward* of μ by T, written $T_{\#}\mu$. In terms of probabilities of sets, $\Pr(X \in A) = \Pr(Z \in T^{-1}(A))$; in terms of expectations, $\mathbb{E}[f(X)] = \mathbb{E}[f(T(Z))]$ for every function f. When T is smooth and invertible, the pushforward has an explicit density.

Theorem 1.1 (Change of variables). Let T be a smooth bijection with smooth inverse, and Z have density μ. Then $X = T(Z)$ has density

$$p_X(x) = \mu\big(T^{-1}(x)\big)\,\big|\det \nabla T^{-1}(x)\big|, \tag{1.1}$$

or equivalently, along a sample $x = T(z)$,

$$\log p_X(T(z)) = \log \mu(z) - \log\big|\det \nabla T(z)\big|.$$

Intuition

T moves mass without creating or destroying it. Where T stretches space ($|\det \nabla T| > 1$), the same mass is spread over more volume, so the density goes down by exactly the stretching factor. Example in one dimension: $X = 2Z$ with $Z \sim \mathcal{N}(0,1)$ stretches by 2, so $p_X(x) = \frac{1}{2}\mu(\frac{x}{2})$, which is the $\mathcal{N}(0,4)$ density.

Proof. For any bounded function f, substituting $z = T^{-1}(x)$, so that $\mathrm{d}z = |\det \nabla T^{-1}(x)|\,\mathrm{d}x$,

$$\mathbb{E}[f(X)] = \int f(T(z))\,\mu(z)\,\mathrm{d}z = \int f(x)\,\mu(T^{-1}(x))\,|\det \nabla T^{-1}(x)|\,\mathrm{d}x.$$

Since this holds for all f, the factor multiplying $f(x)$ is the density of X. The second form uses $\nabla T^{-1}(T(z)) = (\nabla T(z))^{-1}$ and $\det A^{-1} = \frac{1}{\det A}$. ∎

Corollary 1.2 (Composition). If $T = T_K \circ \cdots \circ T_1$ and $z_k = T_k(z_{k-1})$, $z_0 = z$, then $\log p_X(T(z)) = \log \mu(z) - \sum_{k=1}^{K} \log|\det \nabla T_k(z_{k-1})|$.

Proof. The chain rule gives $\nabla T(z) = \nabla T_K(z_{K-1}) \cdots \nabla T_1(z_0)$, and the determinant of a product is the product of determinants. ∎

Intuition

Stretching factors multiply along a chain of maps, so log-stretching factors add. This is why normalising flows are deep stacks of simple layers: each layer has a cheap log-determinant, and the total is their sum.

1.3. Moving densities with velocity fields

Instead of one big map, we can move points continuously in time along a velocity field $v_t(x)$: each point follows the ordinary differential equation $\frac{\mathrm{d}x_t}{\mathrm{d}t} = v_t(x_t)$. If X_0 is random with density p_0, then X_t is random with some density p_t. How does p_t evolve?

Theorem 1.3 (Continuity equation and Liouville formula). Let $X_0 \sim p_0$, let X_t solve $\frac{\mathrm{d}X_t}{\mathrm{d}t} = v_t(X_t)$, and let p_t be the density of X_t. Then

$$\boxed{\partial_t p_t(x) + \nabla\cdot\big(p_t(x)\,v_t(x)\big) = 0,} \tag{1.2}$$

and along each trajectory the log-density changes as

$$\frac{\mathrm{d}}{\mathrm{d}t}\log p_t(X_t) = -\big(\nabla\cdot v_t\big)(X_t). \tag{1.3}$$

Conversely, if a family of positive densities (p_t) solves Equation (1.2) for the field v and $X_0 \sim p_0$, then $X_t \sim p_t$ for all t.

Intuition

Equation (1.2) is conservation of mass: the density at a point decreases by the net amount of mass flowing out, $\nabla\cdot(p_t v_t)$. Equation (1.3) is the time-continuous version of Theorem 1.1: $\nabla\cdot v_t$ is the rate at which the flow stretches volume around the particle, and the density drops at that rate.

Proof. For a smooth test function φ with compact support, the chain rule gives $\frac{\mathrm{d}}{\mathrm{d}t}\,\mathbb{E}[\varphi(X_t)] = \mathbb{E}[\nabla\varphi(X_t)\cdot v_t(X_t)]$. Writing both sides with densities and integrating by parts on the right,

$$\int \varphi\,\partial_t p_t\,\mathrm{d}x = \int \nabla\varphi\cdot v_t\,p_t\,\mathrm{d}x = -\int \varphi\,\nabla\cdot(p_t v_t)\,\mathrm{d}x.$$

As φ is arbitrary, Equation (1.2) holds. For Equation (1.3), expand $\nabla\cdot(p_t v_t) = v_t\cdot\nabla p_t + p_t\nabla\cdot v_t$ and divide Equation (1.2) by p_t: $\partial_t \log p_t + v_t\cdot\nabla\log p_t = -\nabla\cdot v_t$. The left side is exactly $\frac{\mathrm{d}}{\mathrm{d}t}\log p_t(X_t)$ along a trajectory with velocity v_t. The converse holds because the linear equation Equation (1.2) has only one solution with a given initial density when v is Lipschitz (Ambrosio et al., 2008, Ch. 8). ∎

Corollary 1.4 (Likelihood of a continuous normalising flow). Integrating Equation (1.3): $\log p_1(X_1) = \log p_0(X_0) - \int_0^1 (\nabla\cdot v_t)(X_t)\,\mathrm{d}t$. The divergence can be estimated without bias with random projections (Proposition 5.1).

Intuition

Follow one particle and keep a running tally of how much its neighbourhood has been stretched; subtracting the tally from the starting log-density gives the final log-density, without ever computing a determinant.

1.4. Measuring the distance between distributions

Definition 1.5 (Kullback–Leibler divergence). For two distributions P and Q of the same random object, $\mathrm{KL}(P\|Q) = \mathbb{E}_P\left[\log\frac{\mathrm{d}P}{\mathrm{d}Q}\right]$, where $\frac{\mathrm{d}P}{\mathrm{d}Q}$ is the likelihood ratio (Radon–Nikodym derivative); $\mathrm{KL}(P\|Q) = +\infty$ if P gives mass to a set that Q does not. **Density form:** if P and Q have densities p and q, then $\frac{\mathrm{d}P}{\mathrm{d}Q}(x) = \frac{p(x)}{q(x)}$ and

$$\mathrm{KL}(p\|q) = \int p(x)\log\frac{p(x)}{q(x)}\,\mathrm{d}x.$$

The general form is needed because path laws have no densities (preface); for vectors, the density form is all we use.

Lemma 1.6 (Gibbs' inequality). $\mathrm{KL}(P\|Q)\geq 0$, with equality if and only if $P = Q$.

Proof. With densities: $-\mathrm{KL}(p\|q) = \int p\log\frac{q}{p} \leq \log\int p\,\frac{q}{p} = \log\int q = 0$ by Jensen's inequality for the concave function $\log$, with equality only if $\frac{q}{p}$ is constant, i.e. $p = q$. The general case is the same computation with $\frac{\mathrm{d}Q}{\mathrm{d}P}$ in place of $\frac{q}{p}$. ∎

Intuition

On average under P, P explains its own samples at least as well as any other model Q does. This single inequality is why every ELBO is a lower bound and why every KL-based loss in this book is minimised exactly at the target.

Lemma 1.7 (Chain rule). For two random objects (X, Y) with joint densities $p(x, y) = p(x)\, p(y \mid x)$ and $q(x, y) = q(x)\, q(y \mid x)$,

$$\mathrm{KL}\left(p(x,y) \,\|\, q(x,y)\right) = \mathrm{KL}\left(p(x) \,\|\, q(x)\right) + \mathbb{E}_{x\sim p}\left[\mathrm{KL}\left(p(\cdot \mid x) \,\|\, q(\cdot \mid x)\right)\right].$$

The same holds for general laws, with conditional laws in place of conditional densities.

Proof. Take logarithms in $\frac{p(x,y)}{q(x,y)} = \frac{p(x)}{q(x)} \cdot \frac{p(y|x)}{q(y|x)}$ and average over $(x, y) \sim p$; the second term is first averaged over $y \sim p(\cdot|x)$, then over x. ∎

Intuition

The divergence between two joint laws is "how different are the first coordinates" plus "on average, how different is what follows". Applied step by step along a time grid, this turns the KL between two *paths* into a sum of per-step discrepancies, the structure behind every loss in Chapters 6 and 9.

Corollary 1.8 (Data processing). For any function F, the laws of $F(X)$ under P and Q satisfy $\mathrm{KL}(F_{\#}P \| F_{\#}Q) \le \mathrm{KL}(P\|Q)$. In particular, for two processes, $\mathrm{KL}(p_1\|q_1) \le \mathrm{KL}(\mathbb{P}\|\mathbb{Q})$: the KL between the densities at time 1 is at most the KL between the path laws.

Proof. Apply Lemma 1.7 to the pair $(F(X), X)$: the first term is $\mathrm{KL}(F_{\#}P\|F_{\#}Q)$ and the second is non-negative. The pair carries the same information as X alone, so its KL is $\mathrm{KL}(P\|Q)$. ∎

Intuition

Throwing information away (here: forgetting the path and keeping only the endpoint) can only make two distributions harder to tell apart. So if the path of our sampler is close to a target path law, its endpoint is close to π. Path KLs are computable where marginal KLs are not, and this corollary is the licence to optimise them instead.

Remark 1.9 (Other divergences). Besides KL we will meet the χ^2 divergence $\chi^2(p\|q) = \int \frac{(p-q)^2}{q}\,\mathrm{d}x$, which governs importance sampling (Proposition 1.11), and the Rényi divergence of order 2, $D_2(p\|q) = \log \int \frac{p^2}{q}\,\mathrm{d}x = \log(1 + \chi^2(p\|q))$, minimised by FAB (Section 5.4).

1.5. Importance sampling and the normalising constant

Suppose we have a *proposal* density q that we can sample and evaluate, but which is not exactly π. Importance sampling corrects it by giving each sample $X^i \sim q$ the weight

$$w(X^i) = \frac{\tilde{\pi}(X^i)}{q(X^i)} = Z\,\frac{\pi(X^i)}{q(X^i)}.$$

Proposition 1.10 (Unbiased Z, ELBO and EUBO). (i) $\mathbb{E}_{x\sim q}[w(x)] = Z$, so $\hat{Z}_N = \frac{1}{N}\sum_{i=1}^N w(X^i)$ is an unbiased estimate of Z.
(ii) $\mathbb{E}_{x\sim q}[\log w(x)] = \log Z - \mathrm{KL}(q\|\pi) \le \log Z$ (the *evidence lower bound*, ELBO).
(iii) $\mathbb{E}_{x\sim \pi}[\log w(x)] = \log Z + \mathrm{KL}(\pi\|q) \ge \log Z$ (the *evidence upper bound*, EUBO).
(iv) $\mathbb{E}[\log \hat{Z}_N] \le \log Z$, and $\mathbb{E}[\log \hat{Z}_N]$ increases with N.

Intuition

Averaging weights gives Z exactly; averaging *log*-weights loses exactly the KL gap. A sampler is good when its log-weights are nearly constant, and then both gaps close at once.

Proof. (i) $\int q\,\frac{\tilde\pi}{q}\,\mathrm{d}x = \int \tilde\pi\,\mathrm{d}x = Z$. (ii) and (iii): $\log w = \log Z + \log\frac{\pi}{q}$; average under q and under π respectively. (iv) The first claim is Jensen's inequality applied to (i). For the second, $\hat{Z}_{N+1}$ is the average of the $N+1$ estimates $\hat{Z}_N^{(-j)}$ that leave out one sample each; concavity of $\log$ gives $\log \hat{Z}_{N+1} \ge \frac{1}{N+1}\sum_j \log \hat{Z}_N^{(-j)}$, and each term has the same law as $\log \hat{Z}_N$. ∎

Proposition 1.11 (Effective sample size). Define $\mathrm{ESS} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2} \in [1, N]$. As $N\to\infty$, $\frac{\mathrm{ESS}}{N} \to \frac{1}{1+\chi^2(\pi\|q)}$.

Proof. By the law of large numbers, $\frac{1}{N}\sum_i w_i \to Z$ and $\frac{1}{N}\sum_i w_i^2 \to \mathbb{E}_q[w^2]$. Then $\frac{\mathbb{E}_q[w^2]}{Z^2} = \int \frac{\pi^2}{q}\,\mathrm{d}x = 1+\chi^2(\pi\|q)$. ∎

Intuition

The ESS answers: "how many exact samples from π are my N weighted samples worth?" If one weight dominates, $\mathrm{ESS}\approx 1$ however large N is. The cost of correcting a sampler is set by the χ^2 divergence, not the KL.

To estimate an expectation $\mathbb{E}_\pi[f]$ we use the *self-normalised* estimator $\frac{\sum_i w_i f(X^i)}{\sum_i w_i}$, which does not need Z; it is consistent, with a bias of order $\frac{1}{N}$. A warning for high dimension: for product densities, $1+\chi^2$ is multiplied across coordinates, $1+\chi^2(\pi^{\otimes d}\|q^{\otimes d}) = (1+\chi^2(\pi\|q))^d$. A small mismatch per coordinate therefore becomes an exponentially small ESS. This is the main reason why correcting an imperfect sampler is hard for large systems.

1.6. Forward versus reverse KL

Without samples from π, the natural training objective is the *reverse* KL, which only needs samples from the model q_θ:

$$\mathrm{KL}(q_\theta\|\pi) = \mathbb{E}_{x\sim q_\theta}\left[\log q_\theta(x) + \mathcal{E}(x)\right] + \log Z.$$

The constant $\log Z$ does not affect training. The *forward* KL $\mathrm{KL}(\pi\|q_\theta)$ needs samples from π. The two behave very differently.

Proposition 1.12 (Forward KL matches moments). Among Gaussian densities $q = \mathcal{N}(m,\Sigma)$, $\mathrm{KL}(\pi\|q)$ is minimised by the mean $m = \mathbb{E}_\pi[X]$ and the covariance $\Sigma = \mathrm{Cov}_\pi[X]$ of π.

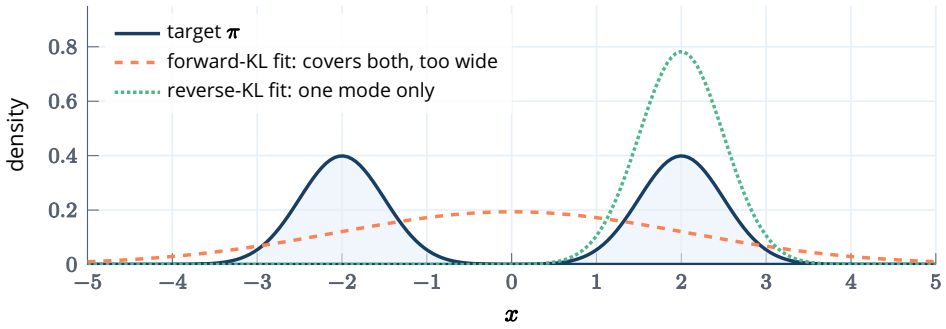


Figure 1.1 Gaussian fits to a two-mode target. The forward KL matches mean and variance (Proposition 1.12) and puts mass between the modes; the reverse KL selects one mode.

Proof. Up to terms that do not depend on (m, Σ), $\mathrm{KL}(\pi\|q) = \frac{1}{2}\,\mathbb{E}_\pi[(X-m)^\top\Sigma^{-1}(X-m)] + \frac{1}{2}\log\det\Sigma$. Setting the derivatives with respect to m and Σ^{-1} to zero gives $m = \mathbb{E}_\pi X$ and $\Sigma = \mathbb{E}_\pi[(X-m)(X-m)^\top]$. ∎

Intuition

The forward KL punishes the model heavily wherever π has mass and the model does not. A unimodal model therefore stretches to cover everything, including the empty space between modes.

The reverse KL behaves the other way round. $\mathbb{E}_q[\log q - \log\pi]$ is huge if q puts mass where $\pi \approx 0$, but it is only mildly increased if q ignores a mode of π altogether. Its minimisers over unimodal families therefore lock onto one mode (Figure 1.1). This *mode collapse* is the central failure of samplers trained by reverse KL, and much of the progress described in this book is about avoiding it without target samples.

Generative-modelling view

Every generative model is a pushforward of noise. Flows make the map invertible so that Theorem 1.1 gives exact likelihoods; diffusion and flow-matching models do not, but their deterministic (ODE) sampler gives one through Corollary 1.4. Training from data minimises the *forward* KL, which is maximum likelihood and covers all modes; training from an energy minimises the *reverse* KL, which is the objective of a VAE encoder or of variational inference, and tends to pick modes. Importance weights play for samplers the role that exact likelihoods play for density models: they turn an approximate model into an asymptotically exact procedure.

Looking ahead

- A sampler is a transport whose output density should equal π. When that density is computable, Proposition 1.10 turns the sampler into a $\log Z$ estimator and an importance sampler.
- Path-space samplers replace the intractable output density by the probability of a whole trajectory; data processing (Corollary 1.8) guarantees that a small path KL implies a small KL at the endpoint.

- The reverse KL is cheap but tends to collapse onto modes.

CHAPTER 2

Diffusions: from random walks to SDEs

Goal	Build stochastic differential equations from Gaussian random walks, so that path laws, marginal densities and transition densities are concrete objects. Then prove the five facts that neural samplers rely on: how densities evolve (Fokker–Planck), how to compare two diffusions (Girsanov), Feynman–Kac, time reversal, and conditioning (Doob's h-transform), and see how to build an SDE that carries one distribution to another.
Prerequisites	Chapter 1.
Key papers	Øksendal (2003); Anderson (1982); Haussmann and Pardoux (1986); Song et al. (2021b); Doob (1957).

Overview. A diffusion is a random walk with infinitely many, infinitely small Gaussian steps. We first study the walk with small but finite steps, where everything has an ordinary density and every formula is a Gaussian computation (Section 2.1). Each continuous-time result of the chapter is then the limit of a discrete one, and we state both.

2.1. A Gaussian random walk

Fix a time grid $0 = t_0 < t_1 < \cdots < t_N = 1$ with steps $h_n = t_{n+1} - t_n$, a *drift* $b_t(x) \in \mathbb{R}^d$ (the deterministic push) and a *noise level* $\sigma_t > 0$. The walk starts at $X_0 \sim p_0$ and moves by

$$X_{n+1} = X_n + h_n\, b_{t_n}(X_n) + \sigma_{t_n}\sqrt{h_n}\,\xi_n, \qquad \xi_n \sim \mathcal{N}(0, \mathrm{I}) \text{ independent.} \tag{2.1}$$

This is the *Euler–Maruyama* scheme; it is also exactly how SDEs are simulated in practice. All three objects of the preface are explicit:

- **Transition density.** Given $X_n = x$, the next point is Gaussian: $p_{n+1|n}(y \mid x) = \mathcal{N}\big(y;\, x + h_n b_{t_n}(x),\, \sigma_{t_n}^2 h_n \mathrm{I}\big)$.
- **Path density.** The whole discrete path $x_{0:N}$ has the density

$$p(x_{0:N}) = p_0(x_0) \prod_{n=0}^{N-1} \mathcal{N}\big(x_{n+1};\, x_n + h_n b_{t_n}(x_n),\, \sigma_{t_n}^2 h_n \mathrm{I}\big). \tag{2.2}$$

- **Marginal density.** The density p_n of X_n alone is obtained by integrating the path density over all the other points; equivalently, $p_{n+1}(y) = \int p_n(x)\, p_{n+1|n}(y \mid x)\, \mathrm{d}x$.

The "variance $\propto$ time" scaling $\sigma^2 h$ is essential: with $b = 0$, $X_N - X_0$ is a sum of independent Gaussians with total variance $\sum_n \sigma_{t_n}^2 h_n \approx \int_0^1 \sigma_t^2\, \mathrm{d}t$, which stays finite as the steps shrink. With any other scaling the walk would either freeze or explode in the limit.

Intuition

The path density Equation (2.2) is a product of Gaussian bumps, one per step, each centred where the drift would push the particle. A path is likely if each step lands near its predicted position. Almost everything in this chapter (comparing two diffusions, reweighting paths, reversing time) is a computation with this product.

2.2. Brownian motion and stochastic differential equations

As the grid gets finer, the walk with $b = 0$ and $\sigma = 1$ converges to *Brownian motion* B_t: a random continuous path with $B_0 = 0$ and independent Gaussian increments $B_t - B_s \sim \mathcal{N}(0, (t-s)\mathrm{I})$. The walk Equation (2.1) converges to the solution of the *stochastic differential equation* (SDE)

$$\mathrm{d}X_t = b_t(X_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t, \qquad X_0 \sim p_0, \tag{2.3}$$

which is shorthand for $X_t = X_0 + \int_0^t b_s(X_s)\,\mathrm{d}s + \int_0^t \sigma_s\,\mathrm{d}B_s$. We write $\mathbb{P}$ for the law of the whole solution path, p_t for the density of X_t, and $p_{t|s}(y|x)$ for its transition density. Under Assumption 0.1 the solution exists and is unique (Øksendal, 2003).

The increments of B over a short time h have size $\sqrt{h}$, much larger than h. This is why ordinary calculus must be corrected.

Theorem 2.1 (Itô's formula). For a smooth function $f(t,x)$ and X solving Equation (2.3),

$$\mathrm{d}f(t, X_t) = \Big(\partial_t f + b_t \cdot \nabla f + \frac{\sigma_t^2}{2}\Delta f\Big)(t, X_t)\,\mathrm{d}t + \sigma_t \nabla f(t, X_t) \cdot \mathrm{d}B_t.$$

Intuition

Expand f to second order over one step. The second-order term $\frac{1}{2}\Delta X^\top \nabla^2 f\, \Delta X$ contains $\frac{\sigma^2}{2}\xi^\top \nabla^2 f\, \xi\, h$, whose average is $\frac{\sigma^2}{2}\Delta f\, h$: because the noise is of size $\sqrt{h}$, its square is of size h and survives in the limit. This extra term is the whole difference between Itô calculus and ordinary calculus.

Proof sketch. Sum the second-order Taylor expansions of f over the steps of Equation (2.1). First-order terms converge to the $\mathrm{d}t$ and $\mathrm{d}B$ integrals. In the second-order term, only $\frac{\sigma^2}{2} h\, \xi_n^\top \nabla^2 f\, \xi_n$ survives, and the sum of these random terms converges to $\int \frac{\sigma^2}{2}\Delta f\,\mathrm{d}t$ because $\xi_n \xi_n^\top$ averages to I (law of large numbers). Higher-order terms vanish. See Øksendal (2003, Thm. 4.2.1). ∎

2.3. How densities evolve

Definition 2.2 (Generator). The generator of Equation (2.3) is the operator $\mathcal{L}_t f = b_t \cdot \nabla f + \frac{\sigma_t^2}{2}\Delta f$; by Itô's formula, $\frac{\mathrm{d}}{\mathrm{d}t}\,\mathbb{E}[f(X_t)] = \mathbb{E}[\mathcal{L}_t f(X_t)]$. Its adjoint, which acts on densities, is $\mathcal{L}_t^* p = -\nabla\cdot(b_t p) + \frac{\sigma_t^2}{2}\Delta p$.

Theorem 2.3 (Fokker–Planck equation). The marginal densities of Equation (2.3) satisfy

$$\partial_t p_t(x) = -\nabla\cdot\big(b_t(x)\, p_t(x)\big) + \frac{\sigma_t^2}{2}\Delta p_t(x). \tag{2.4}$$

Intuition

The drift moves mass (the continuity equation of Theorem 1.3); the noise spreads it like heat. Writing $\frac{\sigma^2}{2}\Delta p = \nabla\cdot\left(\frac{\sigma^2}{2} p \nabla \log p\right)$ shows that the noise acts like an extra velocity $-\frac{\sigma^2}{2}\nabla \log p$, pushing mass from high to low density.

Proof. For a smooth test function φ with compact support, Itô's formula and the fact that the $\mathrm{d}B$ term has mean zero give $\frac{\mathrm{d}}{\mathrm{d}t}\int \varphi\, p_t \,\mathrm{d}x = \int (\mathcal{L}_t\varphi)\, p_t \,\mathrm{d}x$. Integrating by parts (once for the drift term, twice for the Laplacian) moves the derivatives onto p_t: $\int (\mathcal{L}_t\varphi) p_t = \int \varphi\, \mathcal{L}_t^* p_t$. As φ is arbitrary, $\partial_t p_t = \mathcal{L}_t^* p_t$. ∎

The same equation holds for transition densities: $y \mapsto p_{t|s}(y \mid x)$ solves Equation (2.4) in (t, y) with initial condition a point mass at x. The dual statement, about expectations of the future, is the following.

Theorem 2.4 (Backward Kolmogorov equation). For a function g, let $h(t,x) = \mathbb{E}[g(X_1) \mid X_t = x] = \int g(y)\, p_{1|t}(y \mid x)\,\mathrm{d}y$. Then $\partial_t h + \mathcal{L}_t h = 0$ and $h(1,\cdot) = g$.

Intuition

$h(t,x)$ is a forecast: the expected payoff at time 1 given the state now. A forecast cannot be expected to change (otherwise you would already have revised it), and "zero expected change" written with Itô's formula is exactly $\partial_t h + \mathcal{L}_t h = 0$. Value functions in control and the h-functions of guidance are forecasts of this kind.

Proof. By the Markov property, $h(t, X_t) = \mathbb{E}[g(X_1) \mid X_s, s \le t]$, a quantity whose expected future change is zero (a martingale). By Itô's formula, $\mathrm{d}h(t,X_t) = (\partial_t h + \mathcal{L}_t h)\,\mathrm{d}t + \sigma_t \nabla h \cdot \mathrm{d}B_t$. The $\mathrm{d}B$ part has zero mean, so the $\mathrm{d}t$ part must vanish; since X_t can be anywhere (positive density), $\partial_t h + \mathcal{L}_t h = 0$ everywhere. ∎

2.4. Langevin dynamics, Ornstein–Uhlenbeck and metastability

Proposition 2.5 (Langevin dynamics leaves π invariant). If $X_0 \sim \pi \propto e^{-\mathcal{E}}$ and $\mathrm{d}X_t = -\nabla\mathcal{E}(X_t)\,\mathrm{d}t + \sqrt{2}\,\mathrm{d}B_t$, then $X_t \sim \pi$ for all t.

Proof. With $b = -\nabla\mathcal{E} = \nabla \log \pi$ and $\sigma^2 = 2$, the right side of Equation (2.4) at $p = \pi$ is $-\nabla\cdot(\pi \nabla \log \pi) + \Delta\pi = -\nabla\cdot(\nabla\pi) + \Delta\pi = 0$, so π does not change. ∎

Intuition

The drift pushes mass downhill in energy; the noise spreads it out, which acts like an uphill push. The two cancel exactly at $p \propto e^{-\mathcal{E}}$: Langevin dynamics is gradient descent prevented from collapsing onto the minimum.

Theorem 2.6 (Convergence of Langevin dynamics). Suppose π satisfies the log-Sobolev inequality $\mathrm{KL}(p\|\pi) \le \frac{1}{2\lambda} I(p\|\pi)$ for all densities p, where $I(p\|\pi) = \int p \left|\nabla \log \frac{p}{\pi}\right|^2 \mathrm{d}x$ is the relative Fisher information. Then the Langevin marginals satisfy $\mathrm{KL}(p_t\|\pi) \le e^{-2\lambda t}\,\mathrm{KL}(p_0\|\pi)$.

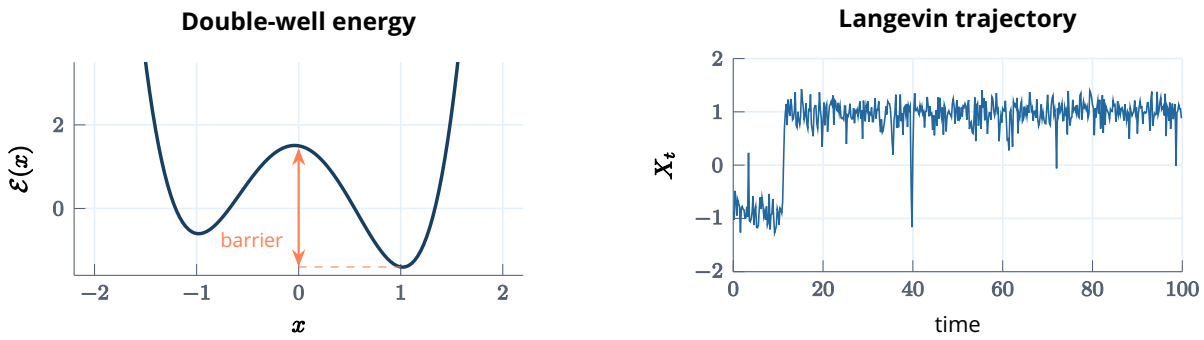


Figure 2.1 Metastability. Langevin dynamics on a double well stays in one basin for long periods. The mean time between switches grows like $e^{\Delta\mathcal{E}}$ with the barrier height $\Delta\mathcal{E}$, so the constant λ of Theorem 2.6 is exponentially small.

Proof. For Langevin dynamics Equation (2.4) reads $\partial_t p_t = \nabla\cdot\big(p_t \nabla \log \frac{p_t}{\pi}\big)$. Differentiating the KL and integrating by parts,

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\mathrm{KL}(p_t\|\pi) = \int \partial_t p_t \,\log\frac{p_t}{\pi}\,\mathrm{d}x = -\int p_t \left|\nabla \log\frac{p_t}{\pi}\right|^2 \mathrm{d}x = -I(p_t\|\pi) \le -2\lambda\,\mathrm{KL}(p_t\|\pi),$$

and integrating this differential inequality gives the claim. ∎

Intuition

KL decreases at a rate equal to the Fisher information, the strength of the "force" pulling p_t towards π. The log-Sobolev inequality says this force cannot be weak while p_t is still far from π. It fails, in the sense that λ becomes tiny, when π has separated modes: mass can sit in the wrong mode and feel almost no force.

Theorem 2.6 is reassuring and useless at the same time: for multimodal π the constant λ is of order $e^{-\Delta\mathcal{E}}$ (Bovier et al., 2004) (Figure 2.1). Neural samplers try to replace this slow *equilibrium* dynamics by a learned *non-equilibrium* transport that reaches π in a fixed, finite time.

Proposition 2.7 (Ornstein–Uhlenbeck process). The solution of $\mathrm{d}X_t = -\frac{\beta_t}{2}X_t\,\mathrm{d}t + \sqrt{\beta_t}\,\mathrm{d}B_t$ has transition density $p_{t|0}(y \mid x) = \mathcal{N}\big(y;\, \alpha_t x,\, (1-\alpha_t^2)\mathrm{I}\big)$ with $\alpha_t = \exp\big(-\frac{1}{2}\int_0^t \beta_s\,\mathrm{d}s\big)$. In particular $\mathcal{N}(0,\mathrm{I})$ is invariant.

Proof. By Itô's formula, $Y_t = \frac{X_t}{\alpha_t}$ satisfies $\mathrm{d}Y_t = \frac{\sqrt{\beta_t}}{\alpha_t}\,\mathrm{d}B_t$, so $X_t = \alpha_t X_0 + \alpha_t\int_0^t \frac{\sqrt{\beta_s}}{\alpha_s}\,\mathrm{d}B_s$. The integral is Gaussian with variance $\alpha_t^2\int_0^t \frac{\beta_s}{\alpha_s^2}\,\mathrm{d}s = \alpha_t^2\big(\frac{1}{\alpha_t^2}-1\big) = 1-\alpha_t^2$, since $\frac{\mathrm{d}}{\mathrm{d}s}\frac{1}{\alpha_s^2} = \frac{\beta_s}{\alpha_s^2}$. ∎

Intuition

The OU process forgets its starting point exponentially fast ($\alpha_t \to 0$) and relaxes to $\mathcal{N}(0,\mathrm{I})$. This is the "noising" process of DDPM-type diffusion models: after a long enough time, any data point has become indistinguishable from Gaussian noise.

2.5. Comparing two diffusions: the Girsanov theorem

Consider two diffusions with the *same* noise and the *same* initial density but different drifts,

$$\mathbb{P}^1:\ \mathrm{d}X_t = b_t^1(X_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t, \qquad \mathbb{P}^2:\ \mathrm{d}X_t = b_t^2(X_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t, \qquad X_0 \sim p_0.$$

How much more likely is a given path under $\mathbb{P}^1$ than under $\mathbb{P}^2$? In discrete time we can just divide the path densities Equation (2.2).

Proposition 2.8 (Discrete path-density ratio). For the Euler walks of $\mathbb{P}^1$ and $\mathbb{P}^2$ on the same grid, with $\Delta x_n = x_{n+1} - x_n$ and $\sigma_n = \sigma_{t_n}$,

$$\log \frac{p^1(x_{0:N})}{p^2(x_{0:N})} = \sum_{n=0}^{N-1} \Big[\frac{(b^1 - b^2)(x_n) \cdot \Delta x_n}{\sigma_n^2} - \frac{|b^1(x_n)|^2 - |b^2(x_n)|^2}{2\sigma_n^2} h_n \Big], \tag{2.5}$$

and the KL between the two discrete path laws is exactly

$$\mathrm{KL}(p^1 \| p^2) = \mathbb{E}_{p^1} \Big[\sum_{n=0}^{N-1} \frac{|b^1(X_n) - b^2(X_n)|^2}{2\sigma_n^2} h_n \Big]. \tag{2.6}$$

Proof. The initial densities cancel. For each step, the two transition densities are Gaussians with the same covariance $\sigma_n^2 h_n \mathbf{I}$ and means $x_n + h_n b^i(x_n)$, so the log-ratio is $\frac{|\Delta x_n - h_n b^2|^2 - |\Delta x_n - h_n b^1|^2}{2\sigma_n^2 h_n}$; expanding the squares gives Equation (2.5). For the KL, apply the chain rule (Lemma 1.7) step by step: the KL between two Gaussians with the same covariance $s^2\mathbf{I}$ and means m_1, m_2 is $\frac{|m_1 - m_2|^2}{2s^2}$ (Lemma A.5), here $\frac{h_n^2 |b^1 - b^2|^2}{2\sigma_n^2 h_n}$. ∎

Intuition

Paths are compared step by step: at each step, which of the two drifts better predicts where the particle actually went? A drift difference $b^1 - b^2$ costs $\frac{|b^1 - b^2|^2}{2\sigma^2}$ per unit of time. More noise makes drift differences harder to detect (cheaper in KL); with no noise, two different drifts produce paths that can be told apart with certainty, and the KL is infinite.

As the grid is refined, both formulas have limits. The sums become integrals, and the ratio of path densities, which have no limit individually, converges to the ratio of path laws $\frac{\mathrm{d}\mathbb{P}^1}{\mathrm{d}\mathbb{P}^2}$.

Theorem 2.9 (Girsanov). The path laws $\mathbb{P}^1, \mathbb{P}^2$ satisfy, for $\mathbb{P}^2$-almost every path X,

$$\log \frac{\mathrm{d}\mathbb{P}^1}{\mathrm{d}\mathbb{P}^2}(X) = \int_0^1 \frac{(b_t^1 - b_t^2)(X_t)}{\sigma_t^2} \cdot \mathrm{d}X_t - \int_0^1 \frac{|b_t^1(X_t)|^2 - |b_t^2(X_t)|^2}{2\sigma_t^2} \, \mathrm{d}t. \tag{2.7}$$

Proof sketch. Equation (2.5) is a Riemann–Itô sum of Equation (2.7). The rigorous statement is proved by showing that, under $\mathbb{P}^2$ reweighted by the exponential of Equation (2.7), the process $\mathrm{d}X_t - b_t^1 \, \mathrm{d}t$ is σ_t times a Brownian motion (Lévy's characterisation), so the reweighted law is $\mathbb{P}^1$; see Øksendal (2003, Thm. 8.6.4). ∎

Corollary 2.10 (KL between diffusions). If $\mathbb{P}^1$ and $\mathbb{P}^2$ start from densities p_0^1 and p_0^2, then

$$\mathrm{KL}(\mathbb{P}^1 \| \mathbb{P}^2) = \mathrm{KL}(p_0^1 \| p_0^2) + \mathbb{E}_{\mathbb{P}^1} \Big[\int_0^1 \frac{|b_t^1(X_t) - b_t^2(X_t)|^2}{2\sigma_t^2} \, \mathrm{d}t \Big]. \tag{2.8}$$

In particular, for a controlled SDE $\mathrm{d}X_t = (b_t + \sigma_t u_t) \, \mathrm{d}t + \sigma_t \, \mathrm{d}B_t$ and the base process $\mathbb{P}^0$ (same start), $\mathrm{KL}(\mathbb{P}^u \| \mathbb{P}^0) = \mathbb{E}_{\mathbb{P}^u} \big[\int_0^1 \frac{1}{2} |u_t(X_t)|^2 \, \mathrm{d}t \big]$.

Proof. The initial term comes from Lemma 1.7. For the rest, take the limit of Equation (2.6); alternatively, substitute $\mathrm{d}X_t = b_t^1\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t$ (valid under $\mathbb{P}^1$) into Equation (2.7) and take expectations, using that the $\mathrm{d}B$ integral has zero mean. ∎

Note what is and is not compared: Equation (2.8) is a statement about *path laws*. By data processing, the KL between the time-1 densities is smaller: $\mathrm{KL}(p_1^1\|p_1^2) \le \mathrm{KL}(\mathbb{P}^1\|\mathbb{P}^2)$.

2.6. The Feynman–Kac formula

Theorem 2.11 (Feynman–Kac). For a function $c_t(x)$ bounded below and a function g,

$$h(t,x) = \mathbb{E}\left[e^{-\int_t^1 c_s(X_s)\,\mathrm{d}s}\, g(X_1)\,\middle|\, X_t = x\right]$$

solves $\partial_t h + \mathcal{L}_t h - c_t h = 0$ with $h(1,\cdot) = g$.

Intuition

Paths are "killed" at rate c; h is the expected reward of the paths that survive. The same idea in the forward direction, particles carrying weights $e^{-\int c}$, is the basis of the Jarzynski weights of Chapter 7.

Proof. $M_t = e^{-\int_0^t c_s(X_s)\,\mathrm{d}s} h(t,X_t)$ is the conditional expectation of a fixed random variable, hence has zero expected change. By Itô's formula and the product rule, $\mathrm{d}M_t = e^{-\int_0^t c}(\partial_t h + \mathcal{L}_t h - c_t h)\,\mathrm{d}t +$ (zero-mean term), so the bracket vanishes, as in Theorem 2.4. ∎

2.7. Time reversal

Theorem 2.12 (Time reversal of diffusions). Let X solve Equation (2.3) with marginal densities p_t. Then the reversed process $Y_s = X_{1-s}$ is again a diffusion with the same noise level, started from p_1:

$$\mathrm{d}Y_s = \Big(-b_{1-s}(Y_s) + \sigma_{1-s}^2 \nabla \log p_{1-s}(Y_s)\Big)\,\mathrm{d}s + \sigma_{1-s}\,\mathrm{d}\bar{B}_s, \qquad Y_0 \sim p_1, \tag{2.9}$$

for a Brownian motion $\bar{B}$. The path law of Y is that of X read backwards. **Density form:** its marginal densities are p_{1-s}, and its transition densities are the backward conditionals of X, $p_{s|t}(x \mid y) = \frac{p_s(x)\,p_{t|s}(y|x)}{p_t(y)}$ for $s < t$.

Intuition

Running the film backwards reverses the drift, but noise does not reverse: forward noise spreads mass out, so the backward dynamics must actively gather it back, by climbing the density along $\sigma^2 \nabla \log p_t$. This *score* correction is what diffusion models learn from data, and what diffusion samplers must learn without data.

Proof. *Marginals.* Let $\hat{b}_s = -b_{1-s} + \sigma_{1-s}^2 \nabla \log p_{1-s}$ and $\hat{p}_s = p_{1-s}$. Using Equation (2.4),

$$\partial_s \hat{p}_s = -(\partial_t p_t)_{t=1-s} = \nabla\cdot(bp) - \frac{\sigma^2}{2}\Delta p = -\nabla\cdot(\hat{b}\,\hat{p}) + \frac{\sigma^2}{2}\Delta\hat{p},$$

because $-\nabla\cdot(\hat{b}\hat{p}) = \nabla\cdot(bp) - \sigma^2\nabla\cdot(\nabla p) = \nabla\cdot(bp) - \sigma^2\Delta p$. So $\hat{p}$ solves the Fokker–Planck equation of Equation (2.9), and by uniqueness the reversed SDE has marginals p_{1-s}.

Transitions. Fix t and y, and let $k(r,x) = p_r(x)\,p_{t|r}(y \mid x)$ for $r < t$, which is proportional (in x) to the backward conditional density of X_r given $X_t = y$. Using the Fokker–Planck equation for p_r and the backward equation (Theorem 2.4) for $p_{t|r}(y|\cdot)$,

$$\partial_r k = -\nabla\cdot(bk) + \frac{\sigma^2}{2}\big(q\Delta p - p\Delta q\big), \qquad q = p_{t|r}(y \mid \cdot),\ p = p_r.$$

Now $q\Delta p - p\Delta q = \nabla\cdot(q\nabla p - p\nabla q) = 2\nabla\cdot(k\nabla\log p) - \Delta k$, so in reversed time $s = 1 - r$ the function k solves the Fokker–Planck equation of Equation (2.9) started at a point mass at y. Hence the transition densities of Y are the backward conditionals of X. A Markov process is determined by its initial density and its transition densities, and the reversal of a Markov process is Markov, so the two path laws agree. For the pathwise construction of $\bar{B}$ see Haussmann and Pardoux (1986). ∎

2.8. Building SDEs that carry one distribution to another

How do we construct an SDE whose marginal density starts at μ and ends at π? All answers rest on one fact.

Theorem 2.13 (Marginal-preserving family). Let $(p_t)_{t\in[0,1]}$ be a smooth path of positive densities and v_t a velocity field with $\partial_t p_t + \nabla\cdot(p_t v_t) = 0$. For every noise level $\sigma_t \geq 0$, the SDE

$$\mathrm{d}X_t = \Big(v_t(X_t) + \frac{\sigma_t^2}{2}\nabla\log p_t(X_t)\Big)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t, \qquad X_0 \sim p_0, \tag{2.10}$$

has marginal densities p_t for all t. With $\sigma \equiv 0$ it is the ODE $\frac{\mathrm{d}x}{\mathrm{d}t} = v_t(x)$ (the *probability flow*).

Proof. The right side of Equation (2.4) for this drift, evaluated at p_t, is $-\nabla\cdot(p_t v_t) - \frac{\sigma_t^2}{2}\nabla\cdot(\nabla p_t) + \frac{\sigma_t^2}{2}\Delta p_t = -\nabla\cdot(p_t v_t) = \partial_t p_t$. So p_t solves the Fokker–Planck equation, and uniqueness identifies it with the marginals. ∎

Intuition

Noise at rate σ^2 spreads mass with effective velocity $-\frac{\sigma^2}{2}\nabla\log p$; adding the drift $+\frac{\sigma^2}{2}\nabla\log p$ cancels it on average. So noise can be traded for drift freely without changing any marginal density. What changes are the *path law* and the *coupling* density $p_{0,1}(x_0, x_1)$ between start and end, which is why the choice of noise matters for samplers even though it does not matter for marginals.

There are four classical ways to obtain such an SDE (Table 2.1).

1. **Noise, then reverse (diffusion models).** Run a noising SDE from π, e.g. the OU process of Proposition 2.7, so that its time-1 density is close to $\mathcal{N}(0, \mathrm{I})$; then reverse it with Theorem 2.12. The reverse drift needs the scores $\nabla\log p_t$ of the noised target (Chapters 6 and 8).
2. **Prescribed interpolation (annealing).** Fix a path of densities, e.g. $p_t \propto \mu^{1-t}\pi^t$, and find a velocity v_t solving the continuity equation. With $\frac{\sigma^2}{2} = \varepsilon$, Equation (2.10) is Langevin dynamics plus a transport correction (Chapter 7).
3. **Conditioning (Doob's h-transform).** Take a reference SDE and condition it on where it ends; the result is again an SDE (Theorem 2.14).

4. **Optimisation (Schrödinger bridge).** Among all path laws with time-0 density μ and time-1 density π, choose the one closest in KL to a reference process (Chapter 3).

Theorem 2.14 (Doob's h-transform). Let $\mathbb{P}^0$ be the law of $\mathrm{d}X_t = b_t\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t$, $X_0 \sim p_0$, with transition densities $p^0_{t|s}$. Let $g > 0$ be a function of the endpoint and $h(t,x) = \mathbb{E}_{\mathbb{P}^0}[g(X_1) \mid X_t = x]$. Define the reweighted path law

$$\frac{\mathrm{d}\mathbb{P}^h}{\mathrm{d}\mathbb{P}^0}(X) = \frac{g(X_1)}{h(0,X_0)}. \tag{2.11}$$

Then $\mathbb{P}^h$ has initial density p_0 and is the law of the SDE

$$\boxed{\mathrm{d}X_t = \Big(b_t(X_t) + \sigma_t^2 \nabla \log h(t,X_t)\Big)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t.} \tag{2.12}$$

Density form: its transition densities are $p^h_{t|s}(y \mid x) = p^0_{t|s}(y \mid x)\,\frac{h(t,y)}{h(s,x)}$.

Intuition

Reweighting whole paths by a function of their endpoint is the same as adding, at every time, a drift towards the regions from which highly weighted endpoints are likely: $\nabla \log h$ points to where the "expected future reward" g is large. The density form says the same thing step by step: a transition is made more likely in proportion to how much it improves the forecast h.

Proof. *Density form.* $\int p^0_{t|s}(y|x)h(t,y)\,\mathrm{d}y = h(s,x)$ by the Markov property, so $p^h_{t|s}$ is a valid transition density, and multiplying these ratios along a path telescopes to $\frac{h(1,X_1)}{h(0,X_0)} = \frac{g(X_1)}{h(0,X_0)}$, which is Equation (2.11). Since $\mathbb{E}_{\mathbb{P}^0}[g(X_1) \mid X_0] = h(0,X_0)$, the initial density is unchanged.

SDE form. By Theorem 2.4, $\partial_t h + \mathcal{L}_t h = 0$. Itô's formula for $\log h$ gives

$$\begin{aligned}\mathrm{d}\log h(t,X_t) &= \frac{\partial_t h + \mathcal{L}_t h}{h}\,\mathrm{d}t - \frac{\sigma_t^2}{2}|\nabla \log h|^2\,\mathrm{d}t + \sigma_t \nabla \log h \cdot \mathrm{d}B_t \\ &= -\frac{1}{2}|\sigma_t \nabla \log h|^2\,\mathrm{d}t + \sigma_t \nabla \log h \cdot \mathrm{d}B_t.\end{aligned}$$

Integrating from 0 to 1, $\log \frac{g(X_1)}{h(0,X_0)} = \int_0^1 \sigma_t \nabla \log h \cdot \mathrm{d}B_t - \frac{1}{2}\int_0^1 |\sigma_t \nabla \log h|^2\,\mathrm{d}t$, which is the Girsanov log-ratio (Theorem 2.9, written with $\mathrm{d}X_t - b_t\,\mathrm{d}t = \sigma_t\,\mathrm{d}B_t$) for the drift change $\sigma_t^2 \nabla \log h$. ∎

Example 2.15 (Brownian bridge). Take $b = 0$, $\sigma = 1$, $X_0 = x_0$, and condition on $X_1 = y$ (formally g is a point mass at y). Then $h(t,x) = \mathcal{N}(y; x, (1-t)\mathrm{I})$, $\nabla \log h(t,x) = \frac{y-x}{1-t}$, and the bridge solves $\mathrm{d}X_t = \frac{y - X_t}{1-t}\,\mathrm{d}t + \mathrm{d}B_t$: the drift pulls harder and harder towards y as $t \to 1$. Its marginal density is $\mathcal{N}\big(x_0 + t(y - x_0),\, t(1-t)\mathrm{I}\big)$; Lemma A.2 gives the version with a time-dependent σ_t.

Generative-modelling view

This chapter contains the theory of diffusion models. "Noise, then reverse" with a learned score is the DDPM and score-SDE recipe (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021b); the score is learned by denoising score matching (Vincent, 2011). The probability-flow ODE of Theorem 2.13 is the deterministic DDIM or flow-matching sampler

Table 2.1 Four ways to build an SDE from μ to π, and the one unknown each must learn.

Construction	Drift	Unknown to be learned
Noise, then reverse	$-b + \sigma^2 \nabla \log p_t$	scores of the noised target
Annealing + transport	$\frac{\sigma^2}{2} \nabla \log p_t + v_t$	velocity v_t solving the continuity equation
h-transform	$b + \sigma^2 \nabla \log h_t$	the forecast h_t (a value function)
Schrödinger bridge	$b + \sigma^2 \nabla \log \varphi_t$	the potentials $(\varphi, \hat{\varphi})$ of Theorem 3.12

(Song et al., 2021a), and the theorem explains why stochastic and deterministic samplers have the same marginal densities. Corollary 2.10 is the likelihood bound of diffusion models: the true and learned reverse SDEs have drifts differing by $\sigma_t^2(\nabla \log p_t - s_\theta)$, so the KL between their path laws is $\mathbb{E} \int \frac{\sigma_t^2}{2} |\nabla \log p_t - s_\theta|^2 \, \mathrm{d}t$ (plus the prior mismatch), a weighted score-matching loss that bounds the KL of the generated density by data processing. Doob's h-transform is conditional generation: classifier guidance is Theorem 2.14 with g the likelihood of a label (Proposition 12.3).

Looking ahead

- Discretised, a diffusion is a Gaussian random walk with an explicit path density; every result of this chapter is the limit of a Gaussian computation.
- Corollary 2.10 turns "distance between two samplers" into "squared difference of their drifts", the basis of every regression loss in this book.
- Theorems 2.12 to 2.14 are three ways to write down an exact sampler. Each has one intractable ingredient (a score, an h-function or a velocity), which a neural network will learn.

CHAPTER 3

Reweighting paths: optimal control and Schrödinger bridges

Goal	See that "sample from π" can be solved by *reweighting the paths* of a simple random process, and that the drift which generates the reweighted paths is the solution of an optimal control problem. Then extend the idea to Schrödinger bridges, which connect two given distributions. These are the theoretical engines of Chapters 6, 9 and 10.
Prerequisites	Chapter 2.
Key papers	Kappen (2005); Dai Pra (1991); Léonard (2014); Chen et al. (2021); Domingo-Enrich et al. (2025).

Overview. Take a simple random process, for example Brownian motion started at the origin, and give each of its paths a weight that depends on where the path ends. The reweighted collection of paths is a new random process. Two facts make this useful: (i) with the right weight, the new process ends exactly at π; (ii) the new process is again a diffusion, whose extra drift is the solution of an optimal control problem and can be learned. Section 3.1 shows both facts in discrete time with elementary calculations; the rest of the chapter is their continuous-time version and its generalisation to Schrödinger bridges.

3.1. Warm-up: reweighting a random walk

Consider a Markov chain $X_0, \dots, X_N$ with initial density q_0 and transition densities $q_{n+1|n}(y \mid x)$, so that its path density is $q(x_{0:N}) = q_0(x_0) \prod_n q_{n+1|n}(x_{n+1} \mid x_n)$. Give each path the weight $g(x_N) > 0$ that depends only on its endpoint, and normalise:

$$p(x_{0:N}) = \frac{q(x_{0:N})\, g(x_N)}{\mathcal{Z}}, \qquad \mathcal{Z} = \int q(x_{0:N})\, g(x_N)\, \mathrm{d}x_{0:N}. \tag{3.1}$$

Proposition 3.1 (A reweighted Markov chain is a Markov chain). Let $h_N = g$ and $h_n(x) = \int q_{n+1|n}(y \mid x)\, h_{n+1}(y)\, \mathrm{d}y$, so that $h_n(x) = \mathbb{E}_q[g(X_N) \mid X_n = x]$. Then p is the path density of a Markov chain with

$$p_0(x) = \frac{q_0(x)\, h_0(x)}{\mathcal{Z}}, \qquad p_{n+1|n}(y \mid x) = q_{n+1|n}(y \mid x)\, \frac{h_{n+1}(y)}{h_n(x)}.$$

Its final density is $p_N(y) = \frac{1}{\mathcal{Z}}\, g(y) \int q_0(x)\, q_{N|0}(y \mid x)\, \mathrm{d}x = \frac{q_N(y)\, g(y)}{\mathcal{Z}}$.

Proof. Each $p_{n+1|n}(\cdot|x)$ integrates to 1 by the definition of h_n. Multiplying the claimed initial density and transitions, the ratios $\frac{h_{n+1}(x_{n+1})}{h_n(x_n)}$ telescope to $\frac{h_N(x_N)}{h_0(x_0)}$, so the product equals $\frac{q_0(x_0)h_0(x_0)}{\mathcal{Z}} \prod_n q_{n+1|n} \cdot \frac{g(x_N)}{h_0(x_0)} = p(x_{0:N})$. Integrating out all points but the last gives p_N. ∎

Intuition

To generate the reweighted paths one step at a time, make each step more likely in proportion to how much it improves the *forecast* h_{n+1} of the final weight. The forecast h_n is computed backwards from the end. Two remarks drive the rest of the chapter. First, if the chain starts at a fixed point x_0, the initial density cannot change, and the final density is simply $q_N \cdot g$, normalised: choosing $g = \frac{\tilde{\pi}}{q_N}$ gives $p_N = \pi$ exactly. Second, if the start is random, the reweighting also changes the initial density (by h_0), which a sampler with a fixed source cannot do.

For a Gaussian random walk, multiplying a Gaussian transition by $\frac{h_{n+1}(y)}{h_n(x)}$ shifts its mean by approximately $\sigma^2 h \nabla \log h_{n+1}(x)$ for small steps. In continuous time this becomes exactly Doob's h-transform (Theorem 2.14): an extra drift $\sigma_t^2 \nabla \log h$. The rest of the chapter makes this precise and connects it to control.

3.2. The control problem and its Gibbs solution

Fix a base diffusion $\mathbb{P}^0$: $\mathrm{d}X_t = b_t(X_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t$. A *control* $u_t(x)$ adds a drift,

$$\mathrm{d}X_t = \big(b_t(X_t) + \sigma_t u_t(X_t)\big)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t, \qquad X_0 = x, \tag{3.2}$$

with path law $\mathbb{P}^u_x$, and is charged the cost

$$J(u;x) = \mathbb{E}_{\mathbb{P}^u_x}\left[\int_0^1 \left(\frac{1}{2}|u_t(X_t)|^2 + c_t(X_t)\right)\mathrm{d}t + g(X_1)\right], \tag{3.3}$$

where c is a running cost and g a terminal cost. The quadratic control cost is not arbitrary: by Corollary 2.10 its expectation is exactly $\mathrm{KL}(\mathbb{P}^u_x \| \mathbb{P}^0_x)$. So J balances "stay close to the base process" against "reach low cost".

Theorem 3.2 (Gibbs variational principle for control). Let $\mathcal{Z}(x) = \mathbb{E}_{\mathbb{P}^0_x}\left[e^{-\int_0^1 c_t(X_t)\,\mathrm{d}t - g(X_1)}\right]$ and define the reweighted path law

$$\frac{\mathrm{d}\mathbb{P}^\star_x}{\mathrm{d}\mathbb{P}^0_x}(X) = \frac{1}{\mathcal{Z}(x)}\exp\Big(-\int_0^1 c_t(X_t)\,\mathrm{d}t - g(X_1)\Big). \tag{3.4}$$

Then for every control u,

$$\boxed{J(u;x) = \mathrm{KL}(\mathbb{P}^u_x \,\|\, \mathbb{P}^\star_x) - \log \mathcal{Z}(x).} \tag{3.5}$$

Hence the minimal cost is $-\log \mathcal{Z}(x)$, attained exactly by the control whose path law is $\mathbb{P}^\star_x$ (Corollary 3.3). **Density form:** in discrete time, the optimal path density is $p^\star(x_{0:N}) = \frac{1}{\mathcal{Z}}\,p^0(x_{0:N})\,e^{-\sum_n c(x_n)h_n - g(x_N)}$, as in Equation (3.1).

Intuition

The optimal controlled process does not have to be searched for: it is the base process with each path reweighted by $e^{-\text{cost}}$, the path-space analogue of the Boltzmann distribution. Control is only needed to *generate* those reweighted paths forward in time, without reweighting.

Proof. Since $\frac{\mathrm{d}\mathbb{P}^u_x}{\mathrm{d}\mathbb{P}^\star_x} = \frac{\mathrm{d}\mathbb{P}^u_x}{\mathrm{d}\mathbb{P}^0_x} \cdot \frac{\mathrm{d}\mathbb{P}^0_x}{\mathrm{d}\mathbb{P}^\star_x}$,

$$\log \frac{\mathrm{d}\mathbb{P}^u_x}{\mathrm{d}\mathbb{P}^\star_x} = \log \frac{\mathrm{d}\mathbb{P}^u_x}{\mathrm{d}\mathbb{P}^0_x} + \int_0^1 c_t(X_t)\,\mathrm{d}t + g(X_1) + \log \mathcal{Z}(x).$$

Take the expectation under $\mathbb{P}^u_x$ and use $\mathrm{KL}(\mathbb{P}^u_x \| \mathbb{P}^0_x) = \mathbb{E}_{\mathbb{P}^u_x} \int_0^1 \frac{1}{2}|u_t|^2\,\mathrm{d}t$ (Corollary 2.10): the right side becomes $J(u;x) + \log \mathcal{Z}(x)$. The minimum follows from Lemma 1.6. ∎

Corollary 3.3 (Optimal control, Hopf–Cole and HJB). Let $h(t,x) = \mathbb{E}_{\mathbb{P}^0}\left[e^{-\int_t^1 c_s(X_s)\,\mathrm{d}s - g(X_1)} \mid X_t = x\right]$ and $V = -\log h$. Then:

1. $\mathbb{P}^\star_x$ is the law of Equation (3.2) with the control $u^\star_t(x) = \sigma_t \nabla \log h(t,x) = -\sigma_t \nabla V(t,x)$;
2. $V(t,x)$ is the optimal cost-to-go from (t,x) (the *value function*) and solves the Hamilton–Jacobi–Bellman equation

$$\partial_t V + b_t \cdot \nabla V + \frac{\sigma_t^2}{2}\Delta V - \frac{\sigma_t^2}{2}|\nabla V|^2 + c_t = 0, \qquad V(1,\cdot) = g. \tag{3.6}$$

Intuition

Reweighting paths by the exponentiated future cost is the same as steering, at every moment, up the gradient of the log-forecast $\log h$ of a good future. The change of variables $V = -\log h$ (Hopf–Cole) turns the *linear* equation satisfied by h into the *nonlinear* HJB equation for the cost-to-go: the hard nonlinear control problem is secretly linear.

Proof. By Feynman–Kac (Theorem 2.11), $\partial_t h + \mathcal{L}_t h - c_t h = 0$. As in the proof of Theorem 2.14, Itô's formula gives

$$-\int_0^1 c_t\,\mathrm{d}t - g(X_1) - \log h(0,X_0) = \int_0^1 \sigma_t \nabla \log h \cdot \mathrm{d}B_t - \frac{1}{2}\int_0^1 |\sigma_t \nabla \log h|^2\,\mathrm{d}t$$

under $\mathbb{P}^0$, which is the Girsanov log-ratio for the extra drift $\sigma_t \cdot \sigma_t \nabla \log h$. Since $h(0,x) = \mathcal{Z}(x)$, this proves (1). Applying Theorem 3.2 on $[t,1]$ shows that V is the cost-to-go. Substituting $h = e^{-V}$ into the Feynman–Kac equation, with $\nabla h = -h\nabla V$ and $\Delta h = h(|\nabla V|^2 - \Delta V)$, gives Equation (3.6). ∎

3.3. Sampling as a control problem

Theorem 3.4 (Terminal-cost design). Let the base process start at a fixed point x_0 and have time-1 density $p^0_1 > 0$. Take $c \equiv 0$ and

$$g(x) = \log \frac{p^0_1(x)}{\tilde{\pi}(x)} = \mathcal{E}(x) + \log p^0_1(x). \tag{3.7}$$

Then the optimally controlled process ends at the target: its time-1 density is $p^\star_1 = \pi$. The minimal cost is $-\log Z$, and for every control, $-J(u;x_0) = \log Z - \mathrm{KL}(\mathbb{P}^u \| \mathbb{P}^\star) \le \log Z$.

Proof. By Equation (3.4), $\mathbb{P}^\star$ reweights $\mathbb{P}^0$ by $\frac{\tilde{\pi}(X_1)}{p^0_1(X_1)\,\mathcal{Z}}$, with $\mathcal{Z} = \int p^0_1 \frac{\tilde{\pi}}{p^0_1}\,\mathrm{d}x = Z$. Reweighting by a function of X_1 multiplies the time-1 density by that function, so $p^\star_1 = p^0_1 \cdot \frac{\tilde{\pi}}{p^0_1 Z} = \pi$. The rest is Equation (3.5). ∎

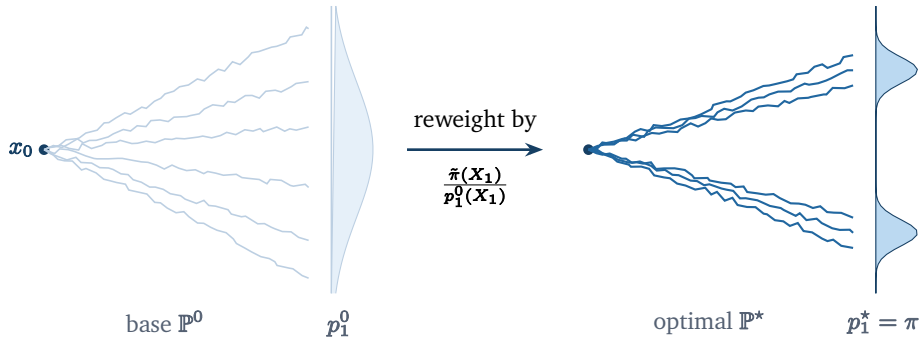


Figure 3.1 Theorem 3.4: the optimal controlled path law is the base path law reweighted at its endpoint. Because all paths start at the same point, the endpoint weight is the only weight, and the final density is exactly π.

Intuition

The terminal cost undoes the base process's own endpoint density and installs π instead: dividing by p_1^0 removes "where the base would have gone anyway", multiplying by $\tilde{\pi}$ installs "where we want to go". Because every path starts at the same point, nothing at the start needs fixing.

Corollary 3.5 (Föllmer drift and the Path Integral Sampler). With $b = 0$, $\sigma \equiv 1$ and $x_0 = 0$ (so $p_1^0 = \mathcal{N}(0, \mathrm{I})$), the optimal control is

$$u_t^\star(x) = \nabla \log \mathbb{E}_{\xi \sim \mathcal{N}(0,\mathrm{I})} \left[\frac{\tilde{\pi}\left(x + \sqrt{1-t}\,\xi\right)}{\mathcal{N}\left(x + \sqrt{1-t}\,\xi;\, 0, \mathrm{I}\right)} \right]. \tag{3.8}$$

The resulting process is the Föllmer process; learning $u^\star$ by minimising $J(u)$ is the Path Integral Sampler (Föllmer, 1985; Zhang and Chen, 2022).

Proof. $u^\star = \nabla \log h$ with $h(t,x) = \mathbb{E}[e^{-g(X_1)} \mid X_t = x]$, and given $X_t = x$ the base endpoint is $X_1 = x + \sqrt{1-t}\,\xi$. ∎

Intuition

At time t and position x the remaining randomness is a Gaussian blob of variance $1 - t$ around x. The optimal drift points towards where, within that blob, the target is most over-represented compared with the Gaussian. Early on the blob is wide and the drift averages over the whole target; near $t = 1$ it is local.

Why start at a point? If $X_0 \sim \mu$ is random, the control cannot change the initial density, so the optimal path law is $\mathbb{P}^\star = \mu(X_0)\, \mathbb{P}^0(\cdot \mid X_0)\, \frac{e^{-g(X_1)}}{h(0,X_0)}$ and its final density is

$$p_1^\star(y) = e^{-g(y)} \int \mu(x)\, \frac{p_{1|0}^0(y \mid x)}{h(0,x)}\, \mathrm{d}x. \tag{3.9}$$

With g as in Equation (3.7) this equals π only if $h(0,x)$ does not depend on x. This *initial value-function bias* is the continuous version of the second remark in Section 3.1. It disappears under a structural condition.

Definition 3.6 (Memoryless base process). The base process is *memoryless* if X_0 and X_1 are independent under $\mathbb{P}^0$, i.e. $p^0_{1|0}(y \mid x) = p^0_1(y)$ for all x.

Proposition 3.7. If the base process is memoryless, $X_0 \sim \mu$ and $g = \log \frac{p^0_1}{\tilde{\pi}}$, then $p^\star_1 = \pi$.

Proof. $h(0,x) = \int p^0_{1|0}(y|x)\, e^{-g(y)}\, \mathrm{d}y = \int p^0_1 \frac{\tilde{\pi}}{p^0_1}\, \mathrm{d}y = Z$ for every x, and Equation (3.9) gives $p^\star_1 = \frac{\tilde{\pi}}{p^0_1} \cdot \frac{p^0_1}{Z} = \pi$. ∎

Intuition

If the start of the base process carries no information about its end, reweighting the end creates no pressure to change the start. Otherwise, starting points from which good endpoints are easy to reach would deserve more weight, which a sampler with a fixed source cannot give them.

A point mass is trivially memoryless; so is a process whose noise is so large that X_1 forgets X_0 (approximately). When neither holds, we need Schrödinger bridges (Section 3.5) or an explicit correction of the source (Section 10.2).

3.4. The adjoint representation of the optimal control

Learning $u^\star$ by minimising $J(u)$ directly requires differentiating through simulated trajectories. The next result instead writes $u^\star$ as an average of a quantity computed *along each path*, which turns control into regression.

Theorem 3.8 (Lean adjoint representation). Assume b, c, g are smooth with bounded derivatives. For a path X, let $a(t) = a(t; X)$ solve the ODE backwards in time

$$\frac{\mathrm{d}a(t)}{\mathrm{d}t} = -\nabla b_t(X_t)^\top a(t) - \nabla c_t(X_t), \qquad a(1) = \nabla g(X_1). \tag{3.10}$$

Then $\nabla V(t,x) = \mathbb{E}_{\mathbb{P}^\star}[a(t;X) \mid X_t = x]$. Hence

$$u^\star_t(x) = -\sigma_t\, \mathbb{E}_{\mathbb{P}^\star}\big[a(t;X) \mid X_t = x\big], \qquad u^\star = \arg\min_u\ \mathbb{E}_{\mathbb{P}^\star}\Big[\int_0^1 \big|u_t(X_t) + \sigma_t a(t;X)\big|^2\, \mathrm{d}t\Big]. \tag{3.11}$$

Intuition

$a(t)$ measures how much the future cost of *this* path would change if we nudged its position at time t. Averaging that sensitivity over all optimal paths through x gives the gradient of the cost-to-go, i.e. the direction in which the control should push. Because the answer is a conditional average, the control can be learned by least-squares regression.

Proof. Let $X_s^{t,x}$ $(s \geq t)$ be the base path started at $X_t = x$. Since the noise does not depend on the state, its sensitivity $J_{t,s} = \frac{\partial X_s^{t,x}}{\partial x}$ solves $\partial_s J_{t,s} = \nabla b_s(X_s) J_{t,s}$, $J_{t,t} = \mathrm{I}$, along the path. With $C = \int_t^1 c_s(X_s)\,\mathrm{d}s + g(X_1)$ and $h(t,x) = \mathbb{E}[e^{-C(X^{t,x})}]$,

$$\nabla V(t,x) = -\frac{\nabla h(t,x)}{h(t,x)} = \frac{\mathbb{E}\left[e^{-C}\left(\int_t^1 J_{t,s}^\top \nabla c_s(X_s)\,\mathrm{d}s + J_{t,1}^\top \nabla g(X_1)\right)\right]}{\mathbb{E}[e^{-C}]}.$$

The ratio of expectations is an expectation under the base paths from (t,x) reweighted by e^{-C}, which by Equation (3.4) is $\mathbb{P}^\star(\cdot \mid X_t = x)$. The bracket solves Equation (3.10): it equals $\nabla g(X_1)$ at $t = 1$, and differentiating in t using $\partial_t J_{t,s} = -J_{t,s} \nabla b_t(X_t)$ gives the ODE. The regression statement holds because least squares is minimised by the conditional expectation. ∎

Corollary 3.9 (Constant adjoint). If $b \equiv 0$ and $c \equiv 0$, then $a(t; X) = \nabla g(X_1)$ for all t, and $u_t^\star(x) = -\sigma_t\, \mathbb{E}_{\mathbb{P}^\star}[\nabla g(X_1) \mid X_t = x]$.

Intuition

Without a base drift, a nudge at time t is carried unchanged to time 1, so the sensitivity of the final cost to the position at any time is just the final gradient. The optimal control is the average, over optimal paths through x, of the force that the target exerts on their endpoints.

Equation (3.11) is a regression under the unknown $\mathbb{P}^\star$. Adjoint Matching (Domingo-Enrich et al., 2025) replaces $\mathbb{P}^\star$ by the current model $\mathbb{P}^{\bar{u}}$, $\bar{u} = \mathrm{sg}(u)$; Adjoint Sampling (Chapter 9) combines Corollary 3.9 with an exact description of $\mathbb{P}^\star$ given its endpoint.

3.5. Schrödinger bridges

So far the source was fixed (often a point) and only the endpoint was reweighted. A Schrödinger bridge connects two given densities.

Definition 3.10 (Schrödinger bridge problem). Given a reference path law $\mathbb{Q}$ (here a diffusion, with marginal densities q_t, transition densities $q_{t|s}$ and joint density $q_{0,1}$) and two densities μ, π, find

$$\mathbb{P}^\star = \arg\min \left\{ \mathrm{KL}(\mathbb{P} \| \mathbb{Q}) :\ p_0 = \mu,\ p_1 = \pi \right\}, \tag{3.12}$$

where p_0, p_1 are the time-0 and time-1 densities of $\mathbb{P}$.

Intuition

Among all random processes that start at μ and end at π, choose the one the reference dynamics would find least surprising. When $\mathbb{Q}$ is Brownian motion with a small variance, the bridge approaches the optimal-transport interpolation between μ and π (Léonard, 2014).

Proposition 3.11 (Only the endpoints are chosen). Let $\mathbb{Q}^{xy} = \mathbb{Q}(\cdot \mid X_0 = x, X_1 = y)$ be the reference bridge. Then $\mathbb{P}^\star$ is obtained by drawing (X_0, X_1) from the coupling density $p_{0,1}^\star$ that solves the *static* problem

$$p_{0,1}^\star = \arg\min \left\{ \mathrm{KL}(p_{0,1} \| q_{0,1}) :\ \int p_{0,1}(x,y)\,\mathrm{d}y = \mu(x),\ \int p_{0,1}(x,y)\,\mathrm{d}x = \pi(y) \right\},$$

and joining the endpoints with reference bridges $\mathbb{Q}^{xy}$.

Proof. By the chain rule (Lemma 1.7) applied to the endpoints and the rest of the path, $\mathrm{KL}(\mathbb{P}\|\mathbb{Q}) = \mathrm{KL}(p_{0,1}\|q_{0,1}) + \mathbb{E}_{(x,y)\sim p_{0,1}}\left[\mathrm{KL}(\mathbb{P}^{xy}\|\mathbb{Q}^{xy})\right]$. The constraints involve only $p_{0,1}$, and the second term is minimised (to zero) by $\mathbb{P}^{xy} = \mathbb{Q}^{xy}$. ∎

Intuition

Only the pairing of starting points with endpoints is really chosen. Once paired, the cheapest way to travel between them is the way the reference would have travelled anyway. The problem over paths reduces to a problem over couplings, an entropy-regularised optimal transport problem.

A path law whose bridges are those of $\mathbb{Q}$ is said to be in the *reciprocal class* of $\mathbb{Q}$. The solution has a product structure.

Theorem 3.12 (Schrödinger system). Suppose there are positive functions f, g such that the coupling density

$$p^\star_{0,1}(x,y) = f(x)\, q_{0,1}(x,y)\, g(y)$$

has marginals μ and π. Then it is the unique solution of the static problem, and $\frac{\mathrm{d}\mathbb{P}^\star}{\mathrm{d}\mathbb{Q}}(X) = f(X_0)\, g(X_1)$. Define the *Schrödinger potentials*

$$\varphi_t(x) = \mathbb{E}_{\mathbb{Q}}[g(X_1) \mid X_t = x], \qquad \hat\varphi_t(x) = \int f(y)\, q_0(y)\, q_{t|0}(x \mid y)\, \mathrm{d}y. \tag{3.13}$$

Then (i) the marginal densities of $\mathbb{P}^\star$ factorise, $p^\star_t = \varphi_t\, \hat\varphi_t$, in particular $\mu = \varphi_0\hat\varphi_0$ and $\pi = \varphi_1\hat\varphi_1$; (ii) $\mathbb{P}^\star$ is the law of the SDE with drift $b_t + \sigma_t^2 \nabla \log \varphi_t$ started at μ; (iii) its time reversal has drift $-b_t + \sigma_t^2 \nabla \log \hat\varphi_t$ and starts at π.

Intuition

The optimal process is the reference process reweighted by one factor at each end: f corrects the start, g the end, and nothing in between. The forward potential φ_t carries g backwards in time ("how good is the future from here?"); the backward potential $\hat\varphi_t$ carries f forwards ("how much mass arrives here?"). Their product is the density, exactly as in the forward–backward algorithm for hidden Markov models.

Proof. *Optimality.* For any coupling density $p_{0,1}$ with marginals μ, π,

$$\begin{aligned}\mathrm{KL}(p_{0,1}\|q_{0,1}) &= \int p_{0,1} \log \frac{p_{0,1}}{p^\star_{0,1}} + \int p_{0,1}(x,y)\big(\log f(x) + \log g(y)\big)\,\mathrm{d}x\,\mathrm{d}y \\ &= \mathrm{KL}(p_{0,1}\|p^\star_{0,1}) + \int \mu \log f\,\mathrm{d}x + \int \pi \log g\,\mathrm{d}y.\end{aligned}$$

The last two terms are the same for all admissible couplings, so $p^\star_{0,1}$ is the unique minimiser by Lemma 1.6; Proposition 3.11 gives $\frac{\mathrm{d}\mathbb{P}^\star}{\mathrm{d}\mathbb{Q}} = f(X_0)g(X_1)$. (i) By the Markov property of $\mathbb{Q}$, $p^\star_t(x) = \int q_0(y) f(y)\, q_{t|0}(x|y)\,\mathrm{d}y \cdot \mathbb{E}_{\mathbb{Q}}[g(X_1) \mid X_t = x] = \hat\varphi_t(x)\varphi_t(x)$. (ii) Given X_0, $\mathbb{P}^\star(\cdot \mid X_0)$ is $\mathbb{Q}(\cdot \mid X_0)$ reweighted by $\frac{g(X_1)}{\varphi_0(X_0)}$, which is Doob's h-transform (Theorem 2.14) with $h = \varphi$. (iii) By Theorem 2.12, the reverse drift is $-(b + \sigma^2\nabla\log\varphi) + \sigma^2\nabla\log(\varphi\hat\varphi) = -b + \sigma^2\nabla\log\hat\varphi$. ∎

Existence of (f, g) holds under mild conditions (Rüschendorf and Thomsen, 1993; Léonard, 2014). The decisive consequence for sampling is that the bridge is itself a control problem with a terminal cost.

Theorem 3.13 (Schrödinger bridge as a control problem). $\mathbb{P}^\star$ solves the control problem Equations (3.2) and (3.3) with $X_0 \sim \mu$, $c \equiv 0$ and terminal cost

$$g_{\mathrm{SB}}(x) = -\log \varphi_1(x) = \log \frac{\hat{\varphi}_1(x)}{\pi(x)} = \log \hat{\varphi}_1(x) + \mathcal{E}(x) + \log Z. \tag{3.14}$$

Proof. For each starting point x_0, Theorem 3.2 says the optimal path law reweights $\mathbb{Q}(\cdot|x_0)$ by $\frac{\varphi_1(X_1)}{\mathbb{E}_{\mathbb{Q}}[\varphi_1(X_1)|x_0]} = \frac{g(X_1)}{\varphi_0(x_0)}$ (note $\varphi_1 = g$), which is $\mathbb{P}^\star(\cdot|x_0)$ by Theorem 3.12(ii). Both start at μ. The second form uses $\pi = \varphi_1\hat{\varphi}_1$. ∎

Intuition

A Schrödinger bridge is a control problem whose terminal cost is "energy, minus a discount for points that the source reaches easily anyway". The discount $\log \hat{\varphi}_1$ is exactly what removes the initial value-function bias of Equation (3.9).

Proposition 3.14 (Corrector identity). $\nabla \log \hat{\varphi}_1(y) = \mathbb{E}_{\mathbb{P}^\star}\left[\nabla_y \log q_{1|0}(y \mid X_0) \,\middle|\, X_1 = y\right]$.

Proof. By Theorem 3.12, the density of X_0 given $X_1 = y$ under $\mathbb{P}^\star$ is

$$p^\star_{0|1}(x \mid y) = \frac{f(x)\, q_0(x)\, q_{1|0}(y \mid x)\, g(y)}{p^\star_1(y)} = \frac{f(x)\, q_0(x)\, q_{1|0}(y \mid x)}{\hat{\varphi}_1(y)}.$$

Differentiating Equation (3.13) under the integral,

$$\frac{\nabla \hat{\varphi}_1(y)}{\hat{\varphi}_1(y)} = \int \nabla_y \log q_{1|0}(y \mid x)\, \frac{f(x)\, q_0(x)\, q_{1|0}(y \mid x)}{\hat{\varphi}_1(y)}\, \mathrm{d}x = \int \nabla_y \log q_{1|0}(y \mid x)\, p^\star_{0|1}(x \mid y)\, \mathrm{d}x. \qquad ∎$$

Intuition

$\nabla \log \hat{\varphi}_1(y)$ averages, over the starting points from which the optimal process plausibly reached y, the pull of the reference transition back towards that start. It measures how much easier y is to reach from the source than an average point, which is what the terminal cost must discount. It is a conditional average, hence a regression target.

Proposition 3.15 (Memoryless bridges). If $q_{0,1}(x, y) = q_0(x)\, q_1(y)$, then $f = \frac{\mu}{q_0}$, $g = \frac{\pi}{q_1}$, $\hat{\varphi}_1 = q_1$, and Equation (3.14) reduces to Equation (3.7).

Proof. $\int f(x) q_0(x) q_1(y) g(y)\, \mathrm{d}y = \mu(x)$, and symmetrically, so the marginals are right. Then $\hat{\varphi}_1(y) = \int \mu(x)\, q_{1|0}(y|x)\, \mathrm{d}x = q_1(y)$ because $q_{1|0}(y|x) = q_1(y)$. ∎

Intuition

When the reference forgets where it started, every start is equally good for every end; the optimal coupling is trivial and the bridge just reweights endpoints, as in Theorem 3.4.

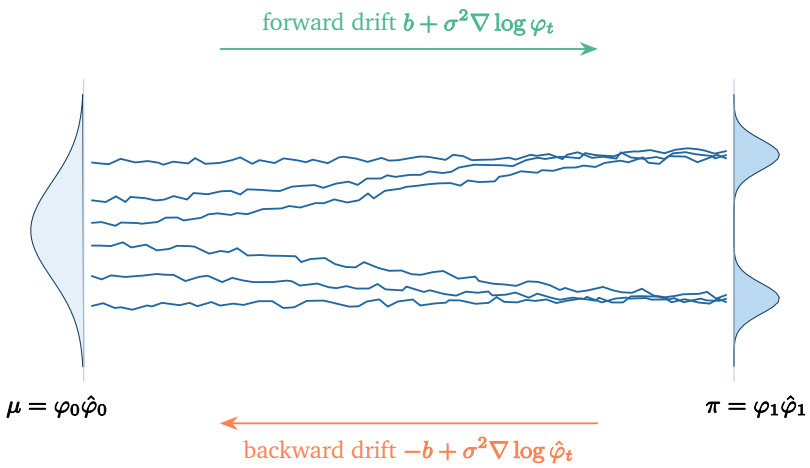


Figure 3.2 A Schrödinger bridge (Theorem 3.12): reference bridges glued with an optimal coupling of μ and π. The forward potential φ steers forward in time, the backward potential $\hat{\varphi}$ steers the time reversal, and their product is the marginal density.

3.5.1. Iterative proportional fitting

The Schrödinger system can be solved by alternately enforcing the two marginal constraints with the smallest possible change (Fortet, 1940; Kullback, 1968; Rüschendorf, 1995).

Lemma 3.16 (Half-bridge projection). For a path law $\mathbb{R}$ with time-1 density r_1, $\arg\min\{\mathrm{KL}(\mathbb{P}\|\mathbb{R}) : p_1 = \pi\}$ is $\mathbb{R}$ reweighted by $\frac{\pi(X_1)}{r_1(X_1)}$; symmetrically for the time-0 constraint. **Density form:** for a coupling density $r_{0,1}$, the projection is $r_{0,1}(x,y)\frac{\pi(y)}{r_1(y)}$, i.e. rescale each column to the right total.

Proof. By Lemma 1.7, conditioning on X_1, $\mathrm{KL}(\mathbb{P}\|\mathbb{R}) = \mathrm{KL}(\pi\|r_1) + \mathbb{E}_{y\sim\pi}\,\mathrm{KL}(\mathbb{P}(\cdot|X_1 = y)\|\mathbb{R}(\cdot|X_1 = y))$, minimised by keeping the conditional laws of $\mathbb{R}$. ∎

Intuition

To impose a marginal at the smallest KL price, change that marginal and keep every conditional law given it. IPF alternates this minimal surgery at the two ends.

IPF starts from $\mathbb{Q}$ and alternates the projections onto $\{p_1 = \pi\}$ and $\{p_0 = \mu\}$. Each iterate keeps the product form $f^{(k)}(X_0)g^{(k)}(X_1)\cdot\mathbb{Q}$, and the iterates converge to $\mathbb{P}^\star$ under integrability conditions (Rüschendorf, 1995; De Bortoli et al., 2021). On a finite grid, IPF is the Sinkhorn algorithm (Listing 3.1). Diffusion Schrödinger bridges (De Bortoli et al., 2021; Shi et al., 2023) implement the projections with learned drifts using samples from both ends; the sampling setting has no samples from π, which is why Chapter 10 uses Theorem 3.13 and Proposition 3.14 instead.

Listing 3.1 Log-domain Sinkhorn (discrete IPF) for the static Schrödinger problem.

```python
import numpy as np
from scipy.special import logsumexp

def sinkhorn_sb(log_mu, log_pi, log_Q01, n_iter=500):
    """Static Schrodinger bridge on a grid by IPF (log-domain Sinkhorn).
    log_mu: (n,), log_pi: (m,), log_Q01: (n, m) reference coupling log Q(x, y).
    Returns log f, log g and the optimal coupling log gamma = log f + log g + log Q01."""
    log_f = np.zeros_like(log_mu)
```

```
    log_g = np.zeros_like(log_pi)
    for _ in range(n_iter):
        # project onto {gamma_1 = pi}: rescale columns
        log_g = log_pi - logsumexp(log_f[:, None] + log_Q01, axis=0)
        # project onto {gamma_0 = mu}: rescale rows
        log_f = log_mu - logsumexp(log_g[None, :] + log_Q01, axis=1)
    log_gamma = log_f[:, None] + log_g[None, :] + log_Q01
    return log_f, log_g, log_gamma

# Example: Brownian reference with variance eps between two 1-D grids
x = np.linspace(-3, 3, 200); eps = 0.5
log_mu = -0.5 * x**2;            log_mu -= logsumexp(log_mu)
log_pi = np.logaddexp(-8 * (x - 1.5)**2, -8 * (x + 1.5)**2); log_pi -= logsumexp(log_pi)
log_Q01 = log_mu[:, None] - (x[None, :] - x[:, None])**2 / (2 * eps)
log_Q01 -= logsumexp(log_Q01, axis=1, keepdims=True) - log_mu[:, None]
log_f, log_g, log_gamma = sinkhorn_sb(log_mu, log_pi, log_Q01)
```

Generative-modelling view

Control with a KL cost is KL-regularised reinforcement learning (Levine, 2018): the control is a policy, $\frac{1}{2}|u|^2$ is the KL penalty to a reference policy, $-g$ is the reward, and Theorem 3.2 is the fact that the optimal maximum-entropy policy is the reference policy reweighted by the exponentiated future reward. $V = -\log h$ is the soft value function and Equation (3.6) the soft Bellman equation. Reward fine-tuning of diffusion models is Theorem 3.4 with $g = -\frac{r}{\lambda}$ and a pretrained base (Theorem 12.2). Schrödinger bridges are entropic optimal transport, solved in ML by Sinkhorn's algorithm (Cuturi, 2013), and with data at both ends they give unpaired translation between domains (De Bortoli et al., 2021; Shi et al., 2023; Liu et al., 2023).

Looking ahead

- Reweighting a process by a function of its endpoint gives another Markov process, generated by an extra drift $\sigma^2 \nabla \log h$ (Proposition 3.1 and Corollary 3.3).
- Sampling is control with terminal cost $\log \frac{p_1^0}{\tilde{\pi}}$, provided the base process is memoryless (Theorem 3.4 and Proposition 3.7).
- The optimal control is an average of a pathwise sensitivity (Theorem 3.8); with zero base drift it is an average of the endpoint force, the key to Adjoint Sampling.
- Without memorylessness the right terminal cost is $\log \frac{\hat{\rho}_1}{\pi}$ (Theorem 3.13), and its gradient is itself a regression target (Proposition 3.14).

CHAPTER 4

The sampling problem: baselines, metrics and benchmarks

Goal	Fix what a sampler must deliver, review the Monte Carlo methods any neural sampler must beat, draw a map of the neural samplers covered in the book, and define the metrics and benchmarks used in the literature.
Prerequisites	Chapter 1; Section 2.4.
Key papers	Neal (2001); Del Moral et al. (2006); Blessing et al. (2024); He et al. (2025).

Overview. Before studying neural samplers we need a baseline and a scoreboard. The baseline is Monte Carlo: Markov chains that converge slowly, and annealed importance sampling, which introduces the key idea of the whole field, comparing a forward and a backward process on the same space of paths. The scoreboard is a set of metrics that together detect the typical failures: missing modes, misweighted modes, and biased normalising constants.

4.1. What does it mean to solve the problem?

Given $\mathcal{E}$ and $\nabla\mathcal{E}$, a sampler may be asked for any of:

1. **Samples** $X^i \sim \pi$, ideally independent and cheap to generate in large numbers;
2. **Expectations** $\mathbb{E}_{x\sim\pi}[f(x)]$ with error bars, which requires exact samples or importance weights;
3. **The normalising constant** $\log Z$, or ratios $\log \frac{Z_1}{Z_0}$ between two targets (free energies, model evidence);
4. **Amortisation**: one model that samples a whole family $\pi_c \propto e^{-\mathcal{E}(\cdot;c)}$, e.g. all molecules c in a chemical space, or all data sets of a Bayesian model.

The cost model matters. When $\mathcal{E}$ is a large neural network, a quantum-chemistry call or a pass over a big data set, the relevant budget is the *number of energy (and gradient) evaluations*, not the cost of the sampler's own network.

4.2. Monte Carlo baselines

Markov chain Monte Carlo (MCMC). Metropolis–Hastings proposes a move from x to y with a proposal density $Q(y \mid x)$ and accepts it with probability $\alpha(x,y) = \min\left\{1, \frac{\pi(y)\,Q(x|y)}{\pi(x)\,Q(y|x)}\right\}$, which only needs $\tilde{\pi}$ because Z cancels. The resulting chain satisfies detailed balance,

$$\pi(x)\,Q(y \mid x)\,\alpha(x,y) = \min\{\pi(x)Q(y \mid x),\, \pi(y)Q(x \mid y)\},$$

which is symmetric in (x,y), so π is invariant. MALA uses one Euler step of Langevin dynamics (Proposition 2.5) as proposal (Roberts and Tweedie, 1996); HMC uses Hamiltonian dynamics (Neal, 2011). All are exact in the limit but suffer from metastability (Figure 2.1).

Annealed importance sampling (AIS). Choose unnormalised densities $\gamma_0, \dots, \gamma_K$ that interpolate between a tractable normalised γ_0 and $\gamma_K = \tilde{\pi}$, e.g. $\gamma_k = \gamma_0^{1-\beta_k}\tilde{\pi}^{\beta_k}$ with $0 = \beta_0 < \cdots < \beta_K = 1$. Write $\pi_k = \frac{\gamma_k}{Z_k}$ and let $K_k(x' \mid x)$ be an MCMC transition density that leaves π_k invariant.

Theorem 4.1 (AIS is unbiased). Draw $X_0 \sim \pi_0$ and $X_k \sim K_k(\cdot \mid X_{k-1})$ for $k = 1, \dots, K$, and set

$$w = \prod_{k=1}^{K} \frac{\gamma_k(X_{k-1})}{\gamma_{k-1}(X_{k-1})}.$$

Then $\mathbb{E}[w] = \frac{Z_K}{Z_0}$, and $\frac{\mathbb{E}[w\, f(X_K)]}{\mathbb{E}[w]} = \mathbb{E}_{\pi_K}[f]$.

Intuition

Each step first changes the target from γ_{k-1} to γ_k under a particle that stands still, which is paid for by the weight; then lets the particle relax towards the new target, which is free because K_k preserves π_k. If the relaxation were perfect at every step, the weights would only account for the changing normalisation.

Proof. The proposal path density is $q(x_{0:K}) = \pi_0(x_0)\prod_k K_k(x_k \mid x_{k-1})$. Let $L_k(x_{k-1} \mid x_k) = \frac{\pi_k(x_{k-1})\,K_k(x_k \mid x_{k-1})}{\pi_k(x_k)}$, the backward transition density of K_k (it integrates to 1 in x_{k-1} because π_k is invariant). Define the unnormalised path density $\Gamma(x_{0:K}) = \gamma_K(x_K)\prod_k L_k(x_{k-1} \mid x_k)$: integrating out $x_0, \dots, x_{K-1}$ leaves $\gamma_K(x_K)$, so its total mass is Z_K and its last coordinate has density π_K. The importance weight on paths is

$$\frac{\Gamma(x_{0:K})}{q(x_{0:K})} = \frac{\gamma_K(x_K)}{\pi_0(x_0)} \prod_{k=1}^{K} \frac{\pi_k(x_{k-1})}{\pi_k(x_k)} = Z_0 \frac{\gamma_K(x_K)}{\gamma_0(x_0)} \prod_{k=1}^{K} \frac{\gamma_k(x_{k-1})}{\gamma_k(x_k)} = Z_0 \prod_{k=1}^{K} \frac{\gamma_k(x_{k-1})}{\gamma_{k-1}(x_{k-1})},$$

by telescoping. Importance sampling on paths (Proposition 1.10) gives the claims. ∎

This proof contains the key idea of the field: *introduce a backward process so that the proposal and the target live on the same space of paths, then compare path densities.* Neural samplers learn the forward transitions (and sometimes the backward ones) instead of fixing them to MCMC moves. **Sequential Monte Carlo (SMC)** samplers (Del Moral et al., 2006) run AIS with N particles and *resample* them (duplicate high-weight particles, discard low-weight ones) when the ESS drops; the estimate of Z remains unbiased. The continuous-time limit of AIS is Jarzynski's equality (Chapter 7).

4.3. A map of neural samplers

Figure 4.1 and Table 4.1 organise the methods of this book by *what is learned* and *how it is trained*.

4.4. Evaluation

No single metric captures sampler quality. Report several, from different groups:

- **Normalising constant.** The ELBO $\mathbb{E}[\log w]$ and, when target samples exist, the EUBO; the importance-sampling estimate $\log \hat{Z}_N$. They are biased in opposite directions (Proposition 1.10), and the gap $\mathrm{EUBO} - \mathrm{ELBO} = \mathrm{KL}(q\|\pi) + \mathrm{KL}(\pi\|q)$ measures mismatch in both directions (Blessing et al., 2024).

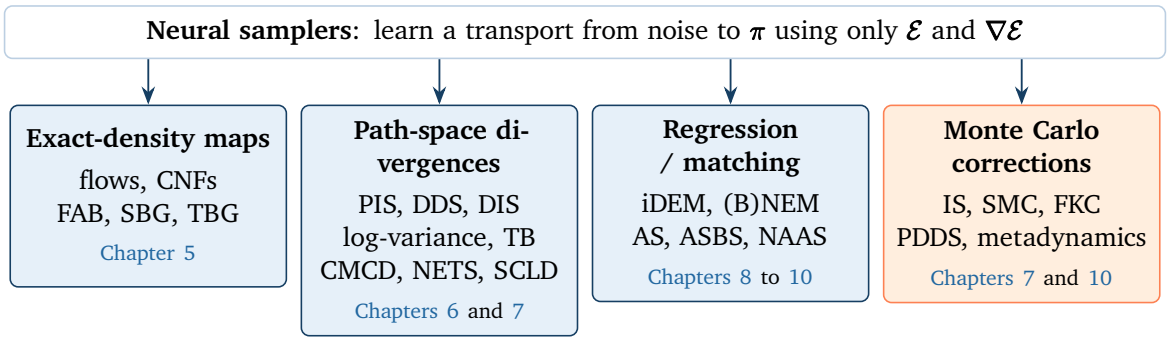


Figure 4.1 A map of neural samplers. The first three columns differ in what is learned and how; Monte Carlo corrections can be added on top of any of them.

Table 4.1 Properties of the main families. "Backprop through SDE" = whether training differentiates through simulated trajectories; "$\nabla\mathcal{E}$ per sample" = energy-gradient evaluations per generated training sample (N = number of time steps, K = Monte Carlo samples per target).

Family	Density?	Backprop through SDE?	$\nabla\mathcal{E}$ per sample	Main weakness
Flows, reverse KL (Chapter 5)	exact	no	1	expressivity; mode collapse
PIS/DDS, reverse KL (Chapter 6)	lower bound	yes	up to N	memory; mode collapse
Log-variance / TB (Chapter 6)	lower bound	no	up to N	variance; needs exploration
CMCD/NETS (Chapter 7)	via weights	optional	N	choice of annealing path
iDEM/NEM (Chapter 8)	no	no	K per target	noisy targets at high noise
Adjoint Sampling (Chapter 9)	via path weights	no	1	mode weights
ASBS/NAAS (Chapter 10)	no	no	1 (ASBS)	extra corrector

- **Weights.** The normalised ESS (Proposition 1.11). It needs tractable densities or path weights, and it can be high for a sampler that covers only one mode well.
- **Sample distances** to reference samples (from long MCMC or MD): maximum mean discrepancy, Sinkhorn or Wasserstein distances on positions; Wasserstein distances between *energy histograms*, which are sensitive to fine structure; KL between low-dimensional projections such as the two backbone angles of a peptide.
- **Mode coverage.** Fraction of known modes found, and their weights compared with the truth. Sample distances average over modes and can miss a missing mode.
- **Cost.** Energy evaluations for training and for sampling, wall-clock time, memory.

4.5. Benchmarks

Exact constants differ between papers, so always check the code. Synthetic targets isolate one difficulty each; particle systems add symmetry (Chapter 11) and steep repulsion, which makes gradients explode and is often handled by smoothing or clipping, a source of hidden bias. The Bayesian targets are standard in machine learning (Arbel et al., 2021; Blessing et al., 2024).

Table 4.2 Common benchmark targets, from synthetic to physical to Bayesian-inference posteriors. d is the dimension; particle systems have n particles and are invariant under translations, rotations and permutations.

Target	d	Definition and difficulty
GMM-40	2 or 50	mixture of 40 Gaussians with means spread over $[-40, 40]^d$; mode coverage
Funnel	10	$x_1 \sim \mathcal{N}(0, 3^2)$, $x_{2:10} \mid x_1 \sim \mathcal{N}(0, e^{x_1}\mathrm{I})$; scales varying by orders of magnitude
Many-well	5–50	sums of independent double wells; $2^{\frac{d}{2}}$ modes
DW-4	4×2	pairwise double-well potential between 4 particles
LJ-13, LJ-55	13×3, 55×3	Lennard-Jones clusters; steep repulsive cores
Alanine dipeptide	66 (or 60)	22-atom peptide at 300 K; metastable states in two backbone angles
Bayesian logistic regression	25–61	German Credit, Ionosphere, Sonar; log-concave but ill-conditioned
Log-Gaussian Cox process	1600	spatial point process on a 40×40 grid; strongly correlated
Brownian-motion inference	32	latent time series with unknown noise scales; funnel-like geometry

Generative-modelling view

Evaluating a sampler resembles evaluating a generative model, with one important difference. *Precision* (are samples in high-density regions?) and *recall* (are all modes covered?) carry over, and sample distances play the role of FID. But a sampler can also be checked against the *exact* unnormalised density: energy histograms, ELBO and ESS are available for free, and they catch a failure that looking at samples never does, namely samples that look right but have the wrong mode *weights*. The habit of training until samples look good is dangerous here, because a mode-collapsed sampler can produce perfect-looking samples.

Open problems and research questions

- **Fair comparisons.** He et al. (2025) report that the gains of several "simulation-free" samplers depend on implementation tricks (energy clipping, buffers, initialisation) and shrink against well-tuned simple baselines. A shared protocol that counts *energy evaluations* is still missing.
- **Metrics without ground truth.** For real molecules there is no exact reference; long simulations are themselves biased by finite length. How do we certify mode coverage of a sampler that is better than its reference?
- **When do neural samplers beat MCMC?** We lack theory, and even a clear empirical picture, of the regimes (dimension, number of modes, barrier height) in which learned transport beats well-tuned parallel tempering or SMC at an equal energy budget.

CHAPTER 5

Boltzmann generators: normalising flows as samplers

Goal	Learn samplers with an exact, computable density (normalising flows): how to train them with and without data, how to correct them with importance weights, how FAB avoids mode collapse, how to build in symmetry, and how recent transferable and sequential Boltzmann generators scale up.
Prerequisites	Chapter 1, in particular Theorem 1.1 and Section 1.5.
Key papers	Noé et al. (2019); Papamakarios et al. (2021); Midgley et al. (2023b); Köhler et al. (2020); Klein et al. (2023); Klein and Noé (2024); Tan et al. (2025a).

Overview. The simplest neural sampler is one big invertible map from noise to samples. Its great advantage is that the density of its output is known exactly (Theorem 1.1), so it can be corrected by importance weights and used to estimate Z. Its weakness is that invertibility limits how flexible the map can be. This chapter shows how to train such maps from energies, from data, or from both.

5.1. The idea

A *Boltzmann generator* (BG) (Noé et al., 2019) is a normalising flow $x = T_\theta(z)$, $z \sim \mu = \mathcal{N}(0, \mathrm{I})$, whose output density q_θ is given by Equation (1.1):

$$\log q_\theta(T_\theta(z)) = \log \mu(z) - \log |\det \nabla T_\theta(z)|.$$

Samples are independent and generated in one pass, and the exact density gives importance weights $w = \frac{\tilde{\pi}(x)}{q_\theta(x)}$, hence unbiased estimates of Z and asymptotically exact expectations (Proposition 1.10).

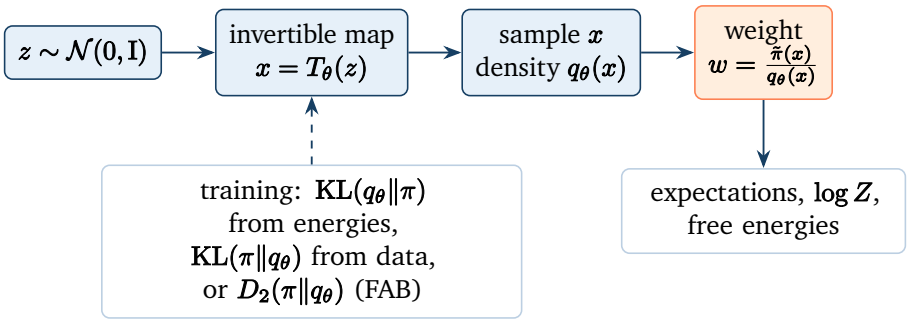


Figure 5.1 A Boltzmann generator: an invertible map with exact output density, trained from energies and/or data and corrected by importance weights.

5.2. Architectures and their densities

Coupling layers. Split $x = (x_a, x_b)$ and set $y_a = x_a$, $y_b = x_b \odot e^{s(x_a)} + t(x_a)$, with arbitrary networks s, t (Dinh et al., 2017). The Jacobian matrix is triangular, so $\log|\det \nabla T| = \sum_j s_j(x_a)$, and the inverse is explicit. Stacking layers that alternate which half is transformed gives an expressive map with a cheap density (Corollary 1.2). Spline couplings (Durkan et al., 2019) replace the affine map by monotone splines; circular splines handle angles (Rezende et al., 2020).

Continuous normalising flows (CNFs). Take T_θ to be the time-1 map of an ODE $\frac{\mathrm{d}x}{\mathrm{d}t} = v_\theta(t, x)$ (Chen et al., 2018; Grathwohl et al., 2019). Any network can be used for v_θ, including equivariant graph networks, and Corollary 1.4 gives the density. The divergence $\nabla\cdot v_\theta$ is the expensive part.

> **Proposition 5.1 (Hutchinson's trace estimator).** If the random vector ϵ has mean 0 and covariance $\mathbf{I}$, then $\mathbb{E}[\epsilon^\top \nabla v(x)\,\epsilon] = \nabla\cdot v(x)$. One vector–Jacobian product per sample suffices (Hutchinson, 1989).

Proof. $\mathbb{E}[\epsilon^\top A \epsilon] = \mathbb{E}[\operatorname{tr}(A\epsilon\epsilon^\top)] = \operatorname{tr}(A\,\mathbb{E}[\epsilon\epsilon^\top]) = \operatorname{tr} A$, and $\operatorname{tr}\nabla v = \nabla\cdot v$. ∎

> **Intuition**
>
> The trace of a matrix is the average of $\epsilon^\top A\epsilon$ over random directions: probe how much the Jacobian stretches along a random direction, and average. The estimate of $\log q$ is unbiased, but the importance weight $e^{\log\tilde\pi - \log q}$ computed from it is not, so exact divergences are preferred when weights are needed.

5.3. Training objectives

From energies (reverse KL). With $x = T_\theta(z)$,

$$\mathrm{KL}(q_\theta\|\pi) = \mathbb{E}_{z\sim\mu}\Big[\log\mu(z) - \log|\det\nabla T_\theta(z)| + \mathcal{E}\big(T_\theta(z)\big)\Big] + \log Z. \tag{5.1}$$

It needs no data, only $\mathcal{E}$ and $\nabla\mathcal{E}$ (gradients flow through $T_\theta(z)$), but it is mode-seeking (Section 1.6).

From data (forward KL). If samples from π (or approximately from π, e.g. from biased simulations) exist, $\mathrm{KL}(\pi\|q_\theta) = -\mathbb{E}_{x\sim\pi}[\log q_\theta(x)] + \text{const}$ is maximum likelihood. It covers all modes present in the data but inherits the data's biases. Noé et al. (2019) combine both.

Reweighting. With $X^i \sim q_\theta$ and $w_i = \frac{\tilde\pi(X^i)}{q_\theta(X^i)}$, the self-normalised estimator removes the remaining bias asymptotically (Section 1.5), at a cost set by $\chi^2(\pi\|q_\theta)$ (Proposition 1.11). Targeted free-energy perturbation (Wirnsberger et al., 2020) uses the same idea to estimate $\log\frac{Z_1}{Z_0}$ between two thermodynamic states.

Listing 5.1 RealNVP-type flow trained by reverse KL on a 2-D double well, with importance-sampling estimates of $\log Z$ and ESS. It recovers $\log Z = 1.558$ (exact 1.5585) with ESS ≈ 0.96.

```python
import torch, torch.nn as nn

class AffineCoupling(nn.Module):
    """x = [x_a, x_b];  y_a = x_a,  y_b = x_b * exp(s(x_a)) + t(x_a)."""
    def __init__(self, d, mask, hidden=64):
        super().__init__()
        self.register_buffer("mask", mask)
```

```
        self.net = nn.Sequential(nn.Linear(d, hidden), nn.SiLU(), nn.Linear(hidden, hidden),
                                 nn.SiLU(), nn.Linear(hidden, 2 * d))
    def forward(self, x):                          # returns y and log|det dy/dx|
        s, t = self.net(x * self.mask).chunk(2, dim=-1)
        s = torch.tanh(s) * (1 - self.mask)        # bounded scales, act only on x_b
        t = t * (1 - self.mask)
        return x * torch.exp(s) + t, s.sum(-1)

class Flow(nn.Module):
    def __init__(self, d, n_layers=8):
        super().__init__()
        masks = [torch.arange(d) % 2 == (i % 2) for i in range(n_layers)]
        self.layers = nn.ModuleList(AffineCoupling(d, m.float()) for m in masks)
    def sample(self, n):                           # x = T(z) and log q(x) by Corollary 1.3
        z = torch.randn(n, self.layers[0].mask.numel())
        logq = -0.5 * (z**2).sum(-1) - 0.5 * z.shape[1] * torch.log(torch.tensor(2 * torch.pi))
        for layer in self.layers:
            z, logdet = layer(z)
            logq = logq - logdet
        return z, logq

def energy(x):   # 2-D double well: modes at x1 = +-2
    return 0.25 * (x[:, 0]**2 - 4)**2 + 0.5 * x[:, 1]**2

def train_reverse_kl(flow, steps=3000, batch=512, lr=1e-3):
    opt = torch.optim.Adam(flow.parameters(), lr=lr)
    for _ in range(steps):
        x, logq = flow.sample(batch)               # reparameterised samples
        loss = (logq + energy(x)).mean()           # KL(q || pi) - log Z
        opt.zero_grad(); loss.backward(); opt.step()
    return flow

if __name__ == "__main__":
    torch.manual_seed(0)
    flow = train_reverse_kl(Flow(2))
    with torch.no_grad():
        x, logq = flow.sample(20000)
        logw = -energy(x) - logq                   # log importance weights
        ess = torch.exp(2 * torch.logsumexp(logw, 0) - torch.logsumexp(2 * logw, 0)) / len(logw)
```

5.4. Flow Annealed importance sampling Bootstrap (FAB)

FAB (Midgley et al., 2023b) trains a flow without data using a mass-covering objective, the Rényi divergence of order 2,

$$D_2(\pi\|q_\theta) = \log \int \frac{\pi(x)^2}{q_\theta(x)}\,\mathrm{d}x = \log\big(1 + \chi^2(\pi\|q_\theta)\big). \tag{5.2}$$

By Proposition 1.11, minimising D_2 is the same as maximising the asymptotic ESS of the flow as an importance sampler.

Proposition 5.2 (FAB gradient). $\nabla_\theta D_2(\pi\|q_\theta) = -\,\mathbb{E}_{x\sim g_\theta}[\nabla_\theta \log q_\theta(x)]$, where $g_\theta(x) = \frac{1}{C_\theta}\frac{\pi(x)^2}{q_\theta(x)}$, with $C_\theta = \int \frac{\pi^2}{q_\theta}\,\mathrm{d}x$, is the density proportional to $\frac{\pi^2}{q_\theta}$.

Proof. $\nabla_\theta \log \int \frac{\pi^2}{q_\theta}\,\mathrm{d}x = \dfrac{\int \frac{\pi^2}{q_\theta}\,(-\nabla_\theta \log q_\theta)\,\mathrm{d}x}{\int \frac{\pi^2}{q_\theta}\,\mathrm{d}x}$. ∎

Intuition

The density $g_\theta \propto \frac{\pi^2}{q_\theta}$ is large where π is large and q_θ is small: exactly the regions the flow is missing. The gradient raises $\log q_\theta$ there.

FAB estimates the expectation over g_θ by AIS (Theorem 4.1) started from q_θ and targeting g_θ, with HMC moves, and stores samples and weights in a replay buffer so that each expensive AIS run serves several gradient steps. FAB was the first data-free method to sample alanine dipeptide in Cartesian coordinates accurately; a symmetry-aware version extends it to particle systems (Midgley et al., 2023a).

5.5. Symmetry

Physical energies do not change under rotations, translations and permutations of identical atoms; many ML targets have symmetries too (e.g. permutations of hidden units in a neural network). A sampler that ignores them must learn them, wasting capacity.

Proposition 5.3 (Equivariant maps preserve invariance). Let a group G act on $\mathbb{R}^d$ by linear maps with $|\det g| = 1$ (e.g. rotations, permutations). If the base density μ is invariant ($\mu(gz) = \mu(z)$) and T is equivariant ($T(gz) = gT(z)$), then the output density is invariant: $q(gx) = q(x)$. If a velocity field is equivariant, so is its flow.

Proof. Equivariance of T implies $T^{-1}(gx) = gT^{-1}(x)$; differentiating, $\nabla T^{-1}(gx)\, g = g\, \nabla T^{-1}(x)$, so the two Jacobian determinants have the same absolute value. By Equation (1.1),

$$q(gx) = \mu\big(gT^{-1}(x)\big)\,|\det \nabla T^{-1}(gx)| = \mu\big(T^{-1}(x)\big)\,|\det \nabla T^{-1}(x)| = q(x).$$

For the flow, $y_t = g\Phi_t(x)$ solves the same ODE as $\Phi_t(gx)$ with the same initial value gx, so they coincide. ∎

Intuition

If the noise looks the same from every orientation and the network treats every orientation alike, the output cannot prefer any orientation. Symmetry built into the architecture costs no data and no energy evaluations to learn.

Rotations and permutations are handled this way, with an isotropic Gaussian base and an equivariant network such as an EGNN (Satorras et al., 2021). Translations need a different treatment, because no probability density on $\mathbb{R}^{3n}$ is translation invariant: one works on the subspace of configurations with zero centre of mass (Section 11.4). Note that equivariance under reflections maps a molecule to its mirror image, which is wrong for chiral molecules.

5.6. Flow matching for Boltzmann generators

When samples from π (or a biased version of it) exist, a CNF can be trained without simulating it, by flow matching (Lipman et al., 2023; Albergo et al., 2023; Tong et al., 2024). Draw a pair (X_0, X_1) with $X_0 \sim \mu$ and X_1 from the data, and form the straight line $X_t = (1-t)X_0 + tX_1$, whose velocity is $X_1 - X_0$.

Theorem 5.4 (Flow matching). Let p_t be the density of X_t and $v_t(x) = \mathbb{E}[X_1 - X_0 \mid X_t = x]$. Then $\partial_t p_t + \nabla\cdot(p_t v_t) = 0$, so the ODE with velocity v moves μ to the data distribution, and

$$\mathbb{E}_{t,X_0,X_1}\left|v_\theta(t,X_t) - (X_1 - X_0)\right|^2 = \mathbb{E}_{t,X_t}\left|v_\theta(t,X_t) - v_t(X_t)\right|^2 + C,$$

with C independent of θ.

Proof. For a test function φ, $\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}[\varphi(X_t)] = \mathbb{E}[\nabla\varphi(X_t)\cdot(X_1 - X_0)] = \mathbb{E}[\nabla\varphi(X_t)\cdot v_t(X_t)]$ by conditioning on X_t; this is the continuity equation in weak form, and Theorem 1.3 applies. The loss identity is the usual decomposition of least squares around the conditional expectation. ∎

Intuition

Each pair defines a trivial straight-line motion. The velocity at a point is the average of the velocities of all straight lines passing through it, and since the continuity equation is linear, this average moves the densities correctly. Regressing on the per-pair velocity learns the average automatically.

Klein et al. (2023) combine this with equivariant networks and pairs aligned over rotations and permutations, which shortens and straightens the paths. The trained CNF still has an exact density (Corollary 1.4), so it remains a Boltzmann generator.

5.7. Transferable and sequential Boltzmann generators

Transferable BGs. Klein and Noé (2024) condition an equivariant CNF on the molecular graph and train it on simulation data of many dipeptides; it samples unseen dipeptides and is corrected by importance weights. This is *amortisation* in the sense of Chapter 4.

Sequential BGs. Tan et al. (2025a) scale BGs to larger peptides by (i) a fast, non-equivariant, exactly invertible transformer flow, with data augmentation for symmetry, and (ii) replacing one-shot importance sampling by SMC at inference: samples from the flow are moved towards π along an annealing path by Langevin steps with Jarzynski weights (Chapter 7) and resampling. Tan et al. (2025b) extend this to a flow transferable across peptide sequences. Both train on simulation data; the energy enters through the correction.

Generative-modelling view

Normalising flows entered ML as flexible variational families (Rezende and Mohamed, 2015), i.e. exactly as energy-trained Boltzmann generators; density estimation from data came later. Today's practice for BGs mirrors that of large generative models: *pretrain* on cheap, biased data by maximum likelihood or flow matching, then *adapt* to the true target using the energy, by reweighting, by SMC, or by fine-tuning with the energy as a reward (Chapter 12). Equivariance versus augmentation is the same trade-off as convolutions versus data augmentation in image models.

Key takeaways

- A flow is a sampler with an exact density; reweighting makes it asymptotically exact, at a cost set by $\chi^2(\pi\|q_\theta)$, which grows quickly with system size.

- Reverse KL is data-free but mode-seeking; FAB's D_2 objective is data-free and mass-covering, at the price of an inner AIS loop.
- For large systems: train a fast flow on (possibly biased) data, then correct with energy-based SMC rather than plain importance sampling.

Open problems and research questions

- **Keeping weights useful at scale.** Importance weights degenerate with system size even for good flows. Can SMC with learned transitions keep the ESS bounded as the number of atoms grows, and at what energy cost?
- **Exact likelihoods for expressive models.** CNF densities need many backward passes; trace estimates bias the weights. Are there architectures with cheap exact Jacobians that match diffusion models in quality?
- **Data-free transferability.** Transferable BGs are trained on simulation data; training them from energies alone, across chemical space, is open (Chapter 13).
- **Symmetry versus speed.** Quantify the bias that approximate symmetry (augmentation instead of equivariance) induces in reweighted estimates.

CHAPTER 6

Diffusion samplers and path-space divergences

Goal	Turn an SDE with a learned drift into a sampler by making its path law match a target path law. Derive the path ELBO in discrete time, see PIS, DDS and time-reversal samplers as special cases, and compare the losses used to train them.
Prerequisites	Chapter 2 (in particular Sections 2.1 and 2.5); Chapter 3.
Key papers	Zhang and Chen (2022); Vargas et al. (2023a); Berner et al. (2024); Richter and Berner (2024); Nüsken and Richter (2021); Malkin et al. (2022); Sendera et al. (2024).

Overview. A diffusion sampler simulates $\mathrm{d}X_t = (b_t + \sigma_t u_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t$ with a neural control u. We cannot compute the density p_1^u of its output, so we cannot compare it to π directly. But we can compute the density of a whole *discretised path* (Equation (2.2)). The trick, already used by AIS (Theorem 4.1), is to build a second, *target* process on the same space of paths whose final density is π, and to compare the two path densities. This chapter develops that trick.

6.1. The discrete-time framework

On a grid $t_0 < \cdots < t_N$, define two Markov chains with path densities

$$p^F(x_{0:N}) = \mu(x_0) \prod_{n=0}^{N-1} F_n(x_{n+1} \mid x_n), \qquad p^L(x_{0:N}) = \pi(x_N) \prod_{n=0}^{N-1} L_n(x_n \mid x_{n+1}). \tag{6.1}$$

The *forward* chain p^F is our sampler: it starts at the source μ and moves with learned transition densities F_n. The *backward* (target) chain p^L starts at π at the final time and moves backwards with transition densities L_n, for example a noising process. We cannot sample p^L (that would require samples of π), but we can evaluate it up to the constant Z by replacing π with $\tilde{\pi}$.

Theorem 6.1 (Path ELBO). Define the path weight

$$w(x_{0:N}) = \frac{\tilde{\pi}(x_N) \prod_n L_n(x_n \mid x_{n+1})}{\mu(x_0) \prod_n F_n(x_{n+1} \mid x_n)} = Z\,\frac{p^L(x_{0:N})}{p^F(x_{0:N})}.$$

Then $\mathbb{E}_{p^F}[w] = Z$ and $\mathbb{E}_{p^F}[\log w] = \log Z - \mathrm{KL}(p^F \| p^L) \le \log Z$. If $\mathrm{KL}(p^F \| p^L) = 0$, the sampler ends exactly at π; in general $\mathrm{KL}(p_N^F \| \pi) \le \mathrm{KL}(p^F \| p^L)$, where p_N^F is the density of the sampler's output x_N.

Proof. $\mathbb{E}_{p^F}[w] = Z \int p^L = Z$, and $\mathbb{E}_{p^F}[\log w] = \log Z - \mathrm{KL}(p^F \| p^L)$. The last claim is data processing (Corollary 1.8) for the map $x_{0:N} \mapsto x_N$, under which p^L becomes π. ∎

Intuition

This is AIS with *learned* transitions. We cannot ask "is the density of my output equal to π?", because we cannot compute that density. We ask instead "is my path law equal to a path law that ends at π?", which only needs path densities. Every diffusion sampler is defined by three choices: the forward transitions, the backward transitions (fixed or learned), and the divergence used to compare the two path laws.

With Euler–Maruyama transitions $F_n(\cdot \mid x) = \mathcal{N}\big(x + h_n(b + \sigma u)(x, t_n),\, \sigma_{t_n}^2 h_n \mathrm{I}\big)$, $\log w$ is a sum of Gaussian log-density ratios, cheap to compute along any given path; as $h \to 0$ it converges to a Girsanov log-ratio (Theorem 2.9).

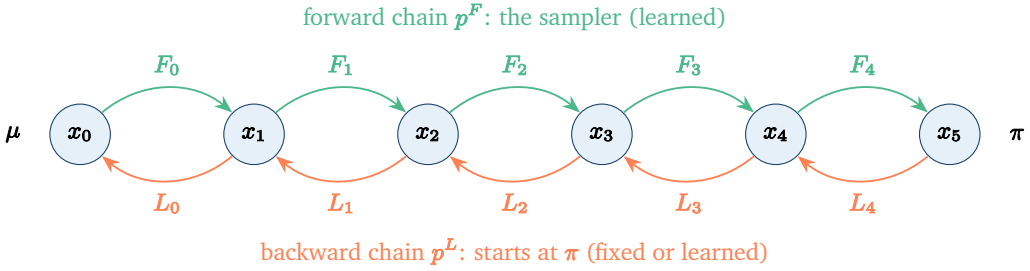


Figure 6.1 Forward and backward chains on the same space of paths. The log-weight $\log w$ compares their path densities along one trajectory.

6.2. Three classical instances

6.2.1. Path Integral Sampler (PIS)

Take $\mu = \delta_0$, base drift $b = 0$, and as backward chain the base random walk $X_{n+1} = X_n + \sigma\sqrt{h}\,\xi_n$ from 0, *conditioned* to end at a point drawn from π. Writing q for the path density of the base walk and q_N for the density of its endpoint, the target path density is $p^L(x_{0:N}) = q(x_{0:N})\frac{\pi(x_N)}{q_N(x_N)}$, and the log-weight is

$$\log w(x_{0:N}) = \log \frac{\tilde\pi(x_N)}{q_N(x_N)} + \sum_{n=0}^{N-1} \log \frac{\mathcal{N}(x_{n+1};\, x_n,\, \sigma^2 h \mathrm{I})}{F_n(x_{n+1} \mid x_n)}. \tag{6.2}$$

In continuous time this is exactly the control problem of Theorem 3.4: the target path law $\mathbb{P}^\star$ reweights the base path law by $\frac{\tilde\pi(X_1)}{Z\, p_1^0(X_1)}$, and

$$\mathrm{KL}(\mathbb{P}^u \| \mathbb{P}^\star) = \mathbb{E}_{\mathbb{P}^u}\left[\int_0^1 \frac{1}{2}|u_t(X_t)|^2\,\mathrm{d}t + \log \frac{p_1^0(X_1)}{\tilde\pi(X_1)}\right] + \log Z. \tag{6.3}$$

The point mass makes the base memoryless, so the optimum is exact (Corollary 3.5). PIS (Zhang and Chen, 2022) minimises Equation (6.3) by backpropagating through the simulation. The noise level should make $p_1^0 = \mathcal{N}(0, \sigma^2 \mathrm{I})$ cover the support of π: too small and the control must be large, too large and paths are very noisy.

6.2.2. Denoising Diffusion Sampler (DDS)

Take the OU process $\mathrm{d}X_t = -\frac{\beta_t}{2} X_t\,\mathrm{d}t + \sqrt{\beta_t}\,\mathrm{d}B_t$ started at $\mathcal{N}(0, \mathrm{I})$. It is stationary and time-reversible; call its path law $\mathbb{Q}$. The target path law runs the OU noising backwards from π: $\mathbb{P}^{\mathrm{tgt}}$

is $\mathbb{Q}$ reweighted by $\frac{\pi(X_1)}{\mathcal{N}(X_1;0,\mathrm{I})}$ (reversibility makes "reweight the endpoint" the same as "start the reversed OU at π"). The sampler starts at $\mu = \mathcal{N}(0,\mathrm{I})$ (Vargas et al., 2023a).

Proposition 6.2 (DDS objective and its bias). $\mathrm{KL}(\mathbb{P}^u \| \mathbb{P}^{\mathrm{tgt}}) = \mathbb{E}_{\mathbb{P}^u}\left[\int_0^1 \frac{1}{2}|u_t|^2\,\mathrm{d}t + \log\frac{\mathcal{N}(X_1;0,\mathrm{I})}{\tilde{\pi}(X_1)}\right] + \log Z$. Its minimum over u is $\mathrm{KL}(\mathcal{N}(0,\mathrm{I}) \,\|\, p_0^{\mathrm{tgt}})$, where p_0^{tgt} is the time-0 density of $\mathbb{P}^{\mathrm{tgt}}$, and the optimal sampler's output density satisfies, with $S = \int_0^1 \beta_t\,\mathrm{d}t$,

$$\mathrm{TV}(p_1^{u^\star}, \pi) \le \mathrm{TV}\big(\mathcal{N}(0,\mathrm{I}), p_0^{\mathrm{tgt}}\big) \le \sqrt{\frac{1}{2}\, e^{-S}\ \mathrm{KL}\left(\pi \| \mathcal{N}(0,\mathrm{I})\right)}.$$

Proof. The identity is Corollary 2.10 together with the definition of $\mathbb{P}^{\mathrm{tgt}}$. By the chain rule conditioning on X_0, $\mathrm{KL}(\mathbb{P}^u \| \mathbb{P}^{\mathrm{tgt}}) = \mathrm{KL}(\mathcal{N}(0,\mathrm{I}) \| p_0^{\mathrm{tgt}}) + \mathbb{E}\,\mathrm{KL}(\mathbb{P}^u(\cdot|X_0) \| \mathbb{P}^{\mathrm{tgt}}(\cdot|X_0))$. Each conditional $\mathbb{P}^{\mathrm{tgt}}(\cdot|x_0)$ reweights the endpoint of $\mathbb{Q}(\cdot|x_0)$, so it is an h-transform, i.e. the law of a controlled SDE (Theorem 2.14); the second term can be made zero. Then $p_1^{u^\star}$ and π are obtained from $\mathcal{N}(0,\mathrm{I})$ and p_0^{tgt} by the *same* transition $x_0 \mapsto \mathbb{P}^{\mathrm{tgt}}(X_1 \in \cdot \mid x_0)$, and transitions can only decrease total variation. Finally, p_0^{tgt} is the density of OU noising run from π, which after a time change is Langevin dynamics for $\mathcal{N}(0,\mathrm{I})$ run for time $\frac{S}{2}$; the Gaussian satisfies Theorem 2.6 with $\lambda = 1$, so $\mathrm{KL}(p_0^{\mathrm{tgt}} \| \mathcal{N}(0,\mathrm{I})) \le e^{-S}\,\mathrm{KL}(\pi \| \mathcal{N}(0,\mathrm{I}))$, and Pinsker's inequality $\mathrm{TV} \le \sqrt{\frac{1}{2}\,\mathrm{KL}}$ concludes. ∎

Intuition

DDS starts at $\mathcal{N}(0,\mathrm{I})$, whereas the exact reversed noising would start at the noised target, which is only approximately Gaussian. The control can fix everything except this initial mismatch, and the noising makes the mismatch exponentially small in the noising time, as in DDPM.

In exchange for this small bias, the OU drift pulls trajectories towards the origin, which makes training better conditioned than PIS on targets far from the origin.

6.2.3. Time-reversal and general bridge samplers (DIS)

Instead of fixing the backward chain, learn it too: L_n with a learned drift. The diffusion-based sampler DIS (Berner et al., 2024) and the general bridge framework of Richter and Berner (2024) minimise a divergence between p^F and p^L over both. By Theorem 2.12, the two are consistent when the backward drift is the score correction of the forward process, so the learned backward drift plays the role of the unknown score. Learning both directions removes the need for a memoryless base or a long noising time, but the problem is not identifiable: infinitely many forward–backward pairs connect μ and π. Fixing one side (as in PIS and DDS) or adding a Schrödinger-bridge penalty (Chapter 10) restores uniqueness.

6.3. Losses on path space

All losses compare $\mathbb{P}^u$ with a target path law $\mathbb{P}^{\mathrm{tgt}}$ through the log-ratio $\ell_u(X) = \log\frac{\mathrm{d}\mathbb{P}^u}{\mathrm{d}\mathbb{P}^{\mathrm{tgt}}}(X) = \log Z - \log w(X)$; in discrete time $\ell_u(x_{0:N}) = \log\frac{p^F(x_{0:N})}{p^L(x_{0:N})}$.

Reverse KL. $\mathbb{E}_{\mathbb{P}^u}[\ell_u]$, i.e. minus the ELBO. Its reparameterised gradient differentiates through the whole simulation: memory grows with the number of steps (or an adjoint ODE costs extra compute), and it is mode-seeking.

Log-variance (LV). For any reference path law $\mathbb{R}$ (with discrete path density r),

$$\mathcal{L}_{\mathrm{LV}}(u;\mathbb{R}) = \mathrm{Var}_{X\sim\mathbb{R}}\left[\ell_u(X)\right] = \mathrm{Var}_{X\sim\mathbb{R}}\left[\log w(X)\right]. \tag{6.4}$$

Proposition 6.3 (Properties of the log-variance loss). (i) If $\mathbb{R}$ gives positive probability to all paths, $\mathcal{L}_{\mathrm{LV}}(u;\mathbb{R}) = 0$ if and only if $\mathbb{P}^u = \mathbb{P}^{\mathrm{tgt}}$. (ii) In discrete time, if the paths are drawn from the current model with gradients stopped ($\mathbb{R} = \mathbb{P}^{\bar{u}}$, $\bar{u} = \mathrm{sg}(u)$), then $\nabla_\theta \mathcal{L}_{\mathrm{LV}} = 2\nabla_\theta \mathrm{KL}(p_\theta^F \| p^L)$ in expectation. (iii) $\mathcal{L}_{\mathrm{LV}}$ never differentiates through the sampling of X.

Proof. (i) Zero variance means $\log w$ is constant on (almost) all paths; then $\mathbb{E}_{p^F}[w] = Z$ forces the constant to be $\log Z$, so $p^F = p^L$. (ii) Write $p_\theta = p_\theta^F$ and $\ell_\theta = \log p_\theta - \log p^L$. With samples from $p_{\bar{\theta}}$, $\nabla_\theta \mathrm{Var}[\ell_\theta] = 2\,\mathbb{E}_{p_{\bar{\theta}}}[(\ell_\theta - \mathbb{E}\,\ell_\theta)\nabla_\theta \ell_\theta]$. At $\bar{\theta} = \theta$, since $\nabla_\theta \ell_\theta = \nabla_\theta \log p_\theta$ has mean zero under p_θ, this equals $2\,\mathbb{E}_{p_\theta}[\ell_\theta \nabla_\theta \log p_\theta]$. On the other hand $\nabla_\theta \mathrm{KL}(p_\theta \| p^L) = \nabla_\theta \int p_\theta \ell_\theta = \int \ell_\theta \nabla_\theta p_\theta + \int p_\theta \nabla_\theta \ell_\theta = \mathbb{E}_{p_\theta}[\ell_\theta \nabla_\theta \log p_\theta] + 0$. (iii) holds by construction. ∎

Intuition

The log-variance loss asks the log-weight to be *constant* on the paths we look at, rather than small on average. Because those paths can come from anywhere (an exploratory policy, a replay buffer, MCMC), the loss is *off-policy*: we can show the sampler regions it would not visit by itself. On-policy, it is the KL gradient with a built-in baseline, a classical variance-reduction device (Richter et al., 2020; Nüsken and Richter, 2021).

Trajectory balance (TB). From the GFlowNet literature (Malkin et al., 2022; Lahlou et al., 2023): $\mathcal{L}_{\mathrm{TB}}(u, c) = \mathbb{E}_{\mathbb{R}}[(\log w(X) - c)^2]$ with a learned scalar c. The best c is $\mathbb{E}_{\mathbb{R}}[\log w]$, at which $\mathcal{L}_{\mathrm{TB}} = \mathcal{L}_{\mathrm{LV}}$; so TB is LV plus an estimate of $\log Z$. *Sub-trajectory* variants enforce the same balance on pieces of paths with a learned estimate of intermediate log-densities, which improves credit assignment (Zhang et al., 2024).

Off-policy exploration. Sendera et al. (2024) study behaviour policies for TB and LV: extra noise in the forward SDE, replay buffers of good endpoints, and *local search* (MCMC moves on buffer samples, followed by backward trajectories from them). Such choices often matter more than the loss itself (Blessing et al., 2024).

Algorithm 1 Diffusion sampler trained with the log-variance loss (PIS target)

Require: energy $\mathcal{E}$, control network u_θ, noise σ, steps N, exploration schedule $\eta_k \geq 0$

for iteration $k = 1, 2, \ldots$ **do**
  simulate B paths $x_{0:N}$ with $\mathrm{sg}(u_\theta)$ from $x_0 = 0$, with noise scale $\sigma(1 + \eta_k)$ ▷ no gradients
  compute $\log w$ on these paths with Equation (6.2), now *with* gradients in θ through F_n
  $\mathcal{L} \leftarrow$ empirical variance of $\log w$ over the batch; gradient step on θ
end for
return u_θ; sample with $\eta = 0$ and report $\mathbb{E}[\log w]$ (ELBO), $\log \frac{1}{M}\sum_i w_i$ and the ESS

Listing 6.1 Algorithm 1 in PyTorch. On a 4-mode Gaussian mixture in 2-D it finds all modes with weights ≈ 0.25 each and $\log \hat{Z} = 1.840$ (exact 1.838), ESS ≈ 0.47.

```python
import math, torch, torch.nn as nn

class Control(nn.Module):
    """u_theta(x, t): small MLP with time embedding."""
```

```
    def __init__(self, d, hidden=128):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(d + 1, hidden), nn.SiLU(), nn.Linear(hidden, hidden),
                                 nn.SiLU(), nn.Linear(hidden, hidden), nn.SiLU(),
     nn.Linear(hidden, d))
    def forward(self, x, t):
        return self.net(torch.cat([x, t.expand(x.shape[0], 1)], -1))

def gmm_energy(x, means, std=0.5):   # E(x) = -log sum_k N(x; m_k, std^2 I) (up to a constant)
    d2 = ((x[:, None, :] - means[None]) ** 2).sum(-1)
    return -torch.logsumexp(-d2 / (2 * std**2), dim=1)

def log_normal(x, mean, var):
    return -0.5 * ((x - mean) ** 2).sum(-1) / var - 0.5 * x.shape[-1] * math.log(2 * math.pi *
     var)

def rollout_logw(u, energy, n, d, sigma=3.0, N=50, explore=0.0):
    """Euler-Maruyama PIS rollout from x_0 = 0. Paths are generated WITHOUT gradients
    (behaviour policy, optionally with extra noise); log w = log dP*/dP^u + log Z is then
    re-evaluated WITH gradients on the fixed paths (discrete Girsanov, Section 6.1)."""
    h = 1.0 / N
    with torch.no_grad():                                  # 1) simulate (off-policy allowed)
        xs = [torch.zeros(n, d)]
        for k in range(N):
            t = torch.tensor([[k * h]])
            mean = xs[-1] + h * sigma * u(xs[-1], t)
            xs.append(mean + sigma * math.sqrt(h) * (1 + explore) * torch.randn(n, d))
    logw = -energy(xs[-1]) - log_normal(xs[-1], 0.0, sigma**2)   # log pi~/p^ref_1 at x_N
    for k in range(N):                                     # 2) + sum log ref/model kernels
        t = torch.tensor([[k * h]])
        x0, x1 = xs[k], xs[k + 1]
        logw = logw + log_normal(x1, x0, sigma**2 * h) \
                    - log_normal(x1, x0 + h * sigma * u(x0, t), sigma**2 * h)
    return logw, xs[-1]

def train_lv(d=2, steps=2000, batch=256, lr=1e-3):
    means = torch.tensor([[3., 3.], [-3., 3.], [3., -3.], [-3., -3.]])
    energy = lambda x: gmm_energy(x, means)
    u = Control(d); opt = torch.optim.Adam(u.parameters(), lr=lr)
    for it in range(steps):
        explore = 0.5 * max(0.0, 1 - it / (0.7 * steps))       # anneal extra exploration noise
        logw, _ = rollout_logw(u, energy, batch, d, explore=explore)
        loss = logw.var()                                      # log-variance loss
        opt.zero_grad(); loss.backward(); opt.step()
```

6.4. Beyond first-order dynamics

Underdamped samplers. Add a velocity, $\mathrm{d}X = V\,\mathrm{d}t$, $\mathrm{d}V = (\cdots)\,\mathrm{d}t + \sigma\,\mathrm{d}B$, and put the target on the pair, $\pi(x)\mathcal{N}(v;0,\mathrm{I})$. Noise enters only through the velocity, paths are smoother, and splitting integrators keep the discrete path densities exact. Blessing et al. (2025) show that underdamped samplers need far fewer steps for the same accuracy.

Langevin preconditioning. Writing $u_t = \alpha_t \nabla \log \tilde{\pi} + \tilde{u}_t$ (a learned correction on top of a Langevin drift) speeds up training, but every step then costs a gradient of $\mathcal{E}$, which is expensive for large energy models. Adjoint Sampling (Chapter 9) avoids this.

Generative-modelling view

A diffusion sampler is a diffusion model trained without data: same networks, same SDEs, same samplers. In a diffusion model the *noising* direction is fixed and the generative direction is learned from data; here the generative direction is the model, and the noising direction is fixed (PIS, DDS) or learned jointly (DIS). The path ELBO of Theorem 6.1 is the diffusion-model ELBO with the roles of model and data swapped. The log-variance loss is the VarGrad estimator of variational inference (Richter et al., 2020); trajectory balance comes from GFlowNets; and the whole set-up is maximum-entropy reinforcement learning with the drift as the policy and $-\mathcal{E}(X_1)$ as the final reward. Exploration noise, replay buffers and local search are RL tools for the same exploration problem.

Key takeaways

- A diffusion sampler is compared with a target *path* law ending at π; the log-weight of a discrete path is a sum of Gaussian log-ratios, and its mean is an ELBO (Theorem 6.1).
- PIS is exact because its base is memoryless; DDS has an exponentially small bias; learning both directions (DIS) is flexible but not identifiable.
- Log-variance and trajectory balance are off-policy, need no backpropagation through the simulation, and equal the KL gradient on-policy (Proposition 6.3). Exploration often matters more than the choice of loss.

Open problems and research questions

- **Energy cost.** Every training path needs at least one energy evaluation and often N gradient evaluations. How few energy calls can a diffusion sampler be trained with, and can paths be reused across many updates without bias?
- **Principled exploration.** Off-policy losses allow any behaviour policy, but there is no theory for choosing it. Can exploration be framed as an experimental-design or bandit problem over paths?
- **Discretisation bias.** Weights computed with Euler transitions are exact for the *discrete* chain; the continuous-time objective is only approximated. How should step sizes be adapted so that training and weights stay accurate?
- **Mode collapse with LV and TB.** LV covers modes only through its reference path law. When does on-policy LV collapse, and which references provably cover all modes?

CHAPTER 7

Annealed transport, Jarzynski weights and SMC hybrids

Goal	Follow a prescribed path of densities from μ to π with Langevin dynamics plus a learned transport drift; correct any lag with exact importance weights (Jarzynski); learn the drift by making the weights constant; and combine learned transitions with resampling.
Prerequisites	Chapter 2 (Feynman–Kac, Theorem 2.13); Theorem 4.1.
Key papers	Jarzynski (1997); Vaikuntanathan and Jarzynski (2008); Albergo and Vanden-Eijnden (2025); Vargas et al. (2024); Chen et al. (2025); Phillips et al. (2024); Skreta et al. (2025).

Overview. Instead of jumping from noise to π in one go, we can deform the density gradually: $\rho_0 = \mu$, then a slightly different ρ_t, and so on until $\rho_1 = \pi$. Particles that follow this deformation with Langevin dynamics lag behind it. There are two exact cures: *weight* the particles to account for the lag, or *add a drift* that moves them at the right speed. This chapter shows that they are two sides of one identity, and turns it into training objectives.

7.1. Annealing paths

Let $\rho_t(x) = \frac{e^{-U_t(x)}}{Z_t}$, $t \in [0,1]$, be a smooth path of densities with $\rho_0 = \mu$ tractable and $U_1 = \mathcal{E}$, so $\rho_1 = \pi$. The default is the geometric path $U_t = (1-t)U_0 + t\mathcal{E}$, i.e. $\rho_t \propto \mu^{1-t}\pi^t$; diffusion-style paths use Gaussian convolutions of π instead. Annealed Langevin dynamics

$$\mathrm{d}X_t = -\varepsilon_t \nabla U_t(X_t)\,\mathrm{d}t + \sqrt{2\varepsilon_t}\,\mathrm{d}B_t$$

would keep $X_t \sim \rho_t$ only if it could relax instantly to each new ρ_t (Proposition 2.5). In finite time it lags, and the density p_t of X_t differs from ρ_t.

7.2. The escorted Jarzynski equality

Theorem 7.1 (Escorted Jarzynski equality). Let b_t be any smooth vector field and $\varepsilon_t \geq 0$. Let $X_0 \sim \rho_0$ and

$$\mathrm{d}X_t = \big(-\varepsilon_t \nabla U_t(X_t) + b_t(X_t)\big)\,\mathrm{d}t + \sqrt{2\varepsilon_t}\,\mathrm{d}B_t, \tag{7.1}$$

$$\mathrm{d}A_t = \big(\nabla\cdot b_t - \nabla U_t \cdot b_t - \partial_t U_t\big)(X_t)\,\mathrm{d}t, \qquad A_0 = 0. \tag{7.2}$$

Then for all t and all bounded φ,

$$\mathbb{E}\left[e^{A_t}\varphi(X_t)\right] = \frac{Z_t}{Z_0}\,\mathbb{E}_{x\sim\rho_t}[\varphi(x)]. \tag{7.3}$$

Density form: the weighted density $f_t(x) = \mathbb{E}[e^{A_t} \mid X_t = x]\, p_t(x)$, where p_t is the density of X_t, equals $\frac{e^{-U_t(x)}}{Z_0} = \frac{Z_t}{Z_0}\rho_t(x)$. In particular $\mathbb{E}[e^{A_1}] = \frac{Z}{Z_0}$, and self-normalised weights e^{A_1} give consistent estimates of $\mathbb{E}_\pi[\varphi]$.

Intuition

A_t is a running log-weight (in physics, minus the dissipated work). The term $-\partial_t U_t$ pays for changing the target under a particle that stands still, as in AIS. The terms $\nabla\cdot b - \nabla U \cdot b$ pay for the transport b: they are the change of $\log \rho_t$ seen by a particle moving with b, plus the volume change. The Langevin part costs nothing, because it preserves the current ρ_t.

Proof. Let $c_t = \nabla\cdot b_t - \nabla U_t \cdot b_t - \partial_t U_t$ and $\mathcal{L}_t$ be the generator of Equation (7.1). By Itô's product rule, for a test function φ, $\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}[e^{A_t}\varphi(X_t)] = \mathbb{E}[e^{A_t}(c_t\varphi + \mathcal{L}_t\varphi)(X_t)]$. In terms of the weighted density f_t (defined so that $\int \varphi f_t = \mathbb{E}[e^{A_t}\varphi(X_t)]$), this says $\partial_t f_t = \mathcal{L}_t^* f_t + c_t f_t$, with $f_0 = \rho_0$. We check that $\tilde{f}_t = \frac{e^{-U_t}}{Z_0}$ solves this equation. On the left, $\partial_t \tilde{f} = -\partial_t U_t\, \tilde{f}$. On the right, using $\nabla \tilde{f} = -\nabla U_t\, \tilde{f}$,

$$\mathcal{L}_t^* \tilde{f} = \nabla\cdot(\varepsilon_t \nabla U_t\, \tilde{f}) - \nabla\cdot(b_t \tilde{f}) + \varepsilon_t \Delta \tilde{f} = -\varepsilon_t \Delta \tilde{f} - (\nabla\cdot b_t)\tilde{f} + (b_t \cdot \nabla U_t)\tilde{f} + \varepsilon_t \Delta \tilde{f},$$

so $\mathcal{L}_t^* \tilde{f} + c_t \tilde{f} = -\partial_t U_t\, \tilde{f}$. By uniqueness of solutions, $f_t = \tilde{f}_t$, which is Equation (7.3). ∎

Corollary 7.2 (Special cases). (i) $b = 0$: $A_1 = -\int_0^1 \partial_t U_t(X_t)\,\mathrm{d}t$, which is Jarzynski's equality (Jarzynski, 1997) and the continuous-time limit of AIS (Theorem 4.1). (ii) $\varepsilon = 0$: a deterministic flow, with $A_1 = \log \frac{\tilde{\pi}(X_1)}{e^{-U_0(X_0)}} + \int_0^1 \nabla\cdot b_t(X_t)\,\mathrm{d}t$, i.e. importance sampling with a continuous normalising flow (Corollary 1.4).

Proof. (i) is immediate. For (ii), along $\frac{\mathrm{d}X}{\mathrm{d}t} = b$, $\frac{\mathrm{d}}{\mathrm{d}t}U_t(X_t) = \partial_t U_t + \nabla U_t \cdot b_t$, so $A_1 = \int_0^1 \nabla\cdot b_t\,\mathrm{d}t - \big(U_1(X_1) - U_0(X_0)\big)$. ∎

Intuition

These are the two extremes of one identity: all noise and no transport (particles relax by Langevin dynamics; weights pay for the moving target), and all transport and no noise (a normalising flow; weights pay for volume changes). Learned samplers sit in between.

Proposition 7.3 (Zero-variance transport). The weight A_t is the same for every path (for any ε) if and only if b_t solves the continuity equation $\partial_t \rho_t + \nabla\cdot(\rho_t b_t) = 0$. Then $A_t = \log \frac{Z_t}{Z_0}$ and $X_t \sim \rho_t$ exactly.

Proof. For $\rho_t = \frac{e^{-U_t}}{Z_t}$, $\partial_t \rho_t = -(\partial_t U_t + \partial_t \log Z_t)\rho_t$ and $\nabla\cdot(\rho_t b_t) = (\nabla\cdot b_t - \nabla U_t \cdot b_t)\rho_t$. So the continuity equation holds if and only if $c_t(x) = \partial_t \log Z_t$ for all x, i.e. c_t does not depend on x; since X_t can be anywhere, this is equivalent to A_t being the same on all paths, with value $\int_0^t \partial_s \log Z_s\,\mathrm{d}s$. Exactness of the marginal densities is Theorem 2.13 with $v = b$ and $\frac{\sigma^2}{2} = \varepsilon$. ∎

Intuition

The Jarzynski weight measures exactly how much the transport b fails the continuity equation. So we can *learn* b by asking the weights to be constant, without knowing the normalising constants: $\partial_t \log Z_t$ is one unknown number per time, learned alongside b.

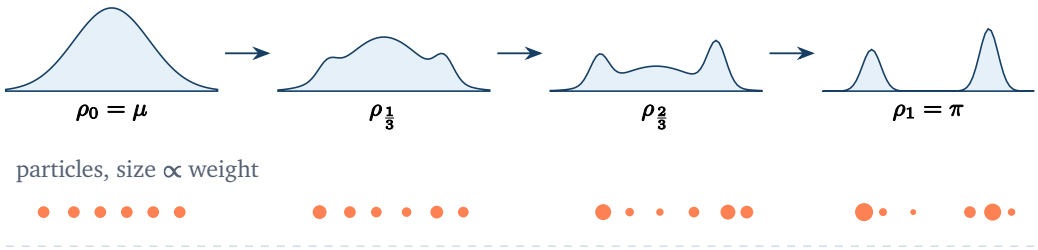


Figure 7.1 Annealed transport with Jarzynski weights. Top: the prescribed path of densities. Bottom: particles following it, drawn with size proportional to their weight. A perfect transport b keeps all weights equal (Proposition 7.3); otherwise the weights correct the lag.

7.3. Learning the transport: NETS and CMCD

NETS. The Non-Equilibrium Transport Sampler (Albergo and Vanden-Eijnden, 2025) learns b_θ and a scalar function $F_\phi(t) \approx -\log Z_t$ by minimising the residual of Proposition 7.3,

$$\mathcal{L}_{\mathrm{PINN}}(\theta, \phi) = \int_0^1 \mathbb{E}_{x\sim\nu_t}\left[\left|\nabla\cdot b_\theta - \nabla U_t \cdot b_\theta - \partial_t U_t + \partial_t F_\phi\right|^2(x)\right] \mathrm{d}t, \tag{7.4}$$

where ν_t can be *any* density that covers the space, e.g. that of the current particles of Equation (7.1), possibly resampled. The loss is off-policy and needs no backpropagation through the SDE; its minimisers give constant weights. Divergences are computed exactly in low dimension or with Proposition 5.1. The Langevin term ($\varepsilon > 0$) makes sampling robust to an imperfect b_θ, and the weights remove the remaining bias.

CMCD. Controlled Monte Carlo Diffusions (Vargas et al., 2024) use the same forward process Equation (7.1) and a backward process with drift $-\varepsilon_t \nabla U_t - b_t$ in reversed time, started at π. By Theorems 2.12 and 2.13, both have marginal densities ρ_t exactly when b solves the continuity equation, and then they are time reversals of each other. CMCD minimises a path divergence (KL or log-variance) between the two path laws, as in Chapter 6. In continuous time, the log-ratio of these two path laws is the Jarzynski weight A_1 up to the constant $\log \frac{Z}{Z_0}$ (Crooks' fluctuation theorem; Crooks, 2000; Vaikuntanathan and Jarzynski, 2008), so CMCD with log-variance and NETS minimise closely related functions of the same weights.

7.4. Sequential Monte Carlo with learned transitions

Weights become uneven when the transport is imperfect; the cure is resampling. All the following methods are SMC samplers (Del Moral et al., 2006) with learned transitions.

- **AFT and CRAFT** (Arbel et al., 2021; Matthews et al., 2022): normalising flows between consecutive annealing levels, used inside SMC.
- **SCLD** (Chen et al., 2025): controlled Langevin segments in the style of CMCD, with resampling between segments and a log-variance loss on sub-trajectories, which makes training stable and cheap.
- **PDDS** (Phillips et al., 2024): SMC along the DDS noising path with a learned approximation of the intermediate densities, consistent as the number of particles grows even with an imperfect network.
- **Feynman–Kac correctors** (Skreta et al., 2025): SMC that corrects a pretrained diffusion model when its path is modified, e.g. lowered in temperature or multiplied by another density;

Algorithm 2 Annealed SMC with learned transport and Jarzynski weights

Require: path U_t, transport b_θ, schedule ε_t, grid $0 = t_0 < \cdots < t_K = 1$, N particles, threshold $\kappa \in (0,1)$

draw $X^i \sim \rho_0$; set $\log w^i \leftarrow 0$ and $\log \hat{Z} \leftarrow \log Z_0$
for $k = 0, \dots, K-1$ **do**
  move each particle with Euler–Maruyama on Equation (7.1) from t_k to t_{k+1}; add the increment of Equation (7.2) to $\log w^i$
  if $\mathrm{ESS}(w) < \kappa N$ **then**
    $\log \hat{Z} \leftarrow \log \hat{Z} + \log\left(\frac{1}{N}\sum_i w^i\right)$; resample the X^i with probabilities $\propto w^i$; reset $\log w^i \leftarrow 0$
  end if
  optionally: a few MALA steps targeting $\rho_{t_{k+1}}$ (exact, since they preserve $\rho_{t_{k+1}}$)
end for
$\log \hat{Z} \leftarrow \log \hat{Z} + \log\left(\frac{1}{N}\sum_i w^i\right)$; **return** the weighted particles and $\log \hat{Z}$

used by Akhound-Sadegh et al. (2025) to cool a diffusion model trained at high temperature down to the target.

- **Sequential Boltzmann generators** (Chapter 5): SMC from a flow's density to π.

Generative-modelling view

AIS is the standard way to estimate the log-likelihood of VAEs and energy-based models, and annealing paths are familiar in ML: likelihood tempering in Bayesian inference ($p(\theta)\,p(\mathcal{D}|\theta)^\beta$ with β from 0 to 1), noise schedules in diffusion models, and guidance schedules. When a diffusion model is run with a modified path (classifier-free guidance, a temperature change, a product of two models), it no longer samples what it seems to; the Jarzynski weights of Theorem 7.1 and SMC resampling are what make such modifications exact (Section 12.3).

Key takeaways

- The escorted Jarzynski equality (Theorem 7.1) makes *any* transport exact after weighting; the weights are constant exactly when the transport solves the continuity equation (Proposition 7.3).
- Learning the transport by making the weights constant (NETS) or the path laws agree (CMCD) is off-policy and needs no target samples. Resampling (SMC) makes the result robust to an imperfect network.

Open problems and research questions

- **Choice of path.** The geometric path can force mass to "teleport" between distant modes when their relative weights change along the path, which no local transport can do cheaply. How should the path be learned jointly with the transport?
- **Cost of exact weights.** The weights need $\nabla \cdot b$ and $\partial_t U_t$ along every path. Cheap divergence estimators that do not spoil the weights are needed in high dimension.

- **Variance in high dimension.** Even a near-perfect transport gives log-weights whose variance grows with dimension. How must the number of steps and particles scale to keep SMC stable for proteins?
- **Combining with control.** Annealed references can be put inside the control and bridge framework (Section 10.2); when is this better than SMC correction?

CHAPTER 8

Energy- and score-matching samplers

Goal	Train a diffusion model for π without data by regressing its score (or its noised energy) onto Monte Carlo estimates built from $\mathcal{E}$; understand exactly where these estimates fail; survey the remedies.
Prerequisites	Sections 2.7 and 2.8; Section 1.5.
Key papers	Akhound-Sadegh et al. (2024); OuYang et al. (2026); De Bortoli et al. (2024); He et al. (2025); Skreta et al. (2025); Havens et al. (2026).

Overview. A diffusion model generates samples by running a noising process backwards (Theorem 2.12); all it needs is the score $\nabla \log p_t$ of the noised data density. Here there are no data, but the noised score can still be written as an average involving only $\mathcal{E}$. This chapter turns that average into a regression target, and explains why the estimate is good at low noise and very poor at high noise.

8.1. Setting

Add Gaussian noise to a sample of π: $X_t = X_0 + \sigma_t \xi$ with $X_0 \sim \pi$, $\xi \sim \mathcal{N}(0, \mathrm{I})$, $\sigma_0 = 0$, and σ_1 larger than the spread of π. The density of X_t is the convolution

$$p_t(x) = \int \pi(y)\, \mathcal{N}(x; y, \sigma_t^2 \mathrm{I})\, \mathrm{d}y,$$

and by Theorem 2.12 the reverse SDE

$$\mathrm{d}Y_s = \left(\frac{\mathrm{d}\sigma_t^2}{\mathrm{d}t}\right)_{t=1-s} \nabla \log p_{1-s}(Y_s)\, \mathrm{d}s + \sqrt{\left(\frac{\mathrm{d}\sigma_t^2}{\mathrm{d}t}\right)_{t=1-s}}\, \mathrm{d}\bar{B}_s, \qquad Y_0 \sim p_1 \approx \mathcal{N}(0, \sigma_1^2 \mathrm{I}), \tag{8.1}$$

ends at π. Diffusion models learn $\nabla \log p_t$ from samples of π. Here we must learn it from $\mathcal{E}$.

8.2. Three expressions for the noised score

Proposition 8.1 (Noised score identities). For every x and $t > 0$:

(a) (Tweedie) $\nabla \log p_t(x) = \frac{\mathbb{E}[X_0 \mid X_t = x] - x}{\sigma_t^2}$;

(b) (target score identity) $\nabla \log p_t(x) = \mathbb{E}[\nabla \log \pi(X_0) \mid X_t = x] = -\mathbb{E}[\nabla \mathcal{E}(X_0) \mid X_t = x]$;

(c) (importance-sampling form) with $Y \sim \mathcal{N}(x, \sigma_t^2 \mathrm{I})$,

$$\nabla \log p_t(x) = \frac{\mathbb{E}\left[-\nabla \mathcal{E}(Y)\, e^{-\mathcal{E}(Y)}\right]}{\mathbb{E}\left[e^{-\mathcal{E}(Y)}\right]}, \qquad -\log p_t(x) = -\log \mathbb{E}\left[e^{-\mathcal{E}(Y)}\right] + \log Z. \tag{8.2}$$

Here the conditional expectations are over the density of X_0 given $X_t = x$, which is $p_{0|t}(y \mid x) = \frac{\pi(y)\, \mathcal{N}(x; y, \sigma_t^2 \mathrm{I})}{p_t(x)}$.

Proof. Since $\mathcal{N}(x; y, \sigma_t^2 \mathrm{I}) = \mathcal{N}(y; x, \sigma_t^2 \mathrm{I})$, the posterior $p_{0|t}(\cdot|x)$ is proportional to $\tilde{\pi}(y)\,\mathcal{N}(y; x, \sigma_t^2 \mathrm{I})$, i.e. to the density of Y reweighted by $e^{-\mathcal{E}(Y)}$; so (b) and (c) express the same average, (c) as importance sampling with proposal Y. For (c), $Z\,p_t(x) = \mathbb{E}[\tilde{\pi}(x + \sigma_t \xi)]$; differentiating in x under the expectation gives $Z \nabla p_t(x) = \mathbb{E}[\nabla \tilde{\pi}(Y)] = \mathbb{E}[-\nabla \mathcal{E}(Y) \tilde{\pi}(Y)]$, and dividing by $Z\,p_t(x)$ gives (c). For (a), $\nabla_x \mathcal{N}(x; y, \sigma_t^2 \mathrm{I}) = \frac{y-x}{\sigma_t^2} \mathcal{N}(x; y, \sigma_t^2 \mathrm{I})$; differentiate $p_t(x)$ under the integral and divide by $p_t(x)$. ∎

Intuition

Tweedie says: the score points from x towards where the clean sample probably came from. The target score identity says: the score is the average clean force over those likely origins. Both need the posterior of X_0 given $X_t = x$, which is easy when σ_t is small (concentrated near x) and hard when σ_t is large (it is then almost the whole target, reweighted).

8.3. Where the estimates fail

Replacing the expectations in Equation (8.2) by averages over K samples $Y_k \sim \mathcal{N}(x, \sigma_t^2 \mathrm{I})$ gives a self-normalised importance sampler for the posterior, with proposal $\mathcal{N}(x, \sigma_t^2 \mathrm{I})$. Its quality is set by the ESS (Proposition 1.11).

Proposition 8.2 (Exponential degradation at high noise). Let $\pi = \mathcal{N}(0, s^2 \mathrm{I})$. The posterior of X_0 given $X_t = x$ is $\mathcal{N}(m, \tau^2 \mathrm{I})$ with $\tau^2 = \frac{s^2 \sigma_t^2}{s^2 + \sigma_t^2}$. Even if the proposal is centred at the posterior mean, i.e. $\mathcal{N}(m, \sigma_t^2 \mathrm{I})$ (the most favourable case), the asymptotic normalised ESS is

$$\frac{\mathrm{ESS}}{K} \to \left(\frac{\sqrt{2r-1}}{r}\right)^d \le \left(\frac{2}{r}\right)^{\frac{d}{2}}, \qquad r = \frac{\sigma_t^2}{\tau^2} = 1 + \frac{\sigma_t^2}{s^2}.$$

Proof. By Proposition 1.11, $\frac{\mathrm{ESS}}{K} \to \left(\int \frac{p^2}{q}\right)^{-1}$ with $p = \mathcal{N}(m, \tau^2)$ and $q = \mathcal{N}(m, \sigma_t^2)$ in each coordinate. A Gaussian integral gives, per coordinate,

$$\int \frac{p^2}{q} = \frac{\sigma_t}{\tau^2 \sqrt{2\pi}} \int_{\mathbb{R}} e^{-z^2 \left(\frac{1}{\tau^2} - \frac{1}{2\sigma_t^2}\right)} \mathrm{d}z = \frac{\sigma_t^2}{\tau \sqrt{2\sigma_t^2 - \tau^2}} = \frac{r}{\sqrt{2r-1}}.$$

Coordinates are independent, so the d-dimensional value is the d-th power; and $\sqrt{2r-1} \le \sqrt{2r}$. ∎

Intuition

At high noise the Gaussian proposal around x is far wider than the posterior, so almost all proposals land where the target is negligible. The fraction that land in the right place shrinks like the ratio of widths to the power d: the curse of dimensionality for importance sampling in its plainest form.

So once σ_t exceeds the scale of the target, the number of samples K needed grows exponentially with the dimension. Table 8.1 shows the effect on a 2-D mixture using Listing 8.1.

Listing 8.1 Monte Carlo targets for the noised score (iDEM) and the noised energy (NEM).

```python
import math, torch

```

Table 8.1 Monte Carlo score estimate Equation (8.2) with $K = 500$ on a 4-mode mixture (component standard deviation 0.5), at 200 points $x \sim \mathcal{N}(0, 9\mathrm{I})$: relative L^2 error against the exact noised score, and mean ESS.

σ_t	0.1	1.0	3.0	10.0
relative error	0.006	0.53	2.8	20.4
mean ESS (out of 500)	244	44	34	8.8

```python
def idem_score(x, sigma_t, energy, K=500):
    """Monte Carlo estimate of grad log p_t(x), p_t = pi * N(0, sigma_t^2 I) (Prop. 8.1(c)).
    Self-normalised IS over y_k ~ N(x, sigma_t^2 I) with weights exp(-E(y_k))."""
    y = x[:, None, :] + sigma_t * torch.randn(x.shape[0], K, x.shape[1])      # (B, K, d)
    y.requires_grad_(True)
    E = energy(y.reshape(-1, x.shape[1])).reshape(x.shape[0], K)
    gradE = torch.autograd.grad(E.sum(), y)[0]                                 # (B, K, d)
    w = torch.softmax(-E.detach(), dim=1)                                      # normalised
     weights
    ess = 1.0 / (w**2).sum(1)                                                  # per-point ESS
    return -(w[..., None] * gradE).sum(1), ess

def idem_energy(x, sigma_t, energy, K=500):
    """NEM target: E_t(x) = -log E_{y ~ N(x, sigma_t^2)}[exp(-E(y))] (biased upwards, Jensen)."""
    y = x[:, None, :] + sigma_t * torch.randn(x.shape[0], K, x.shape[1])
    E = energy(y.reshape(-1, x.shape[1])).reshape(x.shape[0], K)
    return -(torch.logsumexp(-E, dim=1) - math.log(K))
```

8.4. Iterated Denoising Energy Matching (iDEM)

iDEM (Akhound-Sadegh et al., 2024) trains a score network $s_\theta(x, t)$ with the loss

$$\mathcal{L}_{\mathrm{iDEM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],\; x_0\sim\mathcal{B},\; x_t = x_0 + \sigma_t \xi} \left[\left| s_\theta(x_t, t) - S_K(x_t, t) \right|^2 \right], \tag{8.3}$$

where S_K is the K-sample estimate of the score in Equation (8.2) and $\mathcal{B}$ is a replay buffer of the sampler's own outputs.

Algorithm 3 iDEM (two nested loops)

Require: energy $\mathcal{E}$, score network s_θ, schedule σ_t, buffer $\mathcal{B}$ (initially samples of $\mathcal{N}(0, \sigma_1^2 \mathrm{I})$)

loop
  outer: simulate Equation (8.1) with s_θ in place of $\nabla \log p_t$; add the endpoints to $\mathcal{B}$
  for $m = 1, \dots, M$ **do** ▷ inner, no simulation
    draw $x_0 \sim \mathcal{B}$, $t \sim \mathcal{U}[0, 1]$, $x_t = x_0 + \sigma_t \xi$; compute $S_K(x_t, t)$ (K calls to $\nabla \mathcal{E}$)
    gradient step on $|s_\theta(x_t, t) - S_K(x_t, t)|^2$
  end for
end loop

Intuition

The regression target $S_K(x,t)$ is defined at *every* point x, whatever the buffer contains. The buffer only decides where the network is trained. As the sampler improves, the buffer moves to where the noised target has mass, and the network becomes accurate where it is used.

The weaknesses follow from Proposition 8.2: at large σ_t the targets are noisy and biased (the self-normalisation bias is of order $\frac{1}{K}$, with a large constant), which hurts the early part of the reverse SDE, exactly the part that decides how much mass goes to each mode. In practice $\nabla\mathcal{E}$ is clipped, which adds bias for steep energies.

8.5. Noised energy matching and bootstrapping

NEM. OuYang et al. (2026) regress a noised *energy* network $E_\theta(x,t)$ onto $\hat{E}_K(x,t) = -\log\left(\frac{1}{K}\sum_k e^{-\mathcal{E}(Y_k)}\right)$, and use $s_\theta = -\nabla_x E_\theta$ as the score. The energy target is smoother and less noisy than the score target, at the cost of a bias from Jensen's inequality, $\mathbb{E}[\hat{E}_K] \geq -\log\mathbb{E}[e^{-\mathcal{E}(Y)}]$, of size about $\frac{1}{2K}\frac{\mathrm{Var}[e^{-\mathcal{E}(Y)}]}{\mathbb{E}[e^{-\mathcal{E}(Y)}]^2}$.

Proposition 8.3 (Bootstrap identity). For $0 \leq s < t$ and $Y \sim \mathcal{N}\big(x, (\sigma_t^2 - \sigma_s^2)\mathrm{I}\big)$, $p_t(x) = \mathbb{E}[p_s(Y)]$; equivalently, with $E_t = -\log p_t$, $E_t(x) = -\log\mathbb{E}\left[e^{-E_s(Y)}\right]$.

Proof. Gaussian noise adds up: $p_t = \pi * \mathcal{N}(0,\sigma_t^2\mathrm{I}) = \big(\pi * \mathcal{N}(0,\sigma_s^2\mathrm{I})\big) * \mathcal{N}(0,(\sigma_t^2-\sigma_s^2)\mathrm{I}) = p_s * \mathcal{N}(0,(\sigma_t^2-\sigma_s^2)\mathrm{I})$. ∎

Intuition

A lot of noise is the same as a little noise added many times. So a hard estimation problem at high noise can be split into a chain of easy ones, each using the previous noise level as its target, at the price of accumulating the error of each link.

BNEM uses Proposition 8.3 with the learned $E_\theta(\cdot,s)$ in place of E_s to build targets at high noise from lower ones; since $\sigma_t^2 - \sigma_s^2$ is small, the importance sampler is efficient. Bootstrapped targets are used only where they are estimated to be reliable.

8.6. Temperature, interpolants and other remedies

- **Target score matching** (De Bortoli et al., 2024) uses identity (b) at low noise, where it is accurate.
- **Temperature annealing.** At high temperature the target is smooth and nearly unimodal, so samplers are easy to train. Progressive schemes train at high temperature and cool down, using the previous model as a proposal plus a correction: SMC with Feynman–Kac correctors (Skreta et al., 2025; Akhound-Sadegh et al., 2025) or tempering-style buffers (Rissanen et al., 2025).
- **Energy-based flow matching** (Woo and Ahn, 2024) regresses a flow-matching velocity instead of a score, with similar importance-sampling targets.

- **Flow Sampling** (Havens et al., 2026) conditions on a noise sample rather than a data sample and regresses onto a denoising drift built from the energy, using an interpolant to reduce the number of energy evaluations; it extends to spheres and hyperbolic spaces (Chapter 11).
- **One-step samplers** train a generator directly by moving its samples along a smoothed score field (Cao et al., 2026); so far mainly demonstrated in low dimension.

A note of caution. He et al. (2025) re-examine simulation-free samplers including iDEM and report that their performance depends strongly on implementation details, and that their advantage over simpler, well-tuned baselines is smaller than first reported. Read benchmark tables in this sub-field with care (Section 4.4).

Generative-modelling view

Compare with denoising score matching. There, the pair (X_0, X_t) is made by noising a data point, so the posterior of X_0 given X_t is sampled for free and the regression target $\frac{X_0 - X_t}{\sigma_t^2}$ costs nothing. Here, X_0 given X_t must be sampled by importance sampling from $\mathcal{N}(X_t, \sigma_t^2 \mathrm{I})$ with weights $e^{-\mathcal{E}}$, and the target is a weighted average of forces. iDEM is score matching in which the data are replaced by self-normalised importance samples. The Tweedie estimate is also used for guidance in diffusion posterior sampling (Section 12.3), with the same weakness at high noise.

Key takeaways

- The noised score is an average of the clean force over plausible origins (Proposition 8.1); estimating it with Gaussian proposals is accurate at low noise and exponentially hard at high noise (Proposition 8.2).
- iDEM, NEM and BNEM differ in what they regress (score, energy, bootstrapped energy) and therefore in their bias and variance. None provides importance weights by default.
- Most remedies make high-noise targets easier: temperature annealing, bootstrapping, or interpolants.

Open problems and research questions

- **High-noise targets.** Design estimates of $\nabla \log p_t$ at large σ_t whose cost does not grow exponentially with d, e.g. by MCMC on the posterior of X_0 given X_t (as in stochastic localisation, Grenioux et al., 2024) or with learned proposals.
- **Convergence of buffer-based training.** There is no theory for when the iDEM loop converges to π rather than to a collapsed fixed point.
- **Bias accounting.** Score and energy targets are biased (self-normalisation, Jensen, clipping); how do these biases propagate through the reverse SDE to the final density?
- **Energy budget.** Each target costs K energy-gradient calls. Can targets be shared across noise levels or points without losing consistency?

CHAPTER 9

Adjoint Sampling

Goal	Derive Adjoint Sampling from first principles: a control problem whose optimal drift is the average of a single endpoint gradient, where the optimal paths given their endpoints are exactly known Brownian bridges. Understand why it needs only one energy evaluation per sample, how it is implemented, and where it fails.
Prerequisites	Sections 3.3 and 3.4; Lemma A.2.
Key papers	Havens et al. (2025); Domingo-Enrich et al. (2025); Zhang and Chen (2022).

Overview. Adjoint Sampling combines three facts from Chapter 3. (1) If Brownian motion starts at a point and its paths are reweighted by $\frac{\tilde{\pi}(X_1)}{p_1^0(X_1)}$, the reweighted paths end at π (Theorem 3.4). (2) The drift that generates the reweighted paths is the average, over reweighted paths through the current point, of the gradient of the terminal cost at the endpoint (Corollary 3.9). (3) Given its endpoint, a reweighted path is just a Brownian bridge. Together they turn sampling into a regression whose targets cost one energy gradient each.

9.1. Why another diffusion sampler?

When every evaluation of $\mathcal{E}$ or $\nabla\mathcal{E}$ is a call to a large neural network, the samplers of Chapter 6 are expensive: they spend up to N gradient evaluations per training path, or backpropagate through N network calls; energy-matching samplers spend K evaluations per regression target. Adjoint Sampling (AS) (Havens et al., 2025) is designed so that *each energy-gradient evaluation serves many gradient updates*: it evaluates $\nabla\mathcal{E}$ only at endpoints, stores them, and turns each stored endpoint into regression targets at all intermediate times.

9.2. The control problem

The base process is $\mathrm{d}X_t = \sigma_t\,\mathrm{d}B_t$ started at the point $X_0 = x_0$ (usually 0). Its path law is $\mathbb{P}^0$; its time-1 density is $p_1^0 = \mathcal{N}(x_0, \bar{\sigma}_1^2\mathrm{I})$. The sampler is $\mathrm{d}X_t = \sigma_t u_t(X_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t$ from the same point, with path law $\mathbb{P}^u$. By Theorem 3.4, minimising $J(u) = \mathbb{E}_{\mathbb{P}^u}\left[\int_0^1 \frac{1}{2}|u_t(X_t)|^2\,\mathrm{d}t + g(X_1)\right]$ with terminal cost

$$g(x) = \mathcal{E}(x) + \log p_1^0(x) = \mathcal{E}(x) - \frac{|x - x_0|^2}{2\bar{\sigma}_1^2} + \text{const} \tag{9.1}$$

gives a sampler whose output density is exactly π; the point start makes the base memoryless. Since the base drift is zero, Corollary 3.9 gives

$$u_t^\star(x) = -\sigma_t\,\mathbb{E}_{\mathbb{P}^\star}\left[\nabla g(X_1)\,\middle|\,X_t = x\right]. \tag{9.2}$$

This is already a regression problem, but under the unknown optimal path law $\mathbb{P}^\star$. The key observation is that $\mathbb{P}^\star$ is known *once its endpoint is known*.

Lemma 9.1 (Reciprocal property). $\mathbb{P}^\star(\cdot \mid X_1 = y) = \mathbb{P}^0(\cdot \mid X_1 = y)$ for all y: given its endpoint, an optimal path is a base bridge. **Density form:** the density of X_t given $X_1 = y$ under $\mathbb{P}^\star$ is

$$p^\star_{t|1}(x \mid y) = p^0_{t|1}(x \mid y) = \mathcal{N}\Big(x;\ x_0 + \frac{\bar\sigma_t^2}{\bar\sigma_1^2}(y - x_0),\ \frac{\bar\sigma_t^2(\bar\sigma_1^2 - \bar\sigma_t^2)}{\bar\sigma_1^2}\mathrm{I}\Big).$$

Proof. $\mathbb{P}^\star$ is $\mathbb{P}^0$ reweighted by $\frac{e^{-g(X_1)}}{Z}$ (Theorem 3.2). Reweighting by a function of X_1 changes the density of X_1 but not the conditional law of the path given X_1. The base process is a time-changed Brownian motion from x_0, whose bridge marginals are given by Lemma A.2. ∎

Intuition

The energy acts only at the final time. Once we know where a path ended, the energy has nothing more to say about how it got there, so the path is as random as a base path with the same endpoints: a Brownian bridge. All the difficulty of sampling is concentrated in the density of the endpoint.

Combining Equation (9.2) with Lemma 9.1: $u^\star$ minimises

$$\mathbb{E}_{t\sim\mathcal{U}[0,1]}\ \mathbb{E}_{y\sim\pi}\ \mathbb{E}_{x\sim p^0_{t|1}(\cdot|y)}\Big[\big|u_t(x) + \sigma_t\nabla g(y)\big|^2\Big],$$

which would be a closed-form regression if we could sample $y \sim \pi$. We cannot, so we use the current sampler's endpoints instead.

9.3. Reciprocal Adjoint Matching

Definition 9.2 (Reciprocal projection and RAM loss). For a control $\bar u$ with output density $p_1^{\bar u}$, the *reciprocal projection* $\Pi^{\bar u}$ is the path law that keeps the sampler's endpoint density and replaces its paths by base bridges: $X_1 \sim p_1^{\bar u}$, then the path given X_1 is drawn from $\mathbb{P}^0(\cdot \mid X_1)$. In density form, the pair (X_t, X_1) has density $p_1^{\bar u}(y)\, p^0_{t|1}(x \mid y)$. The Reciprocal Adjoint Matching loss is

$$\mathcal{L}_{\mathrm{RAM}}(u;\bar u) = \mathbb{E}_{t\sim\mathcal{U}[0,1]}\ \mathbb{E}_{y\sim p_1^{\bar u}}\ \mathbb{E}_{x\sim p^0_{t|1}(\cdot|y)}\Big[\big|u_t(x) + \sigma_t\nabla g(y)\big|^2\Big], \qquad \bar u = \mathrm{sg}(u). \tag{9.3}$$

Theorem 9.3 (The optimal control is a fixed point). For any $\bar u$, the minimiser of $\mathcal{L}_{\mathrm{RAM}}(\cdot\,;\bar u)$ over all functions is $\mathcal{T}(\bar u)_t(x) = -\sigma_t\, \mathbb{E}_{\Pi^{\bar u}}[\nabla g(X_1) \mid X_t = x]$. The optimal control satisfies $\mathcal{T}(u^\star) = u^\star$.

Proof. The first claim is least squares: for each t, the best function of X_t is the conditional expectation. For the second, $p_1^{u^\star} = \pi = p_1^\star$, so by Lemma 9.1 the reciprocal projection of $\mathbb{P}^{u^\star}$ is $\mathbb{P}^\star$ itself, and Equation (9.2) gives $\mathcal{T}(u^\star) = u^\star$. ∎

Intuition

Each iteration asks: *if the paths ending at my current samples were optimal (that is, base bridges), which control would generate them?* The answer pushes each bridge point in the direction that lowers the terminal cost. Endpoints that are too frequent compared with π produce a ∇g pointing away from them, so their region is emptied; under-represented regions attract mass. Havens et al. (2025) argue that $u^\star$ is the only fixed point under suitable conditions; a general convergence theory is open.

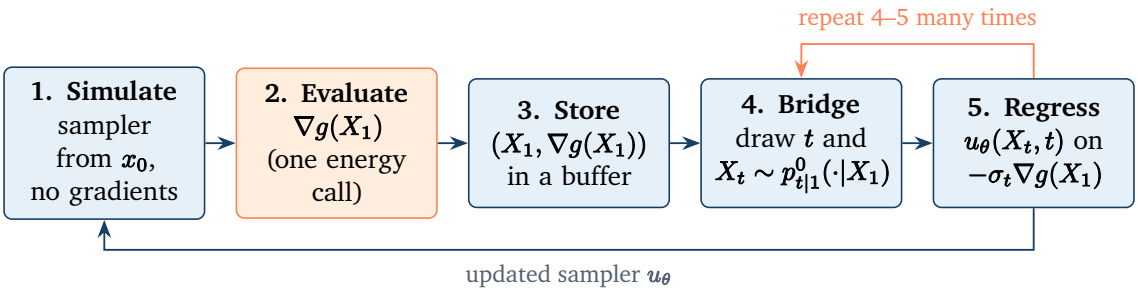


Figure 9.1 Adjoint Sampling. Energy gradients are evaluated only at simulated endpoints (step 2); each stored endpoint generates fresh regression targets at all times through exact Brownian bridges (steps 4–5).

9.4. The algorithm

Algorithm 4 Adjoint Sampling

Require: energy $\mathcal{E}$, noise schedule σ_t with $\bar{\sigma}_t^2 = \int_0^t \sigma_s^2 \, \mathrm{d}s$, network u_θ, buffer $\mathcal{B}$

loop
  simulate n paths of $\mathrm{d}X_t = \sigma_t \, \mathrm{sg}(u_\theta)(X_t, t) \, \mathrm{d}t + \sigma_t \, \mathrm{d}B_t$ from $X_0 = x_0$ ▷ no backpropagation
  compute $\nabla g(X_1^i) = \nabla \mathcal{E}(X_1^i) - \frac{X_1^i - x_0}{\bar{\sigma}_1^2}$ and add $(X_1^i, \nabla g(X_1^i))$ to $\mathcal{B}$ ▷ n energy calls
  for $m = 1, \dots, M$ **do**
    draw $(X_1, \nabla g(X_1))$ from $\mathcal{B}$, $t \sim \mathcal{U}[0,1]$, and X_t from the bridge of Lemma 9.1
    gradient step on $|u_\theta(X_t, t) + \sigma_t \nabla g(X_1)|^2$
  end for
end loop

Implementation choices.

- **Noise schedule.** A geometric schedule $\sigma_t = \sigma_{\min} \left(\frac{\sigma_{\max}}{\sigma_{\min}}\right)^{1-t} \sqrt{2 \log \frac{\sigma_{\max}}{\sigma_{\min}}}$ gives $\bar{\sigma}_t^2 = \sigma_{\max}^2 \left(1 - \left(\frac{\sigma_{\min}}{\sigma_{\max}}\right)^{2t}\right)$: large noise early (exploration), small noise late (refinement). $\bar{\sigma}_1$ must be large enough for p_1^0 to cover π, but not much larger, because large noise makes the dynamics stiff (Section 12.4).
- **Parameterisation.** Writing $u_\theta = -\sigma_t f_\theta$ turns the loss into $|f_\theta(X_t, t) - \nabla g(X_1)|^2$, a regression with targets of the same size at all times.
- **Gradient clipping.** Steep energies (e.g. repulsive cores) make $|\nabla \mathcal{E}|$ huge; clipping ∇g stabilises training at the cost of bias near those regions.
- **Buffer.** A first-in-first-out buffer of recent endpoints. Old entries came from older controls, which slightly biases the reciprocal projection; small buffers and frequent refreshes reduce this.
- **Importance weights for free.** AS is a controlled SDE from a point, so the PIS log-weight Equation (6.2) can be computed along any simulated path at the cost of one energy evaluation; this allows reweighting or SMC at test time (Chapter 7 and Section 12.4).

Listing 9.1 Adjoint Sampling in PyTorch (2-D version of Algorithm 4).

```
import math, torch, torch.nn as nn

```

```python
# ---- base process dX = sigma(t) dB from X_0 = 0, geometric schedule (Section 9.4) ----
S_MIN, S_MAX = 0.05, 5.0
L = math.log(S_MAX / S_MIN)
def sigma(t):  return S_MIN * (S_MAX / S_MIN) ** (1 - t) * math.sqrt(2 * L)
def sbar2(t):  return S_MAX**2 * (1 - torch.exp(-2 * L * t))     # int_0^t sigma^2 ds
SB1 = S_MAX**2 * (1 - math.exp(-2 * L))                          # sbar2(1)

class Net(nn.Module):        # f_theta(x, t);  control u = -sigma(t) * f_theta
    def __init__(self, d, h=128):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(d + 1, h), nn.SiLU(), nn.Linear(h, h), nn.SiLU(),
                                 nn.Linear(h, h), nn.SiLU(), nn.Linear(h, d))
    def forward(self, x, t):
        return self.net(torch.cat([x, t.expand(x.shape[0], 1) if t.dim() == 2 else t], -1))

@torch.no_grad()
def sample(f, n, d, N=200):
    """Euler-Maruyama for dX = sigma u dt + sigma dB with u = -sigma f."""
    x, dt = torch.zeros(n, d), 1.0 / N
    for k in range(N):
        t = torch.tensor([[k * dt]]); s = sigma(k * dt)
        x = x - s**2 * f(x, t) * dt + s * math.sqrt(dt) * torch.randn(n, d)
    return x

def grad_terminal_cost(x, energy, clip=50.0):
    """grad g = grad E + grad log p_1^base = grad E - x / sbar2(1)  (Eq. 9.1)."""
    x = x.clone().requires_grad_(True)
    gE = torch.autograd.grad(energy(x).sum(), x)[0]
    g = gE - x / SB1
    return (g * (clip / g.norm(dim=1, keepdim=True).clamp(min=clip))).detach()  # clip norm

def train(energy, d=2, outer=100, inner=50, n_new=512, batch=512, buf_max=20000, lr=1e-3,
          clip=50.0):
    f = Net(d); opt = torch.optim.Adam(f.parameters(), lr=lr)
    buf_x, buf_g = torch.empty(0, d), torch.empty(0, d)
    for _ in range(outer):
        x1 = sample(f, n_new, d)                                  # 1) simulate, no grad
        buf_x = torch.cat([buf_x, x1])[-buf_max:]                 # 2) buffer endpoints and
        buf_g = torch.cat([buf_g, grad_terminal_cost(x1, energy, clip)])[-buf_max:]  # their grad
        g
        for _ in range(inner):                                    # 3) reciprocal adjoint matching
            i = torch.randint(len(buf_x), (batch,))
            x1, g1 = buf_x[i], buf_g[i]
            t = torch.rand(batch, 1) * 0.99 + 0.005
            m = sbar2(t) / SB1                                    # Brownian bridge 0 -> x1
            xt = m * x1 + torch.sqrt(sbar2(t) * (1 - m)) * torch.randn_like(x1)
            loss = ((f(xt, t) - g1) ** 2).sum(-1).mean()          # u + sigma grad g, rescaled
            opt.zero_grad(); loss.backward(); opt.step()
```

9.5. Symmetries and molecular systems

- **Translations.** Start at $x_0 = 0$ in the subspace of configurations with zero centre of mass and project the noise onto it ($B \mapsto PB$ with $P = (\mathrm{I}_n - \frac{1}{n}\mathbf{1}\mathbf{1}^\top) \otimes \mathrm{I}_3$). The base is then Brownian motion on a $(3n-3)$-dimensional subspace, so $\log p_1^0$ uses that dimension (Proposition 11.7).
- **Rotations and permutations.** The base is invariant; a rotation- and permutation-equivariant network makes u_θ equivariant, so all marginal densities are invariant (Proposition 11.6).

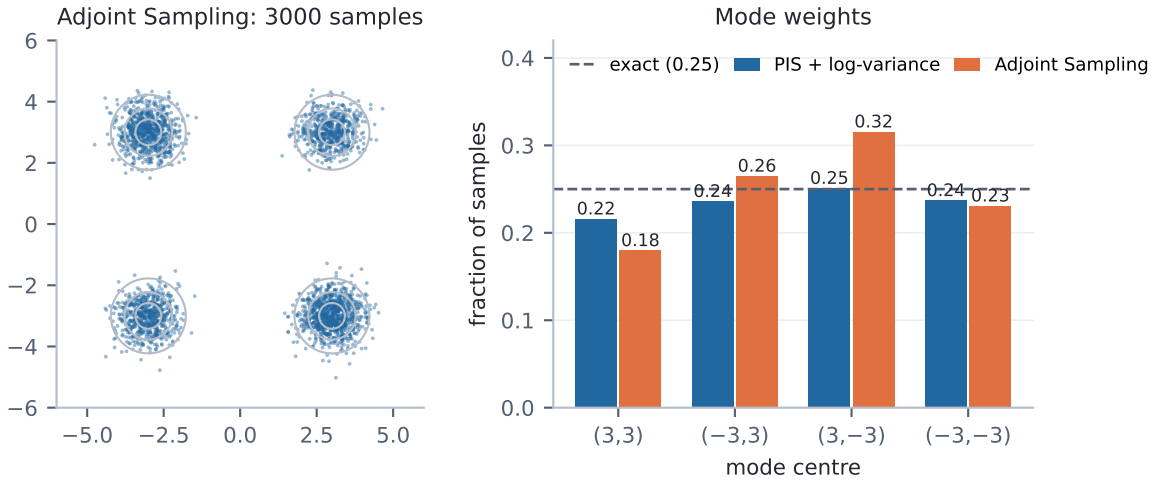


Figure 9.2 Left: samples from Listing 9.1 after 300×512 energy-gradient evaluations on a 4-mode mixture. Right: mode weights for Adjoint Sampling and for the PIS sampler trained with the log-variance loss (Listing 6.1). Adjoint Sampling finds every mode but misweights them (0.18 to 0.32); the log-variance sampler, which used far more energy evaluations (2000×256 paths) and exploration noise, is closer to the exact weights.

- **Amortisation.** Conditioning u_θ on the molecular graph trains one sampler for many molecules; Havens et al. (2025) train on thousands of SPICE molecules with the eSEN potential and generate conformers for unseen ones (Chapter 13).
- **Other geometries.** Torsion angles and periodic boxes live on tori, where the same derivation holds with wrapped bridges (Section 11.6).

9.6. Relations to other methods

- **PIS** (Chapter 6) solves the same control problem with the same optimum, by backpropagating through simulations; AS replaces this by regression, which is far cheaper per energy call.
- **Adjoint Matching** (Domingo-Enrich et al., 2025) is the general version with a non-zero base drift, where the adjoint Equation (3.10) must be integrated backwards along each path; with zero drift it is constant and the bridge trick becomes available.
- **Energy matching** (Chapter 8) also regresses onto targets built from $\nabla\mathcal{E}$, but its targets are importance-sampling estimates, noisy at high noise; AS targets are exact for each path, and the only approximation is in the density of X_1.

Generative-modelling view

Adjoint Sampling is the data-free cousin of flow matching. Flow matching regresses a velocity on pairs (noise, data) joined by simple interpolants; AS regresses a control on pairs (bridge point, endpoint gradient) joined by Brownian bridges, with the sampler's own endpoints standing in for data. Its parent, Adjoint Matching, was designed to fine-tune large text-to-image flow models on a reward (Domingo-Enrich et al., 2025); AS is "reward fine-tuning from scratch" with reward $-\mathcal{E}$ and no pretrained model. Training on one's own samples usually risks the degeneration known from self-consuming generative loops; here it does not, because the targets $\nabla g(X_1)$ come from the energy, not from the model: the buffer only decides *where* the regression is accurate.

Key takeaways

- With zero base drift and a point start, the optimal control is $-\sigma_t \mathbb{E}[\nabla g(X_1) \mid X_t]$, and the optimal path given its endpoint is a Brownian bridge. Together these give a regression with one energy gradient per sample.
- $u^\star$ is a fixed point of the reciprocal-projection iteration (Theorem 9.3); general convergence and uniqueness are open.
- The price of the point start is a memoryless base with large noise; Chapter 10 removes this restriction.

Open problems and research questions

- **Convergence theory.** Prove convergence (with rates) of the reciprocal-projection iteration, with function approximation and a finite buffer.
- **Mode weights.** AS can find all modes yet misweight them (Figure 9.2). Can the free path weights be used *during training* (e.g. weighted buffers) without high variance?
- **Buffer staleness.** Quantify the bias from reusing endpoints of old controls, and design buffers or corrections that are provably consistent.
- **Large noise.** Memorylessness forces p_1^0 to cover π, which means large early noise and many integration steps. What is the best schedule, and can learned bases help (Chapter 10)?
- **Steep energies.** Clipping $\nabla\mathcal{E}$ biases the targets near repulsive cores; principled alternatives are needed.

CHAPTER 10

Schrödinger-bridge samplers

Goal	Remove the point-start restriction of Adjoint Sampling by solving a Schrödinger bridge from any source μ to π; prove that alternating two regressions is exactly iterative proportional fitting; then extend the reference process with annealing and with enhanced-sampling biases.
Prerequisites	Section 3.5; Chapter 9; Theorem 7.1.
Key papers	Liu et al. (2025); Choi et al. (2025); Nam et al. (2025); De Bortoli et al. (2021); Vargas et al. (2023b).

Overview. Adjoint Sampling starts every path at the same point. That is what makes it exact (the base is memoryless), but it wastes prior knowledge: we may know a good source density, for example that bonded atoms sit close together. With a spread-out source the naive terminal cost is biased (Equation (3.9)). The Schrödinger bridge tells us the right terminal cost, $\mathcal{E} + \log \hat{\varphi}_1$, and the extra term $\nabla \log \hat{\varphi}_1$ (the *corrector*) can itself be learned by regression. Alternating the two regressions is exactly IPF.

10.1. The Adjoint Schrödinger Bridge Sampler

Take the base $\mathrm{d}X_t = \sigma_t \, \mathrm{d}B_t$ (zero drift), source $X_0 \sim \mu$, and base transition density $q_{1|0}(y \mid x) = \mathcal{N}(y; x, \bar{\sigma}_1^2 \mathrm{I})$. The Schrödinger bridge $\mathbb{P}^\star$ from μ to π (Section 3.5) solves the control problem with terminal cost $g_{\mathrm{SB}} = \mathcal{E} + \log \hat{\varphi}_1 + \mathrm{const}$ (Theorem 3.13).

Lemma 10.1 (Reciprocal property of the bridge). $\mathbb{P}^\star(\cdot \mid X_0, X_1) = \mathbb{P}^0(\cdot \mid X_0, X_1)$: given both endpoints, the path is a base bridge. **Density form:** X_t given $(X_0, X_1) = (x, y)$ has density $\mathcal{N}\big(\cdot\,; x + \frac{\bar{\sigma}_t^2}{\bar{\sigma}_1^2}(y - x), \frac{\bar{\sigma}_t^2(\bar{\sigma}_1^2 - \bar{\sigma}_t^2)}{\bar{\sigma}_1^2}\mathrm{I}\big)$.

Proof. $\mathbb{P}^\star$ is $\mathbb{P}^0$ reweighted by $f(X_0)g(X_1)$ (Theorem 3.12); a reweighting by a function of (X_0, X_1) does not change the conditional law given (X_0, X_1). The density is Lemma A.2. ∎

Intuition

The bridge only reweights by functions of the two endpoints, so once both are known it has no opinion about the path between them. This is what makes the regression below exact given (X_0, X_1).

Since the base drift is zero, the adjoint is constant (Corollary 3.9) and equals $\nabla g_{\mathrm{SB}}(X_1) = \nabla\mathcal{E}(X_1) + \nabla \log \hat{\varphi}_1(X_1)$. ASBS (Liu et al., 2025) alternates two regressions, adjoint matching (AM) and corrector matching (CM), with a network $h \approx \nabla \log \hat{\varphi}_1$:

$$\text{(AM)}\quad u^{(k)} = \arg\min_{u}\ \mathbb{E}_t\ \mathbb{E}_{(x_0,x_1)\sim p^{\bar{u}}_{0,1}}\ \mathbb{E}_{x_t\sim q_{t|0,1}(\cdot|x_0,x_1)}\left[\left|u_t(x_t) + \sigma_t\big(\nabla\mathcal{E}(x_1) + h^{(k-1)}(x_1)\big)\right|^2\right], \tag{10.1}$$

$$\text{(CM)}\quad h^{(k)} = \arg\min_{h}\ \mathbb{E}_{(x_0,x_1)\sim p^{u}_{0,1}}\left[\left|h(x_1) - \nabla_{x_1}\log q_{1|0}(x_1 \mid x_0)\right|^2\right], \tag{10.2}$$

where $p^{u}_{0,1}$ is the joint density of start and end of the sampler, $q_{t|0,1}$ the base bridge density of Lemma 10.1, $\nabla_{x_1}\log q_{1|0}(x_1 \mid x_0) = -\frac{x_1 - x_0}{\bar{\sigma}_1^2}$, and $h^{(0)} = 0$. Each AM stage is solved by the fixed-point iteration of Chapter 9, now with bridges pinned at both ends; CM is plain regression on stored pairs.

Proposition 10.2 (Point source: back to Adjoint Sampling). If $\mu = \delta_{x_0}$, the exact corrector is $\nabla \log \hat{\varphi}_1 = \nabla \log q_1$, with $q_1 = \mathcal{N}(x_0, \bar{\sigma}_1^2\mathrm{I})$ the base endpoint density, and ASBS reduces to Adjoint Sampling.

Proof. With $X_0 = x_0$ always, the target in Equation (10.2) is the function $-\frac{X_1 - x_0}{\bar{\sigma}_1^2} = \nabla \log q_1(X_1)$; equivalently, Proposition 3.15 gives $\hat{\varphi}_1 = q_1$. Then g_{SB} is Equation (9.1). ∎

Intuition

With a point source every path starts at the same place, so "where did a path ending at y come from?" has a trivial answer, and the corrector is the known Gaussian score of the base. ASBS only learns something new when the source is spread out.

Theorem 10.3 (ASBS stages are IPF). Assume each regression is solved exactly, and let $\mathbb{P}^{(k)} = \mathbb{P}^{u^{(k)}}$. Then $\mathbb{P}^{(1)}$ is the optimally controlled process for terminal cost $\mathcal{E}$, and for every $k \geq 2$, $\mathbb{P}^{(k)}$ is obtained from $\mathbb{P}^{(k-1)}$ by the two IPF half-bridge projections of Lemma 3.16: first onto $\{p_1 = \pi\}$, then onto $\{p_0 = \mu\}$. Consequently the stages converge to the Schrödinger bridge whenever IPF does (Rüschendorf, 1995; De Bortoli et al., 2021).

Proof. Write $\mathbb{P}^0_\mu$ for the base started at μ. By Theorem 3.2, $\mathbb{P}^{(1)}$ is $\mathbb{P}^0_\mu$ reweighted by $\tilde{f}_1(X_0)e^{-\mathcal{E}(X_1)}$ with $\tilde{f}_1(x) = \frac{1}{\mathbb{E}[e^{-\mathcal{E}(X_1)}|X_0 = x]}$. Suppose $\mathbb{P}^{(k-1)}$ is $\mathbb{P}^0_\mu$ reweighted by $\tilde{f}_{k-1}(X_0)\,\tilde{g}_{k-1}(X_1)$. The computation of Proposition 3.14, which only uses this product form, shows that the exact corrector of stage $k-1$ is $\nabla \log \hat{\varphi}_1^{(k-1)}$ with $\hat{\varphi}_1^{(k-1)}(y) = \int \mu(x)\tilde{f}_{k-1}(x)q_{1|0}(y \mid x)\,\mathrm{d}x$, and that the time-1 density of $\mathbb{P}^{(k-1)}$ is $\hat{\varphi}_1^{(k-1)}\tilde{g}_{k-1}$.

Projection onto $\{p_1 = \pi\}$. By Lemma 3.16, it reweights $\mathbb{P}^{(k-1)}$ by $\frac{\pi(X_1)}{\hat{\varphi}_1^{(k-1)}(X_1)\tilde{g}_{k-1}(X_1)}$, giving $\mathbb{P}^0_\mu$ reweighted by $\tilde{f}_{k-1}(X_0)\,\frac{\pi(X_1)}{\hat{\varphi}_1^{(k-1)}(X_1)}$.

Projection onto $\{p_0 = \mu\}$. Again by Lemma 3.16, this replaces $\tilde{f}_{k-1}(X_0)$ by the factor that restores the time-0 density μ, giving $\mathbb{P}^0_\mu$ reweighted by $\frac{e^{-g_k(X_1)}}{\mathbb{E}[e^{-g_k(X_1)}|X_0]}$ with $g_k = \log\frac{\hat{\varphi}_1^{(k-1)}}{\pi} = \mathcal{E} + \log\hat{\varphi}_1^{(k-1)} + \mathrm{const}$. By Theorem 3.2, applied for each starting point, this is the optimally controlled process for terminal cost g_k, i.e. the solution of stage k of Equation (10.1). It again has the product form, so the induction continues. ∎

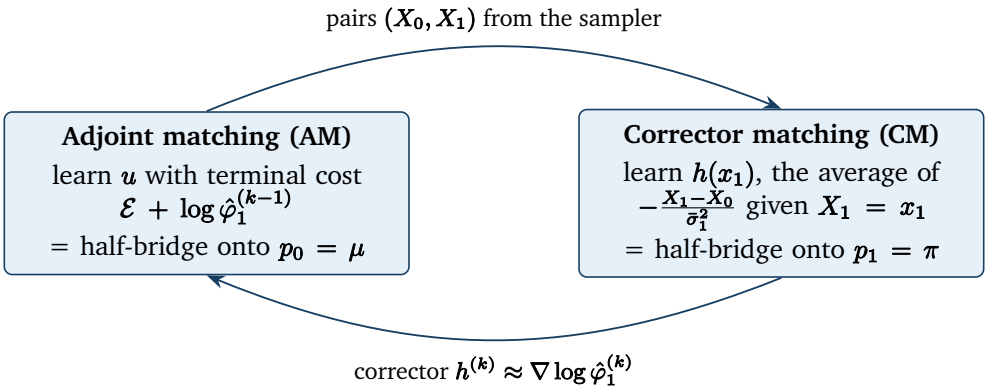


Figure 10.1 ASBS alternates two regressions that implement the two half-bridge projections of IPF, without any samples from π. Its fixed point is the Schrödinger bridge from μ to π (Theorem 10.3).

Intuition

The corrector answers: "given that a path ended at y, where did it typically start?", and turns this into the gradient of the backward potential. Endpoints that are easy to reach from the source get a discount in the terminal cost, exactly enough that the final density is not biased towards them.

Listing 10.1 ASBS on a 2-D mixture with source $\mu = \mathcal{N}(0, \mathrm{I})$, reusing the network of Listing 9.1. On a Gaussian target $\mathcal{N}((2, -2), 0.25\,\mathrm{I})$ the corrector reduces the bias of the mean from $(-0.11, 0.06)$ to $(-0.07, 0.03)$ after four stages; the remaining excess variance (0.29 instead of 0.25) is exactly the noise variance $\sigma^2 h = 0.04$ of the last Euler step.

```python
import math, torch
from adjoint_sampling import Net          # same MLP f_theta(x, t)

SIG = 2.0                                  # constant sigma: base dX = SIG dB, sbar_t^2 = SIG^2 t
def bridge(x0, x1, t):                     # Brownian bridge marginal X_t | X_0, X_1
    return x0 + t * (x1 - x0) + SIG * torch.sqrt(t * (1 - t)) * torch.randn_like(x0)

@torch.no_grad()
def sample(f, x0, N=100):                  # dX = SIG*u dt + SIG dB,  u = -SIG f
    x, dt = x0.clone(), 1.0 / N
    for k in range(N):
        x = x - SIG**2 * f(x, torch.tensor([[k * dt]])) * dt + SIG * math.sqrt(dt) *
     torch.randn_like(x)
    return x

def grad_E(x, energy, clip=50.0):
    x = x.clone().requires_grad_(True)
    g = torch.autograd.grad(energy(x).sum(), x)[0]
    return (g * (clip / g.norm(dim=1, keepdim=True).clamp(min=clip))).detach()

def train_asbs(energy, prior_sample, d=2, stages=4, outer=60, inner=50, n=512, batch=512):
    f, h = Net(d), Net(d)                  # controller f (u = -SIG f) and corrector h ~ grad log
     phi_hat_1
    opt_f, opt_h = torch.optim.Adam(f.parameters(), 1e-3), torch.optim.Adam(h.parameters(), 1e-3)
    zero = torch.zeros(1, 1)
    for k in range(stages):
        buf = []
```

```
        for _ in range(outer):              # --- adjoint matching with terminal cost E + log
    phi_hat_1
            x0 = prior_sample(n); x1 = sample(f, x0)
            with torch.no_grad():
                corr = h(x1, zero.expand(n, 1)) if k > 0 else torch.zeros_like(x1)
            buf = (buf + [(x0, x1, grad_E(x1, energy) + corr)])[-20:]
            X0, X1, G = map(torch.cat, zip(*buf))
            for _ in range(inner):
                i = torch.randint(len(X0), (batch,)); t = torch.rand(batch, 1) * 0.99 + 0.005
                loss = ((f(bridge(X0[i], X1[i], t), t) - G[i]) ** 2).sum(-1).mean()
                opt_f.zero_grad(); loss.backward(); opt_f.step()
        for _ in range(outer * inner // 5):  # --- corrector matching: h(X1) ~ -(X1 - X0)/sbar_1^2
            i = torch.randint(len(X0), (batch,))
            loss = ((h(X1[i], zero.expand(batch, 1)) + (X1[i] - X0[i]) / SIG**2) **
    2).sum(-1).mean()
            opt_h.zero_grad(); loss.backward(); opt_h.step()
    return f

if __name__ == "__main__":
    torch.manual_seed(0)
    means = torch.tensor([[3., 3.], [-3., 3.], [3., -3.], [-3., -3.]])
```

Priors for molecules. ASBS lets the source carry knowledge. The *harmonic prior* $\mu(x) \propto \exp\big(-\frac{1}{2\ell^2}\sum_{(i,j)\in\text{bonds}}|x_i - x_j|^2\big)$, on the zero-centre-of-mass subspace, is a Gaussian that places bonded atoms close together. On amortised conformer generation (SPICE molecules, eSEN energy), Liu et al. (2025) report that ASBS with this prior raises coverage from 56.75% (AS) to 74.05%; on alanine dipeptide it finds the high-density states but misses some low-density ones.

10.2. Annealed references: NAAS

Another way to inject structure is to change the *reference process*. Take an annealing path U_t (Chapter 7) and the reference

$$\mathrm{d}X_t = -\frac{\sigma_t^2}{2}\nabla U_t(X_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t, \tag{10.3}$$

i.e. annealed Langevin dynamics with $\varepsilon_t = \frac{\sigma_t^2}{2}$, with running cost $c_t = \partial_t U_t$ and no terminal cost.

Proposition 10.4 (Annealed control and the initial density). Let $h_0(x) = \mathbb{E}_{\mathrm{ref}}\big[e^{-\int_0^1 \partial_t U_t(X_t)\,\mathrm{d}t} \mid X_0 = x\big]$. The optimally controlled process started at μ has time-1 density π if $\mu \propto e^{-U_0}h_0$, i.e. $\mu \propto e^{-U_0 - V_0}$ with $V_0 = -\log h_0$.

Proof. By Theorem 3.2, the optimal path law is

$$\mathbb{P}^\star = \mu(X_0)\,\mathbb{P}^{\mathrm{ref}}(\cdot \mid X_0)\,\frac{e^{-\int_0^1 \partial_t U_t(X_t)\,\mathrm{d}t}}{h_0(X_0)}.$$

If $\mu = \frac{1}{C}e^{-U_0}h_0$, this is $\frac{1}{C}e^{-U_0(X_0)}\,\mathbb{P}^{\mathrm{ref}}(\cdot \mid X_0)\,e^{-\int_0^1 \partial_t U_t\,\mathrm{d}t}$, i.e. the reference started from $\rho_0 \propto e^{-U_0}$ and reweighted by the Jarzynski weight. By Jarzynski's equality (Theorem 7.1 with $b = 0$), $\mathbb{E}_{X_0\sim\rho_0}\big[e^{-\int_0^1 \partial_t U_t\,\mathrm{d}t}\varphi(X_1)\big] = \frac{Z}{Z_0}\,\mathbb{E}_\pi[\varphi]$ for bounded φ, so the time-1 density of $\mathbb{P}^\star$ is π. ∎

Intuition

Annealed Langevin dynamics with Jarzynski weights already ends exactly at π if it starts from ρ_0. Turning those weights into a control moves the reweighting of the *starting points* into the initial density, which is why NAAS needs a second, learned process that produces the tilted start $e^{-U_0-V_0}$.

The Non-equilibrium Annealed Adjoint Sampler (Choi et al., 2025) learns (i) a control on $[-1,0]$ that carries a point mass to the tilted start (an Adjoint-Sampling problem whose terminal cost involves V_0, estimated by the adjoint at $t=0$), and (ii) the annealed control on $[0,1]$ by adjoint matching with the adjoint Equation (3.10): here $\nabla b_t = -\frac{\sigma_t^2}{2}\nabla^2 U_t$ and $\nabla c_t = \partial_t \nabla U_t$, so the adjoint needs Hessian–vector products of the energy. The annealing drift guides paths towards π from the start, which helps on multimodal targets, at the cost of energy gradients along every path.

10.3. Enhanced sampling with collective variables: WT-ASBS

Metadynamics accelerates simulations by adding a bias $V(\xi(x))$ that grows where the system has already been, along a low-dimensional *collective variable* ξ (e.g. backbone angles) (Laio and Parrinello, 2002); the well-tempered variant converges to $V = -(1-\frac{1}{\gamma})F$, where F is the free energy along ξ and $\gamma > 1$ a bias factor (Barducci et al., 2008). Nam et al. (2025) run ASBS on the biased target $\pi_V \propto e^{-\beta(\mathcal{E}+V\circ\xi)}$ and update V from the sampler's own samples with the well-tempered rule, more slowly than the sampler. Unbiased averages are recovered with the weights $e^{\beta V(\xi(X))}$, and free energies along ξ from $F = -\frac{\gamma}{\gamma-1}V + \text{const}$. The bias flattens the landscape along ξ, so the sampler visits low-population states that plain ASBS misses; they report accurate free energies on small peptides and on reaction energy surfaces, with several times fewer energy evaluations than well-tempered metadynamics.

Generative-modelling view

In generative modelling, Schrödinger bridges are learned with samples at *both* ends (DSB, DSBM, I^{2}SB; De Bortoli et al., 2021; Shi et al., 2023; Liu et al., 2023), by alternating half-bridge fits that are again IPF. ASBS solves the same problem with samples at *neither* end: the source is sampled, the target is known only through its energy, and the backward half-bridge is replaced by corrector matching. An informative source (a harmonic prior) is the sampling analogue of data-dependent priors in protein and molecule diffusion models: the closer the source is to the target, the less the control has to do.

Key takeaways

- ASBS = adjoint matching with terminal cost $\mathcal{E} + \log\hat{\varphi}_1$, plus corrector matching for $\nabla\log\hat{\varphi}_1$; with exact regressions the stages are exactly IPF (Theorem 10.3), and with a point source it is Adjoint Sampling (Proposition 10.2).
- Freedom in the source (harmonic priors) and in the reference (annealed Langevin, metadynamics biases) is where domain knowledge enters.

Open problems and research questions

- **Inexact stages.** IPF can accumulate errors when projections are approximate. How do regression errors in AM and CM propagate across stages?
- **Mode seeking.** ASBS can miss low-density modes. Combining it with importance weights, SMC, or CV biases in a principled way is open; WT-ASBS needs good collective variables.
- **Learning the reference.** The source, noise and annealing path are hand-designed. Can they be learned jointly with the bridge?
- **Amortised bridges.** The corrector depends on the target; amortising it across molecules or data sets raises new generalisation questions.
- **Cost of annealing.** NAAS-type references need ∇U_t and Hessian–vector products along paths, against the energy efficiency that motivated adjoint methods.

CHAPTER 11

Sampling on manifolds and with symmetries

Goal	Carry the ideas of Chapters 2 and 3 to curved spaces and to spaces with symmetries: subspaces with zero centre of mass, tori of angles, spheres and rotation groups. See which results transfer unchanged and which must be rebuilt; derive an exact Adjoint Sampler on tori; survey Riemannian samplers.
Prerequisites	Chapters 2, 3 and 9; basic geometry of curves and surfaces.
Key papers	Hsu (2002); De Bortoli et al. (2022); Chen and Lipman (2024); Jing et al. (2022); Mosso et al. (2026); Havens et al. (2026).

Overview. Many targets do not live on $\mathbb{R}^d$. Angles live on circles; rotations on the rotation group; molecular energies are unchanged by rigid motions, so configurations that differ by a rotation are "the same". Working directly on the right space removes redundant directions and builds the structure into the sampler. The good news of this chapter is that every statement about generators, densities and KL divergences transfers to manifolds unchanged. What must be rebuilt are the explicit Euclidean formulas (Gaussian transitions, straight-line bridges), which Adjoint Sampling relies on.

11.1. A minimal geometric toolkit

A Riemannian manifold $(\mathcal{M}, g)$ is a space that looks locally like $\mathbb{R}^m$ and comes with a way to measure lengths and angles of tangent vectors. Three operations generalise their Euclidean versions:

- the *gradient* $\operatorname{grad} f$ (the direction of steepest increase, now a tangent vector);
- the *divergence* $\operatorname{div} v$ of a vector field (the rate of volume change), with the divergence theorem $\int_{\mathcal{M}} \varphi \operatorname{div} v \, \mathrm{d}\operatorname{vol} = -\int_{\mathcal{M}} \langle \operatorname{grad} \varphi, v \rangle \, \mathrm{d}\operatorname{vol}$ on a closed manifold;
- the *Laplace–Beltrami operator* $\Delta_g f = \operatorname{div} \operatorname{grad} f$.

Densities are taken with respect to the volume measure vol. The exponential map $\operatorname{Exp}_x(v)$ follows the geodesic (straightest curve) from x in direction v for unit time; Log_x is its local inverse; $d(x, y)$ is the geodesic distance. For a surface embedded in $\mathbb{R}^D$, the gradient of a function is the Euclidean gradient projected onto the tangent plane.

11.2. Diffusions on manifolds

Definition 11.1 (Riemannian diffusion). A process on $\mathcal{M}$ is a diffusion with drift b_t (a tangent vector field) and noise level σ_t if, for every smooth f, $\frac{\mathrm{d}}{\mathrm{d}t} \mathbb{E}[f(X_t)] = \mathbb{E}[\mathcal{L}_t f(X_t)]$ with $\mathcal{L}_t f = \langle b_t, \operatorname{grad} f \rangle + \frac{\sigma_t^2}{2} \Delta_g f$. Brownian motion $B^{\mathcal{M}}$ has generator $\frac{1}{2}\Delta_g$.

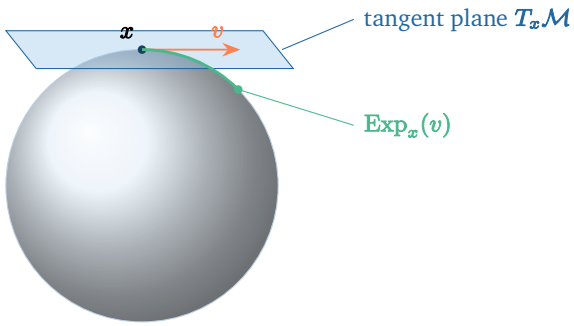


Figure 11.1 Tangent plane, exponential map and geodesic on a sphere. Noise and drifts are drawn in the tangent plane at x and mapped onto the manifold by Exp_x.

In practice such a diffusion is simulated by a *geodesic random walk*: $X_{k+1} = \mathrm{Exp}_{X_k}\big(h\,b(X_k) + \sigma\sqrt{h}\,\xi_k\big)$ with ξ_k a standard Gaussian in the tangent plane at X_k (Figure 11.1); this converges to the diffusion as $h \to 0$ (De Bortoli et al., 2022). For embedded manifolds one can equivalently take a Euclidean step and project back onto $\mathcal{M}$.

Theorem 11.2 (Fokker–Planck on $\mathcal{M}$). The density p_t of a Riemannian diffusion (with respect to vol) satisfies $\partial_t p_t = -\operatorname{div}(p_t b_t) + \frac{\sigma_t^2}{2}\Delta_g p_t$.

Proof. As for Theorem 2.3: $\frac{\mathrm{d}}{\mathrm{d}t}\int \varphi\, p_t \,\mathrm{d}\,\mathrm{vol} = \int (\mathcal{L}_t\varphi)\, p_t \,\mathrm{d}\,\mathrm{vol}$, and the divergence theorem moves the derivatives onto p_t. ∎

Intuition

Mass is still conserved and noise still spreads it out; only the meaning of "outflow" and "spreading" changes, through a divergence and a Laplacian that account for curvature.

Proposition 11.3 (Riemannian Langevin dynamics). The density $\pi \propto e^{-\mathcal{E}}$ (with respect to vol) is invariant for the diffusion with drift $-\operatorname{grad}\mathcal{E}$ and $\sigma^2 = 2$.

Proof. The right side of Theorem 11.2 at π is $\operatorname{div}(\pi \operatorname{grad}\mathcal{E}) + \Delta_g \pi = \operatorname{div}(\pi \operatorname{grad}\mathcal{E} + \operatorname{grad}\pi) = 0$, because $\operatorname{grad}\pi = -\pi \operatorname{grad}\mathcal{E}$. ∎

Intuition

Exactly as in $\mathbb{R}^d$: go downhill along the Riemannian gradient and let the noise spread you out; the two balance at $e^{-\mathcal{E}}$, measured against the Riemannian volume.

Theorem 11.4 (Transfer principle). Let $\mathbb{P}^u$, $\mathbb{P}^0$ be the path laws of Riemannian diffusions with drifts $b + \sigma u$ and b (u tangent), the same noise and the same initial density. Then $\mathrm{KL}(\mathbb{P}^u \| \mathbb{P}^0) = \mathbb{E}_{\mathbb{P}^u}\big[\int_0^1 \tfrac{1}{2}|u_t(X_t)|_g^2 \,\mathrm{d}t\big]$. Consequently, with ∇, $\nabla\cdot$, Δ replaced by grad, div, Δ_g, the following hold unchanged on $\mathcal{M}$: the Gibbs principle of control (Theorem 3.2), Doob's h-transform (Theorem 2.14), the HJB equation (Corollary 3.3), the marginal-preserving family (Theorem 2.13), time reversal (Theorem 2.12), the Schrödinger system and its control form (Theorems 3.12 and 3.13) and the corrector identity (Proposition 3.14).

Proof sketch. The KL formula is Girsanov's theorem on manifolds (Hsu, 2002, Ch. 8). Each listed result was proved using only Itô's formula through the generator, the Fokker–Planck equation

with integration by parts, the Markov property, and the Girsanov ratio. All four are available on $\mathcal{M}$. For time reversal see De Bortoli et al. (2022). ∎

Intuition

Statements about generators, densities and KL are coordinate-free, so they survive. What does *not* survive are the Euclidean closed forms: Gaussian transition densities, straight-line bridges, and the constant adjoint of Corollary 3.9, whose proof differentiated the flow, which on a curved space involves parallel transport and curvature.

11.3. Heat kernels and bridges

On a compact manifold, Brownian motion converges to the uniform density, so a long base process is approximately memoryless from any start (useful for Proposition 3.7). Its transition density, the *heat kernel* $p_t(x \mid y)$, is explicit on a few spaces:

- **Torus** $\mathbb{T}^m = [-\pi, \pi)^m$ with opposite edges glued: a *wrapped Gaussian*, the sum of Euclidean Gaussians over all copies $\theta + 2\pi k$ (Proposition 11.5).
- **Sphere** S^2 and rotation group $\mathrm{SO}(3)$: series that are truncated in practice (Yim et al., 2023).
- **General** $\mathcal{M}$: for short times, $\log p_t(x \mid y) \approx -\frac{d(x,y)^2}{2t}$ (Varadhan), so $\operatorname{grad}_x \log p_t(x \mid y) \approx \frac{\operatorname{Log}_x(y)}{t}$.

Proposition 11.5 (Brownian motion and bridges on the torus). Let $\tilde{\Theta}_t = \theta_0 + \int_0^t \sigma_s \, \mathrm{d}B_s$ in $\mathbb{R}^m$ and $\Theta_t = \tilde{\Theta}_t$ wrapped into $[-\pi, \pi)^m$. Then Θ is Brownian motion on the torus (with time change $\bar{\sigma}_t^2$), its transition density is the wrapped Gaussian, and its bridge from θ_0 to θ_1 is the mixture

$$p(\theta_t \mid \theta_0, \theta_1) = \sum_{k \in \mathbb{Z}^m} w_k \, \beta_k(\theta_t), \qquad w_k = \frac{\mathcal{N}(\theta_1 - \theta_0 + 2\pi k;\, 0,\, \bar{\sigma}_1^2 \mathrm{I})}{\sum_{k'} \mathcal{N}(\theta_1 - \theta_0 + 2\pi k';\, 0,\, \bar{\sigma}_1^2 \mathrm{I})}, \tag{11.1}$$

where β_k is the density of the Euclidean bridge from θ_0 to $\theta_1 + 2\pi k$ (Lemma A.2), wrapped onto the torus.

Proof. Wrapping is locally an isometry, so Θ has generator $\frac{\sigma_t^2}{2}\Delta$ on the torus, and the density of Θ_t at θ sums the Euclidean density over all copies $\theta + 2\pi k$. Given $\Theta_1 = \theta_1$, the unwrapped endpoint $\tilde{\Theta}_1$ is one of the copies $\theta_1 + 2\pi k$, with probability proportional to its Euclidean density, i.e. w_k; given the unwrapped endpoint, the unwrapped path is a Euclidean bridge, and wrapping it gives Equation (11.1). ∎

Intuition

On a circle, "going from θ_0 to θ_1" is ambiguous: one can also wind around once, twice, or backwards. Unroll the circle into a line, where each number of windings is a different endpoint; pick one with the probability that Brownian motion would reach it, draw the usual straight-line bridge, and roll it back up.

11.4. Symmetry: quotients and equivariance

Let a group G act on $\mathbb{R}^d$ by orthogonal maps (rotations, permutations), and let $\mathcal{E}$ be invariant: $\mathcal{E}(gx) = \mathcal{E}(x)$.

Proposition 11.6 (Equivariant SDEs preserve invariance). If the initial density is invariant and the drift is equivariant, $b_t(gx) = gb_t(x)$ for all $g \in G$, then every marginal density of $\mathrm{d}X_t = b_t(X_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t$ is invariant. In particular, an equivariant control with an invariant base produces invariant samples.

Proof. $Y_t = gX_t$ solves $\mathrm{d}Y_t = b_t(Y_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}(gB)_t$, and gB is again a Brownian motion because g is orthogonal. Since Y_0 and X_0 have the same density, uniqueness gives that Y_t and X_t have the same density for every t. ∎

Intuition

Rotating a path of an equivariant SDE gives another path of the same SDE, driven by rotated noise, which is just as likely. So every rotated version of every outcome is equally probable.

Translations act by isometries too, but no probability density on $\mathbb{R}^{3n}$ is translation invariant (it would have to be constant). The standard construction removes them.

Proposition 11.7 (Zero-centre-of-mass subspace). Let $P = (\mathrm{I}_n - \frac{1}{n}\mathbf{1}\mathbf{1}^\top) \otimes \mathrm{I}_3$ and $V = \{x \in \mathbb{R}^{3n} : \sum_i x_i = 0\}$, of dimension $3n - 3$. Then P is the orthogonal projection onto V; PB is a standard Brownian motion on V; and $\mathcal{N}(0, s^2P)$ has density $(2\pi s^2)^{-\frac{3n-3}{2}} e^{-\frac{|x|^2}{2s^2}}$ with respect to the volume on V. For a translation-invariant $\mathcal{E}$, the restriction of $e^{-\mathcal{E}}$ to V describes the distribution of shapes up to translation.

Proof. P is symmetric and $P^2 = P$, and its kernel consists of the configurations where all particles are shifted by the same vector; so it projects orthogonally onto V. PB has independent Gaussian increments with covariance $(t - s)P$, which is the identity on V. In an orthonormal basis of V, $\mathcal{N}(0, s^2P)$ is $\mathcal{N}(0, s^2\mathrm{I}_{3n-3})$. Every configuration can be translated to V in exactly one way, so V represents each shape once. ∎

Intuition

Translating a molecule changes nothing physical, so three of the $3n$ coordinates are pure redundancy. Centring removes them; the noise must be centred too, otherwise the process drifts out of the subspace. The only bookkeeping cost is that Gaussian densities now live in dimension $3n - 3$.

Rotations are handled by equivariance rather than by quotienting (the quotient by rotations is not a manifold where configurations are collinear). Reflection equivariance maps a molecule to its mirror image, which is wrong for chiral molecules; use networks equivariant to rotations only.

11.5. Internal coordinates and torsion tori

Placing atom i by its bond length r_i, bond angle θ_i and dihedral angle ϕ_i relative to three earlier atoms is a change to spherical coordinates around the bonded atom, so the Cartesian volume element is $\prod_i r_i^2 \sin\theta_i\ \mathrm{d}r_i\ \mathrm{d}\theta_i\ \mathrm{d}\phi_i$ (ignoring the six rigid-body coordinates). A density in internal coordinates must therefore include the term $\sum_i (2\log r_i + \log\sin\theta_i)$, as Boltzmann generators in internal coordinates do (Noé et al., 2019).

If bond lengths and angles are frozen, configurations form an m-dimensional torus of dihedral angles ϕ, embedded in V (after removing rigid motions) by a map $x(\phi)$.

Proposition 11.8 (Density on the torsion torus). Let $J(\phi) = \frac{\partial x}{\partial \phi}$ (rigid motions removed). The density on the torus, with respect to $\mathrm{d}\phi$, induced by $e^{-\mathcal{E}}$ on the embedded torus with its own surface measure is

$$\pi(\phi) \propto e^{-\mathcal{E}(x(\phi))} \det\left(J(\phi)^\top J(\phi)\right)^{\frac{1}{2}}.$$

Proof. The surface measure of the embedded torus, written in the coordinates ϕ, is $\det(J^\top J)^{\frac{1}{2}}\,\mathrm{d}\phi$ (area formula). ∎

Intuition

Describing a curved surface by angles distorts areas, as a world map distorts Greenland. A uniform density in angle space is not uniform on the surface; the factor $\det(J^\top J)^{\frac{1}{2}}$ undoes the distortion, so that the energy, not the parameterisation, decides where probability goes.

This is a *Fixman*-type correction (Fixman, 1974). Torsional generative models usually omit it; it varies slowly with ϕ but is a systematic bias, and the true torsional distribution of a flexible molecule also involves averaging over bond and angle fluctuations. A torus sampler should use the energy $\mathcal{E}_{\mathrm{tor}}(\phi) = \mathcal{E}(x(\phi)) - \frac{1}{2}\log\det J^\top J$.

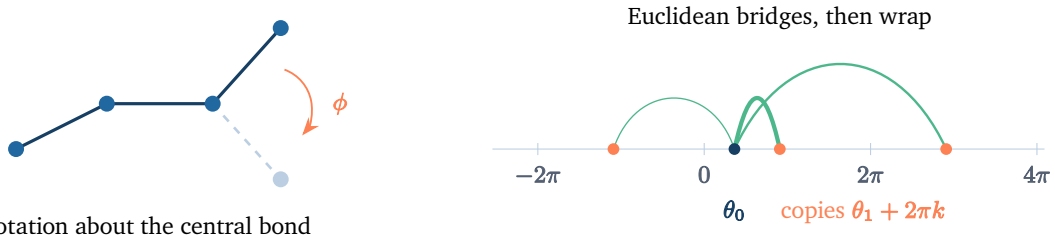


Figure 11.2 Left: a dihedral (torsion) angle. Right: a bridge on the circle is a mixture of Euclidean bridges to all copies of the endpoint, weighted by the heat kernel (Proposition 11.5); line width indicates the weight w_k.

11.6. Adjoint Sampling on tori

Flat tori are the one non-Euclidean space where Adjoint Sampling is exact with no approximation. They cover torsion-angle samplers and periodic simulation boxes.

Proposition 11.9 (Exact Adjoint Sampling on $\mathbb{T}^m$). Let $\mathcal{E}$ be a smooth function on the torus (i.e. 2π-periodic), the base Θ_t as in Proposition 11.5 started at θ_0, and $g = \mathcal{E} + \log p_1^0$ with p_1^0 the wrapped Gaussian. Then (i) the optimally controlled process ends at $\pi \propto e^{-\mathcal{E}}$; (ii) $u_t^\star(\theta) = -\sigma_t\,\mathbb{E}_{\mathbb{P}^\star}[\nabla g(\Theta_1) \mid \Theta_t = \theta]$; (iii) given Θ_1, an optimal path is the wrapped bridge Equation (11.1). Hence Algorithm 4 applies with Lemma 9.1 replaced by Equation (11.1).

Proof. Unwrap to $\mathbb{R}^m$, where the base is Euclidean Brownian motion from θ_0 and the terminal cost $\tilde{g}$ (the periodic extension of g) is periodic. The Euclidean results apply. By Theorem 3.2, the optimal unwrapped path law reweights the base by $\frac{e^{-\tilde{g}(\tilde{\Theta}_1)}}{\mathcal{Z}}$, so the wrapped endpoint has density $\frac{p_1^0 e^{-g}}{\mathcal{Z}} = \frac{e^{-\mathcal{E}}}{\mathcal{Z}}$, which is (i). The function $h(t,x) = \mathbb{E}[e^{-\tilde{g}(\tilde{\Theta}_1)} \mid \tilde{\Theta}_t = x]$ is periodic, so $u^\star = \sigma\nabla\log h$ is a well-defined field on the torus, and Corollary 3.9 gives (ii). For (iii), given the unwrapped endpoint the optimal path is a Euclidean bridge (Lemma 9.1); given only the wrapped endpoint θ_1, the unwrapped endpoint is $\theta_1 + 2\pi k$ with probability proportional to $p_1^0(\theta_1 + 2\pi k)\,e^{-\tilde{g}(\theta_1 + 2\pi k)}$ (Euclidean base density), and since $\tilde{g}$ is periodic the factor $e^{-\tilde{g}}$ cancels, leaving w_k. ∎

Intuition

Unroll the torus into Euclidean space, where the energy becomes periodic. Every Euclidean statement of Chapter 9 applies, and periodicity guarantees that the answers roll back up onto the torus. The only new ingredient is to keep track of the number of windings, with exact weights.

Listing 11.1 Wrapped-Gaussian log-density and the exact torus bridge of Proposition 11.5. Against brute-force simulation of 4×10^6 wrapped Brownian paths, the histogram total-variation distance is 0.016 (sampling noise), against 0.19 for a naive bridge that ignores windings.

```python
import math, torch

TWO_PI = 2 * math.pi
def wrap(theta):                                    # map R^m -> [-pi, pi)^m
    return (theta + math.pi) % TWO_PI - math.pi

def log_wrapped_normal(theta, mean, var, K=3):
    """log of sum_k N(theta - mean + 2 pi k; 0, var) per coordinate, summed over coordinates."""
    k = torch.arange(-K, K + 1, dtype=theta.dtype) * TWO_PI
    diff = wrap(theta - mean)[..., None] + k                    # (..., m, 2K+1)
    lp = -diff**2 / (2 * var) - 0.5 * math.log(2 * math.pi * var)
    return torch.logsumexp(lp, -1).sum(-1)

def torus_bridge(theta0, theta1, s2_t, s2_1, K=3):
    """Sample theta_t | theta_0, theta_1 for wrapped Brownian motion (Prop. 11.5):
    choose the winding k with prob. propto N(theta1 - theta0 + 2 pi k; 0, s2_1), then sample
    the Euclidean bridge from theta0 to the lifted endpoint and wrap."""
    k = torch.arange(-K, K + 1, dtype=theta0.dtype) * TWO_PI
    d = wrap(theta1 - theta0)[..., None] + k                    # lifted displacements
    winding = torch.distributions.Categorical(logits=-d**2 / (2 * s2_1)).sample()
    lift = torch.gather(d, -1, winding[..., None])[..., 0]      # chosen lifted displacement
    m = s2_t / s2_1
    xt = theta0 + m * lift + (s2_t * (1 - m)) ** 0.5 * torch.randn_like(theta0)
    return wrap(xt)
```

11.7. Riemannian samplers

Generative models on manifolds. Riemannian score-based models (De Bortoli et al., 2022; Huang et al., 2022) use time reversal (Theorem 11.4) with heat-kernel or short-time score targets; Riemannian flow matching (Chen and Lipman, 2024) replaces straight lines by geodesics, $x_t = \operatorname{Exp}_{x_1}\big((1-t)\operatorname{Log}_{x_1} x_0\big)$, and Theorem 5.4 holds with the manifold continuity equation. These underlie protein backbone models on $\mathrm{SE}(3)^L$ (Yim et al., 2023; Bose et al., 2024) and torsional diffusion for conformers (Jing et al., 2022). The latter also gives a *torsional Boltzmann generator*: the exact density of its ODE sampler allows importance reweighting towards the Boltzmann distribution on the torsion torus.

Riemannian ASBS. Mosso et al. (2026) extend ASBS to manifolds defined by constraints, $\mathcal{M} = \{x : c(x) = 0\}$, with controls in the tangent planes and the terminal cost of Theorem 3.13. Two Euclidean shortcuts are replaced: (i) the corrector target $\operatorname{grad} \log q_{1|0}(x_1|x_0)$ is approximated by the short-time formula $\frac{\operatorname{Log}_{x_1}(x_0)}{\bar\sigma_1^2}$ plus a volume correction; (ii) the adjoint is no longer constant, and is approximated by transporting $\operatorname{grad} g(X_1)$ back along the bridge geodesic. For manifolds

without explicit $\mathbf{Log}$ maps they use projections instead. Their experiments are on spheres, matrix manifolds, robot kinematic chains and rotation estimation; molecular applications (e.g. fixed bond lengths) are a natural next step.

Other approaches. Havens et al. (2026) derive closed-form drifts for Flow Sampling on spheres and hyperbolic spaces. Durmus et al. (2026) propose a training-free sampler on manifolds that follows a geometric interpolant, estimating its drift by inner Monte Carlo, aimed at multimodal targets. Riemannian MCMC (Girolami and Calderhead, 2011) remains the baseline.

Algorithm 5 Riemannian Adjoint Sampling (sketch; exact on flat tori)

Require: manifold $\mathcal{M}$ with $\mathbf{Exp}$, $\mathbf{Log}$ and tangent projections; energy $\mathcal{E}$ on $\mathcal{M}$; start x_0

loop
  simulate geodesic random walks $X_{k+1} = \mathrm{Exp}_{X_k}\left(h\sigma_{t_k} u_\theta(X_k, t_k) + \sigma_{t_k}\sqrt{h}\,\xi_k\right)$, ξ_k Gaussian in the tangent plane
  store $(X_1, \mathbf{grad}\, g(X_1))$ with $g = \mathcal{E} + \log p_1^0$ (heat kernel or short-time approximation)
  for inner steps **do**
    draw a base bridge point X_t given (x_0, X_1): exact on tori (Proposition 11.5), else geodesic approximation
    target $a_t \approx \mathbf{grad}\, g(X_1)$ transported to X_t (no transport needed on flat tori); step on $|u_\theta(X_t, t) + \sigma_t a_t|^2$
  end for
end loop

Generative-modelling view

Riemannian generative models are standard wherever data live on curved spaces: protein backbones as rigid frames (Yim et al., 2023; Bose et al., 2024), molecular torsions on tori (Jing et al., 2022), earth and climate data on spheres, robot poses on rotation groups. The same tools serve neural samplers, with an extra benefit: building symmetries and constraints into the state space removes redundant directions that would otherwise cost energy evaluations to explore. Equivariant networks are the architectural counterpart of the constructions of Section 11.4.

Key takeaways

- Results about generators, densities and KL transfer to manifolds unchanged (Theorem 11.4); explicit bridges, transitions and constant adjoints do not.
- Translations are removed by centring (Proposition 11.7); rotations and permutations by equivariant drifts (Proposition 11.6).
- On flat tori (torsions, periodic boxes) Adjoint Sampling is exact with wrapped bridges (Proposition 11.9); torsional targets need the volume correction of Proposition 11.8.

Open problems and research questions

- **Curvature-aware adjoints.** Quantify the error of the geodesic-transport approximation of the adjoint, and find exact or unbiased alternatives on $\mathrm{SO}(3)$ and $\mathrm{SE}(3)^n$.

- **Mixed spaces.** Molecules mix tori (torsions), Euclidean factors (soft bonds) and rotation groups (frames). Design samplers on such products with consistent noise across factors.
- **Constrained targets.** Samplers on the manifold of fixed bond lengths would give direct access to constrained ensembles; the reference measure (with or without the Fixman correction) must be chosen consciously.
- **Densities on quotients.** Free energies require densities on configurations modulo rigid motions; equivariant models give invariant densities on $\mathbb{R}^{3n}$, and the normalisation relative to the quotient needs care.

CHAPTER 12

Neural samplers in machine learning and generative modelling

Goal	See that many central problems of modern machine learning are sampling problems from an unnormalised density: fine-tuning generative models with a reward, conditioning a pretrained diffusion model on observations, Bayesian posteriors, training energy-based models, and variational inference. Re-derive their standard algorithms from the theory of this book.
Prerequisites	Chapters 1 to 3; Chapters 6 and 9.
Key papers	Domingo-Enrich et al. (2025); Dhariwal and Nichol (2021); Chung et al. (2023); Wu et al. (2023); Skreta et al. (2025); Welling and Teh (2011); Du and Mordatch (2019); Rezende and Mohamed (2015); Bengio et al. (2021).

Overview. A generative model trained on data gives a sampler for the data density p_{data}. As soon as we want a density *defined from* p_{data} (data that score well, data consistent with a measurement) or a density defined by a formula (a posterior, an energy-based model), we face the problem of this book. This chapter goes through the main cases and shows that their standard algorithms are the tools of Chapters 2 and 3 in disguise.

12.1. Where unnormalised densities appear in ML

Table 12.1 Sampling problems in machine learning. p_{base} is a pretrained generative model, r a reward, $p(y \mid x)$ a likelihood.

Problem	Unnormalised target density $\tilde{\pi}(x)$	Section
Reward fine-tuning, alignment	$p_{\text{base}}(x)\, e^{\frac{r(x)}{\lambda}}$	12.2
Conditional generation, inverse problems	$p_{\text{base}}(x)\, p(y \mid x)$	12.3
Composition of models	$p_1(x)^{\gamma_1}\, p_2(x)^{\gamma_2}$	12.3
Bayesian inference	$p(\theta) \prod_i p(y_i \mid x_i, \theta)$	12.4
Energy-based models	$e^{-E_\theta(x)}$	12.5
Latent-variable models	$p_\theta(x, z)$ as a function of z	12.6
Discrete objects	$e^{r(x)}$ on graphs or sequences	12.7

Intuition

Generative modelling learns a distribution from *samples*; neural sampling learns it from an *energy*. Real systems need both: a model is first trained on data (cheap, broad coverage) and then reshaped by an energy (a reward, a likelihood, physics). The reshaping is a sampling problem whose "prior" is the pretrained model.

12.2. Reward fine-tuning is sampling from a tilted density

Fine-tuning a generative model to increase a reward r (an aesthetic score, a binding affinity, a human preference) while staying close to the pretrained density p_{base} is usually written as KL-regularised reward maximisation, the objective of RL from human feedback (Ouyang et al., 2022; Black et al., 2024).

Proposition 12.1 (KL-regularised reward maximisation). For $\lambda > 0$ and any density q with $\mathrm{KL}(q\|p_{\text{base}}) < \infty$,

$$\mathbb{E}_{x\sim q}[r(x)] - \lambda\,\mathrm{KL}(q\|p_{\text{base}}) = \lambda \log Z_\lambda - \lambda\,\mathrm{KL}(q\|p_\lambda), \qquad p_\lambda(x) = \frac{p_{\text{base}}(x)\, e^{\frac{r(x)}{\lambda}}}{Z_\lambda}.$$

Hence the unique maximiser is the tilted density p_λ, and the maximal value is $\lambda \log Z_\lambda$.

Proof. $\mathbb{E}_q[r] - \lambda\,\mathbb{E}_q\left[\log \frac{q}{p_{\text{base}}}\right] = -\lambda\,\mathbb{E}_q\left[\log \frac{q}{p_{\text{base}} e^{\frac{r}{\lambda}}}\right] = -\lambda\,\mathrm{KL}(q\|p_\lambda) + \lambda \log Z_\lambda$; conclude with Lemma 1.6. ∎

Intuition

"Maximise reward, but do not drift too far" has a closed-form answer: multiply the old density by $e^{\frac{r}{\lambda}}$. Small λ concentrates on the reward's maximisers (risking reward hacking); large λ keeps the base model. So fine-tuning is a *sampling* problem with energy $\mathcal{E} = -\log p_{\text{base}} - \frac{r}{\lambda}$, and every tool of this book applies. Loss of diversity in fine-tuned models is the mode-seeking behaviour of the reverse KL in Proposition 12.1.

For a diffusion or flow model, p_{base} is the time-1 density of a generative SDE with path law $\mathbb{P}^{\text{base}}$: $\mathrm{d}X_t = b_t(X_t)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t$, $X_0 \sim \mathcal{N}(0, \mathrm{I})$, where b is the pretrained network. We cannot evaluate p_{base}, but we do not need to.

Theorem 12.2 (Fine-tuning by control needs a memoryless schedule). Consider the control problem Equations (3.2) and (3.3) with base $\mathbb{P}^{\text{base}}$, $c \equiv 0$ and terminal cost $g = -\frac{r}{\lambda}$. If $\mathbb{P}^{\text{base}}$ is memoryless (X_0 and X_1 independent), the optimally controlled process has time-1 density p_λ, and for every control

$$\mathbb{E}_{\mathbb{P}^u}[r(X_1)] - \lambda\,\mathrm{KL}(\mathbb{P}^u\|\mathbb{P}^{\text{base}}) = -\lambda J(u) \;\le\; \lambda \log Z_\lambda,$$

with equality if and only if $\mathbb{P}^u = \mathbb{P}^\star$. At the optimum the KL between path laws equals the KL between time-1 densities, $\mathrm{KL}(p_\lambda\|p_{\text{base}})$. Without memorylessness the time-1 density is biased by the initial value function (Equation (3.9)).

Proof. The identity is Corollary 2.10 applied to J. Memorylessness makes $h(0,x) = \mathbb{E}\left[e^{\frac{r(X_1)}{\lambda}} \mid X_0 = x\right] = Z_\lambda$ the same for all x, so by Theorem 3.2 $\mathbb{P}^\star$ is $\mathbb{P}^{\text{base}}$ reweighted by $\frac{1}{Z_\lambda} e^{\frac{r(X_1)}{\lambda}}$, whose time-1 density is $\frac{1}{Z_\lambda} p_{\text{base}}\, e^{\frac{r}{\lambda}} = p_\lambda$ (this is Proposition 3.7 with $\tilde\pi = p_{\text{base}} e^{\frac{r}{\lambda}}$). The maximal value follows from Equation (3.5). Since $\mathbb{P}^\star$ reweights only the endpoint, its conditional law given X_1 equals that of the base, and Lemma 1.7 gives $\mathrm{KL}(\mathbb{P}^\star\|\mathbb{P}^{\text{base}}) = \mathrm{KL}(p_\lambda\|p_{\text{base}})$. ∎

Intuition

Fine-tuning by control changes the trajectories but not the initial noise. If the initial noise has no influence on what the pretrained sampler generates (memoryless), all the reweighting can be done along the trajectory, and the result is exactly the tilted density. If the initial noise matters, as for a deterministic ODE sampler, some noises lead to high-reward outputs more often, and the correct answer would require reweighting the noise itself.

Memoryless schedules. The deterministic sampler of a flow model (the probability-flow ODE, or DDIM; Song et al., 2021a) is the opposite of memoryless: X_1 is a function of X_0. By Theorem 2.13, the same marginal densities can be generated by an SDE with any noise level, using the model's score to correct the drift. Domingo-Enrich et al. (2025) identify the noise level that makes this SDE memoryless; for the linear interpolant $x_t = (1-t)x_0 + tx_1$ (noise at $t=0$) it is $\sigma_t^2 = \frac{2(1-t)}{t}$, infinite at $t = 0$ so that the first instants erase all memory of the initial noise. The DDPM ancestral sampler is approximately memoryless in the same sense. Fine-tuning then proceeds by Adjoint Matching: the adjoint Equation (3.10) is integrated backwards along each simulated path from $a(1) = -\frac{\nabla r(X_1)}{\lambda}$, using vector–Jacobian products of the pretrained drift, and the control is regressed onto $-\sigma_t a(t)$ (Theorem 3.8). Adjoint Sampling (Chapter 9) is the special case with zero base drift and no pretrained model: "fine-tuning from scratch".

Algorithm 6 Adjoint Matching for reward fine-tuning (memoryless schedule)

Require: pretrained drift b, memoryless σ_t, reward r, temperature λ, fine-tuned drift $b + \sigma u_\theta$

loop
  simulate $X_{0:N}$ with $\mathrm{sg}(u_\theta)$ (no gradients); compute $\nabla r(X_N)$
  $a_N \leftarrow -\frac{\nabla r(X_N)}{\lambda}$; for $n = N-1, \dots, 0$: $a_n \leftarrow a_{n+1} + h\,\nabla b_{t_n}(X_n)^\top a_{n+1}$ ▷ vector–Jacobian products
  gradient step on $\sum_n |u_\theta(X_n, t_n) + \sigma_{t_n} a_n|^2$
end loop

12.3. Guidance and inverse problems

Given a pretrained diffusion model for p_{base} and an observation y with likelihood $p(y \mid x)$ (a classifier, a blurred image, a partial structure), we want the posterior density $p(x \mid y) \propto p_{\text{base}}(x)\, p(y \mid x)$.

Proposition 12.3 (Guidance is a Doob h-transform). Let $\mathbb{P}^{\text{base}}$ be the generative SDE with time-1 density p_{base}, and define $h(t,x) = \mathbb{E}_{\mathbb{P}^{\text{base}}}[p(y \mid X_1) \mid X_t = x] =: p_t(y \mid x)$. The posterior path law, $\mathbb{P}^{\text{base}}$ reweighted by $\frac{p(y|X_1)}{p(y)}$, has time-1 density $p(x \mid y)$ and is the law of the SDE

$$\mathrm{d}X_t = \big(b_t(X_t) + \sigma_t^2 \nabla_x \log p_t(y \mid X_t)\big)\,\mathrm{d}t + \sigma_t\,\mathrm{d}B_t, \tag{12.1}$$

started from the density $\mathcal{N}(x; 0, \mathrm{I})\,\frac{h(0,x)}{p(y)}$, which is $\mathcal{N}(0, \mathrm{I})$ when the generative SDE is memoryless.

Proof. Apply Theorem 2.14 with $g = p(y \mid \cdot)$: $\mathbb{P}^{\text{base}}$ reweighted by $\frac{g(X_1)}{h(0,X_0)}$ is the law of Equation (12.1) started at $\mathcal{N}(0, \mathrm{I})$. The posterior path law differs by the factor $\frac{h(0,X_0)}{p(y)}$, which depends only on X_0; so it has the same conditional law given X_0, and initial density $\mathcal{N}(0, \mathrm{I})\frac{h(0,\cdot)}{p(y)}$. Its time-1 density is $\frac{p_{\text{base}}(x)p(y|x)}{p(y)}$. Under memorylessness, $h(0,x) = \mathbb{E}[p(y \mid X_1)] = p(y)$. ∎

Intuition

Classifier guidance (Dhariwal and Nichol, 2021) adds $\sigma_t^2 \nabla \log p_t(y \mid x)$, the gradient of "how likely is the observation, given that I am at x now and will continue with the pretrained dynamics". It is exact only with this *time-dependent* likelihood, which is intractable. Diffusion posterior sampling (Chung et al., 2023) approximates it by $p(y \mid \hat{x}_1(x,t))$ with the Tweedie denoiser $\hat{x}_1$ (Proposition 8.1), which is biased. Removing this bias is a neural-sampling problem: either learn h (it is a value function, Corollary 3.3), or correct the approximate guidance with SMC.

Exact corrections. Twisted SMC (Wu et al., 2023) uses an approximate h as a twisting function inside SMC, and becomes exact as the number of particles grows. Feynman–Kac correctors (Skreta et al., 2025) handle products and powers of densities, $p_1^{\gamma_1} p_2^{\gamma_2}$, which include classifier-free guidance (Ho and Salimans, 2021) with weight γ: the usual guided sampler does *not* sample $p(x \mid y)^\gamma p(x)^{1-\gamma}$, but SMC with Jarzynski weights (Theorem 7.1) makes it exact. Finally, h can be learned by solving the control problem with terminal cost $-\log p(y \mid x)$, i.e. Theorem 12.2 with $r = \log p(y \mid \cdot)$ and $\lambda = 1$.

12.4. Bayesian inference

For parameters θ, prior density $p(\theta)$ and data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, the posterior has energy

$$\mathcal{E}(\theta) = -\log p(\theta) - \sum_{i=1}^{n} \log p(y_i \mid x_i, \theta).$$

Three features distinguish it from physical targets. (i) *Cost grows with n*: one exact energy evaluation is a pass over the data, so energy-efficient samplers (Chapter 9) matter; mini-batches give unbiased gradients, as in stochastic-gradient Langevin dynamics (Welling and Teh, 2011), but biased energies and hence biased importance weights. (ii) *Concentration*: the posterior narrows like $\frac{1}{\sqrt{n}}$, so its curvature grows like n, and samplers whose drift contains $\sigma_t^2 \nabla \mathcal{E}$ become stiff and need small steps. (iii) *Symmetries*: for neural networks, permuting hidden units gives exponentially many equivalent modes, which should be removed (Section 11.4) rather than covered. Standard benchmarks are Bayesian logistic regression (German Credit, Ionosphere, Sonar), a log-Gaussian Cox process and Brownian-motion inference (Arbel et al., 2021; Blessing et al., 2024); neural Schrödinger–Föllmer samplers have been used for Bayesian neural networks (Vargas et al., 2023b).

Table 12.2 Bayesian logistic regression ($n = 60$ data points, $d = 5$ parameters, prior $\mathcal{N}(0, 2\mathrm{I})$) with Adjoint Sampling (150×512 energy-gradient evaluations, $\sigma_{\max} = 3$), against a 35,000-sample MALA reference. Errors are in units of the posterior standard deviation.

	posterior-mean error	mean posterior std	normalised ESS
MALA (reference)	–	0.463	–
Adjoint Sampling	0.059	0.461	–
Adjoint Sampling + path weights	0.025	–	0.436

Table 12.2 is produced by Listing 12.1. It also illustrates point (ii): on the same model with $n = 300$ and $d = 10$, the default schedule ($\sigma_{\max} = 5$) diverged, and $\sigma_{\max} = 2$ gave posterior widths twice too large and an ESS near zero, because the drift was too stiff for the Euler steps early in the trajectory. The path weights of Adjoint Sampling are the discrete-time log-weights of Theorem 6.1, exact for the simulated chain, and they halve the error of the posterior mean.

Listing 12.1 Path log-weights for Adjoint Sampling (exact for the Euler chain), used for Bayesian logistic regression.

```python
@torch.no_grad()
def sample_with_logw(f, energy, n, d, N=200):
    """Simulate AS and accumulate the discrete-time path log-weight (Algorithm 1, line 3):
    log w = log pi~(x_N) - log p_1^base(x_N) + sum_k [log N(x_{k+1}; x_k, s_k^2 h) - log
     N(x_{k+1}; x_k + s_k u h, s_k^2 h)]."""
    x, h = torch.zeros(n, d), 1.0 / N
    logw = torch.zeros(n)
    for k in range(N):
        s = AS.sigma(k * h)
        drift = -s**2 * f(x, torch.tensor([[k * h]])) * h          # sigma * u * h with u =
     -sigma f
        noise = s * math.sqrt(h) * torch.randn(n, d)
        x_new = x + drift + noise
        # log N(x_new; x, s^2 h) - log N(x_new; x + drift, s^2 h)
        logw += (-(noise + drift)**2 + noise**2).sum(-1) / (2 * s**2 * h)
        x = x_new
    v = sum(AS.sigma(k * h)**2 * h for k in range(N))           # exact terminal variance of the
    log_pbase = -(x**2).sum(-1) / (2 * v) - 0.5 * d * math.log(2 * math.pi * v)   # discrete base
     chain
    return x, logw - energy(x) - log_pbase
```

12.5. Training energy-based models

An energy-based model (EBM) $p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}$ is the most flexible density model, and training it by maximum likelihood requires sampling at every step.

Proposition 12.4 (Maximum-likelihood gradient of an EBM). $\nabla_\theta \mathbb{E}_{x\sim p_{\text{data}}}[\log p_\theta(x)] = -\mathbb{E}_{x\sim p_{\text{data}}}[\nabla_\theta E_\theta(x)] + \mathbb{E}_{x\sim p_\theta}[\nabla_\theta E_\theta(x)]$.

Proof. $\log p_\theta = -E_\theta - \log Z_\theta$, and $\nabla_\theta \log Z_\theta = \frac{1}{Z_\theta}\int -\nabla_\theta E_\theta(x)\, e^{-E_\theta(x)}\,\mathrm{d}x = -\mathbb{E}_{p_\theta}[\nabla_\theta E_\theta]$. ∎

Intuition

Push the energy down on data (positive phase) and up on the model's own samples (negative phase). The negative phase needs samples from the current model, classically from short MCMC runs (Hinton, 2002; Du and Mordatch, 2019), which are biased when chains do not mix. A neural sampler trained on E_θ can reuse its work across iterations because E_θ changes slowly; if it has a density or path weights, the negative phase can be importance-weighted.

12.6. Variational inference and latent-variable models

In a latent-variable model with joint density $p_\theta(x, z)$, the posterior $p_\theta(z \mid x) \propto p_\theta(x, z)$ is an unnormalised density in z whose normalising constant is $p_\theta(x)$. A variational autoencoder (Kingma and Welling, 2014) trains an amortised sampler $q_\phi(z \mid x)$ with the ELBO of Proposition 1.10, i.e. by reverse KL. Normalising flows (Rezende and Mohamed, 2015) and diffusion samplers (Theorem 6.1) are richer variational families for the same objective, and their path ELBOs are valid lower bounds on $\log p_\theta(x)$. Every neural sampler is therefore a variational-inference method; what is specific to sampling is that the target is fixed and the sampler is judged by sample quality and $\log Z$.

12.7. Discrete and structured spaces

On combinatorial spaces (graphs, strings, sequences), the analogue of a diffusion sampler is a policy that builds an object step by step. GFlowNets (Bengio et al., 2021; Malkin et al., 2022) sample objects with probability $\propto e^{r(x)}$ and are trained with trajectory balance, the log-variance loss of Proposition 6.3 on construction paths; discrete diffusion samplers and their enhanced-sampling variants (e.g. metadynamics-biased discrete samplers, Du et al., 2026) extend this to Ising-type models and combinatorial optimisation. KL-regularised fine-tuning of language models is Proposition 12.1 on token sequences, which is why sampling methods (SMC over partial sequences, twisted proposals) are natural there too.

Key takeaways

- Reward fine-tuning, guidance, Bayesian posteriors, EBM training and variational inference are all "sample from $\tilde{\pi}$" with a pretrained model, data or a likelihood inside the energy.
- KL-regularised reward maximisation samples the tilted density (Proposition 12.1); doing it by control needs a memoryless base (Theorem 12.2).
- Guidance is a Doob h-transform whose h is the time-dependent likelihood (Proposition 12.3); standard approximations are biased and can be corrected by SMC or by learning h.

Open problems and research questions

- **Reward hacking as mode collapse.** Fine-tuned models collapse onto a few high-reward outputs. Mass-covering objectives (Chapters 6 and 7) applied to fine-tuning could preserve diversity; how should λ and the divergence be chosen?
- **Exact guidance at scale.** Twisted SMC is exact but needs many particles per sample; learned value functions are cheap but biased. Combine them with guarantees for image- and protein-scale models.
- **Samplers for big data.** Posteriors with $n \sim 10^6$ need samplers trained with mini-batch energies; how can path weights stay consistent when $\mathcal{E}$ is only estimated?
- **Amortised posteriors.** Train one sampler for all data sets of a model class, and know when to trust it on a new data set.
- **Joint EBM and sampler training.** The sampler chases a moving target E_θ; conditions for stable joint training are open.

CHAPTER 13

Application: molecular generation and equilibrium sampling

Goal	Map the methods of the book onto concrete molecular tasks, with their representations, energy models, evaluation protocols and pitfalls, and give a reference pipeline.
Prerequisites	All previous chapters; basic chemistry vocabulary (bonds, torsions, conformers).
Key papers	Noé et al. (2019); Havens et al. (2025); Liu et al. (2025); Nam et al. (2025); Jing et al. (2022); Klein and Noé (2024); Tan et al. (2025a); Lewis et al. (2025).

Overview. Molecules are where neural samplers were born and where they are tested hardest: the energy is expensive, the density is concentrated on narrow, well-separated regions, and the answers can be checked against physics. This chapter connects the general theory to the specific choices one has to make for molecules.

13.1. The target

For a molecule with n atoms at positions $x \in \mathbb{R}^{3n}$, the equilibrium (Boltzmann) density at temperature T is

$$\pi(x) = \frac{1}{Z} \exp\Big(-\frac{U(x)}{k_B T}\Big), \qquad \mathcal{E}(x) = \frac{U(x)}{k_B T},$$

where U is the potential energy. At $300\,\mathrm{K}$, $k_B T \approx 0.596\,\mathrm{kcal/mol}$, so an error of $1\,\mathrm{kcal/mol}$ in U changes probabilities by a factor $e^{\frac{1}{0.596}} \approx 5$; an error of $0.1\,\mathrm{eV}$ ($2.3\,\mathrm{kcal/mol}$) changes them by a factor of about 50. The energy model must therefore be accurate. In increasing cost and accuracy, energy models are classical force fields (with implicit solvent for peptides), semi-empirical quantum methods, machine-learned interatomic potentials (MLIPs, e.g. NequIP, MACE, eSEN; Batzner et al., 2022; Batatia et al., 2022; Fu et al., 2025), and density functional theory. With an MLIP, one force evaluation costs about one forward and backward pass of a large equivariant network; this is the cost model of Chapter 4. A perfect sampler of an MLIP's density inherits the MLIP's errors, so sampler and energy model should be evaluated separately.

13.2. Choosing a representation

Cartesian coordinates with equivariant networks are the default for neural samplers: they are general and the energy model works in them natively. Internal coordinates make stiff bond vibrations explicit and are standard for flows on small peptides. Torsional spaces are low-dimensional but freeze bonds and angles, so they target an approximation of the Boltzmann density.

13.3. Tasks

Table 13.1 State spaces for molecules and how each handles symmetry and densities.

Space	dim.	Symmetry handling	Density bookkeeping
Cartesian, zero CoM	$3n-3$	equivariant network for rotations and permutations; centring (Proposition 11.7)	volume on V; exact
Internal coordinates	$3n-6$	invariant by construction	factor $\prod_i r_i^2 \sin\theta_i$ (Section 11.5)
Torsions only	m	invariant; torus $\mathbb{T}^m$	Fixman factor (Proposition 11.8); bonds and angles frozen
Residue frames	$6L$	one rigid frame per residue, global equivariance	invariant measure on rotations
Periodic box	$3n$	torus $\mathbb{T}^{3n}$	flat; wrapped bridges (Proposition 11.5)

13.3.1. Equilibrium sampling of a fixed molecule

Goal: samples from π for one molecule (alanine dipeptide, small peptides) with the correct populations of its metastable states, and free-energy differences between them. *Methods:* Boltzmann generators with reweighting (FAB, Section 5.4; sequential BGs), diffusion samplers (Chapter 6), ASBS and NAAS (Chapter 10), WT-ASBS when good collective variables are known. *Evaluation:* KL or Wasserstein distances between the densities of the two backbone angles (Ramachandran plots), Wasserstein distance between energy histograms, state populations, and ESS when weights exist, compared with long simulations or enhanced sampling. Current energy-only samplers typically find the high-density states; low-population states are where they still fail (Liu et al., 2025), and collective-variable biases recover them (Nam et al., 2025).

13.3.2. Amortised conformer generation

Goal: for any molecular graph c, generate a diverse set of low-energy 3-D structures. In the *data-driven* setting, models learn from conformer databases (GEOM, generated with the CREST workflow; Axelrod and Gómez-Bombarelli, 2022; Pracht et al., 2020): GeoDiff, torsional diffusion, ET-Flow, MCF (Xu et al., 2022; Jing et al., 2022; Hassan et al., 2024; Wang et al., 2024). In the *energy-only* setting, samplers learn $\pi_c \propto e^{-\mathcal{E}(\cdot;c)}$ without conformer data: Adjoint Sampling and ASBS trained on SPICE molecules (Eastman et al., 2023) with the eSEN potential, and evaluated on held-out SPICE and GEOM-DRUGS molecules (Havens et al., 2025; Liu et al., 2025). Strictly, conformer generation asks for the set of low-energy basins, not for the Boltzmann density; samples are often relaxed by a few optimisation steps before evaluation.

Metrics. With a generated set S_g, a reference set S_r, and the RMSD after optimal alignment (Kabsch, proper rotations only):

$$\text{COV-R}(\delta) = \frac{\left|\{r \in S_r : \min_{g\in S_g} \mathrm{RMSD}(g,r) < \delta\}\right|}{|S_r|}, \qquad \text{AMR-R} = \frac{1}{|S_r|}\sum_{r\in S_r} \min_{g\in S_g} \mathrm{RMSD}(g,r),$$

and the precision versions COV-P and AMR-P with the roles of S_g and S_r swapped; δ is typically between 0.5 and 1.25 Å. Recall rewards diversity, precision rewards quality; report both.

Listing 13.1 Centring, Kabsch RMSD without reflections, and coverage metrics.

```python
import torch

def com_project(x):
    """Project (B, n, 3) coordinates or noise onto the zero-centre-of-mass subspace (Prop.
    11.7)."""
```

```
    return x - x.mean(dim=1, keepdim=True)

def kabsch_rmsd(a, b):
    """RMSD between conformers a, b of shape (n, 3) after optimal rotation (no reflection)."""
    a, b = a - a.mean(0), b - b.mean(0)
    U, S, Vt = torch.linalg.svd(a.T @ b)
    d = torch.sign(torch.linalg.det(U @ Vt))                # forbid reflections (chirality)
    D = torch.diag(torch.tensor([1.0, 1.0, d.item()]))
    R = U @ D @ Vt                                          # rotation minimising |a R - b|
    return ((a @ R - b) ** 2).sum(-1).mean().sqrt()

def coverage_amr(gen, ref, delta=0.75):
    """COV-R, AMR-R (recall: every reference matched?) and COV-P, AMR-P (precision)."""
    D = torch.stack([torch.stack([kabsch_rmsd(g, r) for r in ref]) for g in gen])  # (G, R)
    rec, prec = D.min(0).values, D.min(1).values
    return {"COV-R": (rec < delta).float().mean().item(), "AMR-R": rec.mean().item(),
            "COV-P": (prec < delta).float().mean().item(), "AMR-P": prec.mean().item()}

if __name__ == "__main__":
    torch.manual_seed(0)
    ref = torch.randn(5, 12, 3)                              # 5 reference conformers, 12 atoms
```

13.3.3. Transferable sampling

Goal: one model for a family of molecules (e.g. all dipeptides), with Boltzmann accuracy on unseen members after reweighting. Transferable BGs and their successors are trained on simulation data and corrected with energies (Klein and Noé, 2024; Tan et al., 2025b). Energy-only amortised samplers (AS, ASBS) transfer across SPICE molecules but provide no importance weights by default; their path weights (Section 9.5) are the natural bridge between the two lines of work.

13.3.4. Free energies

For two states (or conditions) A and B, the free-energy difference is $\Delta F = -k_B T \log \frac{Z_B}{Z_A}$. Neural samplers provide $\log Z$ estimates from weights (Proposition 1.10); targeted free-energy perturbation with a learned map between states (Wirnsberger et al., 2020); SMC and Jarzynski estimates along annealing paths (Theorem 7.1); and free-energy profiles along collective variables from biased samplers with reweighting (Nam et al., 2025). Samples from several states are best combined with MBAR (Shirts and Chodera, 2008). An unbiased $\hat{Z}$ does not give an unbiased $\log \hat{Z}$; report confidence intervals from independent runs.

13.3.5. Transition paths

Rare transitions from a state A to a state B are path sampling problems: condition the dynamics on $X_0 \in A$ and $X_T \in B$. By Theorem 2.14, the conditioned process has the extra drift $\sigma^2 \nabla \log h$ with $h(t, x) = \Pr(X_T \in B \mid X_t = x)$, a control problem with terminal reward $\mathbb{1}_B$. Holdijk et al. (2023) learn this control without collective variables; Du et al. (2024) use a variational form of Doob's h-transform that avoids simulation.

13.3.6. Proteins and materials

For proteins, the strongest ensemble generators are trained on structures and simulations: the diffusion module of AlphaFold 3 (Abramson et al., 2024), AlphaFlow (Jing et al., 2024), and BioEmu (Lewis et al., 2025), which emulates equilibrium ensembles and is fine-tuned to match measured stabilities. Purely energy-based samplers have not reached protein scale; data-trained

proposals corrected by energy-based SMC are the realistic route. For amorphous materials, ATLAS (Cheng et al., 2026) trains a periodic (torus) diffusion sampler from energies and forces, transfers from small to larger simulation cells, and computes free energies with path weights and MBAR.

13.4. A reference pipeline

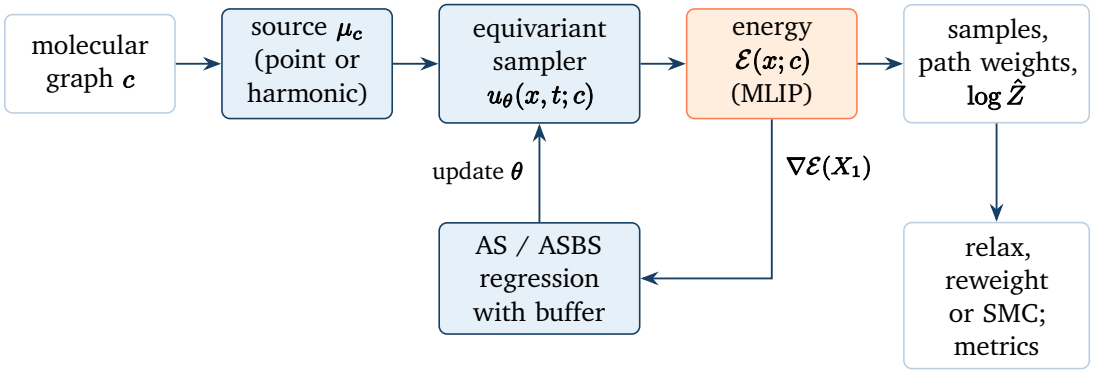


Figure 13.1 A reference pipeline for amortised, energy-only sampling of molecules.

Algorithm 7 Amortised conformer or equilibrium sampling with ASBS (practical recipe)

Require: training graphs $\{c\}$, MLIP $\mathcal{E}(\cdot;c)$, temperature T, equivariant network $u_\theta(x,t;c)$, corrector $h_\phi(x;c)$

choose the source μ_c: a point (AS) or the harmonic prior on the centred subspace (ASBS)
for stage $k = 1,\dots,K$ **do**
  for many iterations **do**
    draw a batch of graphs c and $X_0 \sim \mu_c$; simulate the sampler with centred noise
    evaluate $\nabla\mathcal{E}(X_1;c)$ once per sample and clip its norm; store $(c, X_0, X_1, \nabla\mathcal{E})$ in the buffer
    regression steps on Equation (10.1) with bridges pinned at (X_0, X_1), centred
  end for
  corrector regression Equation (10.2) on stored pairs
end for
Inference: for a new graph, simulate; then relax with a few MLIP steps (conformers), or reweight or run SMC with path weights (equilibrium); evaluate with Listing 13.1 and angle and energy metrics

Pitfalls.

- **Units and temperature.** Check that $\mathcal{E} = \frac{U}{k_B T}$ in consistent units; an error by a factor $k_B T$ silently samples the wrong temperature.
- **Stiff bonds.** Bond vibrations are fast; Euler steps that are too large break bonds and produce huge forces. Use more steps near $t = 1$, harmonic priors, or internal coordinates for bonds.
- **Force clipping** biases samples near repulsive walls; report the clipping threshold and inspect the tail of the energy histogram.
- **Chirality and permutations.** Use rotation equivariance without reflections; align with proper rotations only (Listing 13.1); decide whether equivalent hydrogens are interchangeable when computing RMSD.

- **References.** Conformer references (CREST) are produced with a different energy model; disagreements may reflect the energy model rather than the sampler.

Generative-modelling view

Molecular generation is where data-driven generative modelling and energy-based sampling meet most directly. Data-driven conformer and ensemble models (GeoDiff, torsional diffusion, ET-Flow, BioEmu) are trained like image diffusion models and inherit the biases of their training sets; energy-driven samplers (AS, ASBS) are unbiased in principle but expensive and mode-seeking. The productive pattern is the one of Chapter 12: a data-trained model as proposal or base, an energy as tilt or correction, and SMC or importance weights to make the result exact.

Key takeaways

- Molecular tasks differ in what they demand: correct populations (peptides, free energies), sets of low-energy structures (conformers), or rare paths (transitions). Choose the sampler and the metrics accordingly.
- Energy-only, amortised, regression-based samplers (AS, ASBS) are the most scalable data-free methods today; exactness requires weights or SMC; at protein scale, data-trained proposals with energy-based correction lead.

Open problems and research questions

- **Boltzmann accuracy at scale.** No energy-only method yet samples a solvated protein's equilibrium ensemble with correct populations. Which combination of amortised proposals, SMC and enhanced-sampling biases closes the gap?
- **Joint learning of energy and sampler.** MLIPs are trained near equilibrium; samplers visit high-energy regions where MLIPs extrapolate badly. Active-learning loops that improve the MLIP where the sampler goes are largely unexplored.
- **Explicit solvent.** Sampling a solute with thousands of solvent molecules in a periodic box is a different scale; implicit-solvent targets are approximations whose error is hard to control.
- **Learned collective variables.** WT-ASBS needs collective variables; learning them from the sampler's own exploration would remove a manual step.
- **Evaluation standards.** Shared reference ensembles with uncertainties, for a ladder of systems from dipeptides to small proteins, are needed to compare samplers fairly.

CHAPTER 14

Open problems and a research roadmap

Goal	Step back: see every method in the book as one template, collect the open questions by theme, and turn them into a reading plan and concrete first projects.
Prerequisites	The whole book.
Key papers	Blessing et al. (2024); He et al. (2025); Liu et al. (2025); Albergo and Vanden-Eijnden (2025).

14.1. One template for all samplers

Almost every method in this book is an instance of the following recipe.

1. **A reference process** $\mathbb{Q}$: a point mass or a prior followed by Brownian motion, OU noise, annealed Langevin dynamics, or a manifold diffusion.
2. **A target path measure**, a tilt of the reference by the energy: $\mathbb{P}^\star \propto \mathbb{Q} \cdot (\text{endpoint factor}) \cdot e^{-\text{running cost}}$, possibly with a correction that makes the initial law consistent (memorylessness, Schrödinger potentials, or NAAS's prior process).
3. **A parameterisation** of the generative side: a flow map, a control, a score, an energy, or a transport field.
4. **A loss** that vanishes exactly at the target: a path-space divergence needing simulation (reverse KL), an off-policy divergence (log-variance, trajectory balance), or a fixed-point regression (energy matching, adjoint matching, corrector matching, PINN residuals).
5. **A correction**: importance weights, SMC with resampling, MCMC steps, or none.

Table 14.1 places the main methods in this template. Reading new papers through it makes it quick to see what is genuinely new.

14.2. The big open questions

Theory.

- *Convergence of fixed-point regressions.* Adjoint Sampling, iDEM, ASBS and NETS are trained by iterating "sample with the current model, regress onto targets". Beyond the exact-regression IPF argument (Theorem 10.3), there are almost no convergence or stability results with function approximation, finite buffers and stale samples.
- *Sample complexity versus MCMC.* When can learned transport provably beat Langevin or parallel tempering in energy evaluations, as a function of barrier heights and dimension? Lower bounds for any method using only $\mathcal{E}$ and $\nabla\mathcal{E}$ queries would sharpen the question.
- *A theory of mode collapse.* Reverse-KL-type objectives are mode-seeking, but samplers trained with exploration often find all modes yet misweight them (Figure 9.2). Characterise the error in mode weights in terms of the loss, the reference process and the exploration strategy.
- *Discretisation.* Most theory is in continuous time; training and weights are in discrete time. Exactly reversible discretisations and their effect on bias and variance remain under-explored.

Table 14.1 Main methods in the common template.

Method	Reference	Learned object	Loss	Correction
BG / FAB	none (map)	flow	reverse KL / D_2 via AIS	IS
PIS	δ_0 + BM	control	reverse KL	IS
DDS	OU	control	reverse KL	IS
DIS / bridges	learned	both drifts	KL / LV	IS
LV / TB samplers	any	control (+ $\log Z$)	log-variance	IS
CMCD / NETS	annealed Langevin	transport b_t	LV / PINN	Jarzynski, SMC
SCLD, PDDS	annealed / OU	control	sub-trajectory LV	SMC
iDEM / BNEM	VE noising	score / energy	MC regression	none
AS	δ_0 + BM	control	reciprocal adjoint matching	optional IS
ASBS	prior + BM	control + corrector	adjoint + corrector matching	none
NAAS	annealed Langevin	two controls	adjoint matching	none
WT-ASBS	prior + BM, biased	control + bias	ASBS + metadynamics	reweighting
Adjoint Matching (fine-tuning)	pretrained model, memoryless	control	adjoint matching	none
Guidance / twisted SMC	pretrained model	guidance h (approx.)	none or value regression	SMC
VAE encoder	prior × likelihood	amortised $q_\phi(z\|x)$	reverse KL (ELBO)	IS (IWAE)
GFlowNets	construction policy	forward/backward policies	trajectory balance	none

Algorithms.

- *Mass-covering without data*: objectives whose minimisers have correct mode weights, at a cost comparable to adjoint matching.
- *Energy efficiency*: count and minimise energy evaluations, not gradient steps; reuse each energy evaluation as many times as possible without bias.
- *Combining control with Monte Carlo*: use path weights and SMC not only at test time but during training, with controlled variance.
- *Learning the reference*: priors, noise schedules, annealing paths and CV biases are hand-designed; learning them is a meta-level control problem.
- *Amortisation*: one sampler for a chemical space, with guarantees on unseen molecules (calibrated weights, or detectable failure).
- *Fast inference*: distillation to few-step or one-step samplers that keep tractable weights.

Machine learning and generative modelling. *Mass-covering fine-tuning*: reward fine-tuning collapses diversity exactly as reverse-KL samplers collapse modes; transfer the mass-covering objectives of Chapters 6 and 7 to fine-tuning of image, protein and language models. *Exact guidance at scale*: unbiased conditional sampling from pretrained diffusion models with a bounded number of particles. *Samplers for big-data posteriors*: training and weighting with minibatch energies. *Neural samplers as components*: amortised negative phases for energy-based models, and expressive variational posteriors for latent-variable models, trained jointly with the model they serve (Chapter 12).

Geometry. Curvature-aware adjoints and bridges on $\mathrm{SO}(3)$ and $\mathrm{SE}(3)^n$; consistent schedules on product spaces; samplers on constraint manifolds with the correct reference measure (Chapter 11).

Molecules. Boltzmann accuracy at protein scale; co-training of MLIPs and samplers; explicit solvent and periodic systems; learned collective variables; shared reference ensembles and protocols (Chapter 13).

14.3. A reading plan

1. **Weeks 1–2: foundations.** Chapters 1 to 3 with Øksendal (2003) (Ch. 4–8) and Léonard (2014). Re-derive Theorems 2.12, 3.2 and 3.12 without notes.
2. **Weeks 3–4: classical neural samplers.** Noé et al. (2019); Midgley et al. (2023b); Zhang and Chen (2022); Vargas et al. (2023a); Richter and Berner (2024); the benchmark study Blessing et al. (2024). Run Listings 5.1 and 6.1.
3. **Weeks 5–6: annealing and energy matching.** Vaikuntanathan and Jarzynski (2008); Vargas et al. (2024); Albergo and Vanden-Eijnden (2025); Akhound-Sadegh et al. (2024); OuYang et al. (2026); He et al. (2025).
4. **Weeks 7–8: adjoint and bridge samplers.** Domingo-Enrich et al. (2025); Havens et al. (2025); Liu et al. (2025); Choi et al. (2025); Nam et al. (2025). Run Listings 9.1 and 10.1.
5. **Weeks 9–10: machine learning.** Chapter 12 with Domingo-Enrich et al. (2025); Dhariwal and Nichol (2021); Chung et al. (2023); Wu et al. (2023); Skreta et al. (2025); Levine (2018). Run Listing 12.1.
6. **Weeks 11–12: geometry and molecules.** De Bortoli et al. (2022); Chen and Lipman (2024); Jing et al. (2022); Mosso et al. (2026); Klein and Noé (2024); Tan et al. (2025a).

14.4. Starter projects

Each is scoped for a first paper and builds directly on the book.

1. **Torsional Adjoint Sampling.** Implement Proposition 11.9 with an energy on the torsion torus of drug-like molecules, including the Fixman correction (Proposition 11.8); compare with torsional diffusion as a data-trained baseline and with Cartesian AS at equal energy budget.
2. **Weighted buffers for Adjoint Sampling.** Use the free path weights of AS (Section 9.5) to reweight or resample the replay buffer; analyse bias and variance on mixtures with known mode weights, then on alanine dipeptide.
3. **Error propagation in ASBS.** Measure how corrector errors propagate across IPF stages (Theorem 10.3) on Gaussian targets where everything is closed-form; derive bounds.
4. **SMC on top of amortised samplers.** Use an amortised AS/ASBS model as the proposal for annealed SMC (Algorithm 2) on unseen molecules; quantify the energy calls needed to reach a target ESS.
5. **Mass-covering reward fine-tuning.** Fine-tune a small pretrained diffusion model (e.g. on a 2-D mixture or MNIST) towards $p_{\text{base}} e^{\frac{r}{\lambda}}$ with Adjoint Matching and with a log-variance loss on paths (Proposition 6.3); measure reward, diversity and the true tilted mode weights, which are computable on toy problems.
6. **Exact guidance with learned value functions.** Learn h of Proposition 12.3 by adjoint matching on an inverse problem, then use it as the twist in twisted SMC; compare bias and particle counts against diffusion posterior sampling.
7. **Energy-budget benchmark.** Re-run a set of samplers on DW-4, LJ-13 and alanine dipeptide with matched energy-evaluation budgets and shared metrics (Section 4.4), following the caution of He et al. (2025).

> *Start every new idea on a target where the answer is known in closed form (Gaussians, mixtures, the torus), count energy evaluations from day one, and never trust a single metric.*
>
> — Advice for the first year

APPENDIX A

Gaussian toolbox

Lemma A.1 (Gaussian conditioning). If $\begin{pmatrix} A \\ B \end{pmatrix} \sim \mathcal{N}\Big(\begin{pmatrix} m_A \\ m_B \end{pmatrix}, \begin{pmatrix} \Sigma_{AA} & \Sigma_{AB} \\ \Sigma_{BA} & \Sigma_{BB} \end{pmatrix} \Big)$ with Σ_{BB} invertible, then $A \mid B = b \sim \mathcal{N}\big(m_A + \Sigma_{AB}\Sigma_{BB}^{-1}(b - m_B),\ \Sigma_{AA} - \Sigma_{AB}\Sigma_{BB}^{-1}\Sigma_{BA}\big)$.

Proof. Let $K = \Sigma_{AB}\Sigma_{BB}^{-1}$ and $R = A - m_A - K(B - m_B)$. Then R is Gaussian with $\operatorname{Cov}(R, B) = \Sigma_{AB} - K\Sigma_{BB} = 0$, hence independent of B, with covariance $\Sigma_{AA} - K\Sigma_{BA}$. Writing $A = m_A + K(B - m_B) + R$ gives the conditional law. ∎

Intuition

Conditioning a Gaussian is linear regression: predict A from B with the best linear predictor K, and what is left over, the residual R, is independent of B. The conditional mean is the prediction and the conditional covariance is the residual covariance, which does not depend on the observed value b.

Lemma A.2 (Time-changed Brownian bridge). Let $X_t = x_0 + \int_0^t \sigma_s \, \mathrm{d}B_s$ and $\bar{\sigma}_{s,t}^2 = \int_s^t \sigma_r^2 \, \mathrm{d}r$. For $s < t < 1$,

$$X_t \mid (X_s = a, X_1 = y) \ \sim \ \mathcal{N}\Big(a + \frac{\bar{\sigma}_{s,t}^2}{\bar{\sigma}_{s,1}^2}(y - a),\ \frac{\bar{\sigma}_{s,t}^2 \, \bar{\sigma}_{t,1}^2}{\bar{\sigma}_{s,1}^2}\mathrm{I}\Big).$$

With $s = 0$, $a = x_0$, this is the bridge used in Lemmas 9.1 and 10.1. For $\sigma \equiv 1$ it is $\mathcal{N}(a + \frac{t-s}{1-s}(y-a), \frac{(t-s)(1-t)}{1-s}\mathrm{I})$.

Proof. By the Markov property, condition on $X_s = a$ and apply Lemma A.1 to (X_t, X_1): $X_t - a \sim \mathcal{N}(0, \bar{\sigma}_{s,t}^2\mathrm{I})$ and $X_1 - X_t \sim \mathcal{N}(0, \bar{\sigma}_{t,1}^2\mathrm{I})$ independent, so $\operatorname{Cov}(X_t, X_1) = \bar{\sigma}_{s,t}^2\mathrm{I}$, $\operatorname{Var} X_1 = \bar{\sigma}_{s,1}^2\mathrm{I}$. The conditional mean is $a + \frac{\bar{\sigma}_{s,t}^2}{\bar{\sigma}_{s,1}^2}(y - a)$ and the variance $\bar{\sigma}_{s,t}^2 - \frac{\bar{\sigma}_{s,t}^4}{\bar{\sigma}_{s,1}^2} = \frac{\bar{\sigma}_{s,t}^2 \bar{\sigma}_{t,1}^2}{\bar{\sigma}_{s,1}^2}$. ∎

Intuition

A bridge interpolates linearly between its pinned values in the "noise clock" $\bar{\sigma}_t^2$, not in physical time, and its variance is largest in the middle, vanishing at both pinned ends: the product $\bar{\sigma}_{s,t}^2\bar{\sigma}_{t,1}^2$ is the familiar $t(1-t)$ of the Brownian bridge in the right clock.

Lemma A.3 (Sequential bridge sampling). A whole bridge path on a grid $0 < t_1 < \cdots < t_K < 1$ can be sampled exactly by applying Lemma A.2 successively with $(s, a) = (t_{k-1}, X_{t_{k-1}})$; this is the backward kernel of PIS in Chapter 6.

Proof. By the Markov property of the bridge, X_{t_k} given $(X_0, \ldots, X_{t_{k-1}}, X_1)$ depends only on $(X_{t_{k-1}}, X_1)$. ∎

Intuition

A bridge is itself a Markov process, so a whole bridge path can be drawn one step at a time, each time treating the current point as the new start of a shorter bridge to the same end.

Lemma A.4 (OU transitions and bridges). For the OU process of Proposition 2.7, $\mathrm{Cov}(X_s, X_t \mid X_0) = \frac{\alpha_t}{\alpha_s}(1-\alpha_s^2)\mathrm{I}$ for $s < t$, so OU bridges are Gaussian with parameters given by Lemma A.1. In particular $X_s \mid (X_0 = x_0, X_t = x_t)$ has mean $\alpha_s x_0 + \frac{\alpha_t(1-\alpha_s^2)}{\alpha_s(1-\alpha_t^2)}(x_t - \alpha_t x_0)$.

Proof. From the explicit solution in Proposition 2.7, $X_t = \frac{\alpha_t}{\alpha_s}X_s +$ (independent noise), so $\mathrm{Cov}(X_s, X_t \mid X_0) = \frac{\alpha_t}{\alpha_s}\mathrm{Var}(X_s \mid X_0)$. Apply Lemma A.1. ∎

Intuition

The OU process is a Gaussian process with exponentially decaying memory, so everything about it, including its bridges, follows from its covariance, which is the decay factor times the variance accumulated so far.

Lemma A.5 (Gaussian KL). $\mathrm{KL}(\mathcal{N}(m_1,\Sigma_1)\|\mathcal{N}(m_2,\Sigma_2)) = \frac{1}{2}\big[\mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (m_2-m_1)^\top\Sigma_2^{-1}(m_2-m_1) - d + \log\frac{\det\Sigma_2}{\det\Sigma_1}\big]$.

Proof. Take $\mathbb{E}_{\mathcal{N}(m_1,\Sigma_1)}$ of the log-density ratio, using $\mathbb{E}[(X-m)^\top A(X-m)] = \mathrm{tr}(A\Sigma_1) + (m_1-m)^\top A(m_1-m)$. ∎

Intuition

The three terms measure mismatch of shape (trace), of location (Mahalanobis distance of the means) and of volume (log-determinant ratio); the $-d$ makes the total vanish when the two Gaussians coincide.

APPENDIX B

Code companion

All listings in the book are excerpts of the scripts below, which are shipped with the LaTeX source in the folder `code/`. They need Python ≥ 3.10, PyTorch ≥ 2.0, NumPy and SciPy, run on a laptop CPU, and were run to produce the numbers quoted in the text (random seed 0 unless stated).

Table B.1 Scripts, what they demonstrate, and their output when run as `python <script>` from the `code/` folder.

Script	Demonstrates	Output (CPU time)
`sinkhorn.py`	static Schrödinger bridge by log-domain IPF (Listing 3.1)	marginal errors $\approx 10^{-17}$ (< 1 s)
`flow_reverse_kl.py`	RealNVP Boltzmann generator, reverse KL, IS (Listing 5.1)	$\log \hat{Z} = 1.558$ vs. 1.5585; ESS 0.96 (≈ 35 s)
`pis_lv.py`	PIS reference + log-variance loss, off-policy exploration (Listing 6.1)	all 4 modes, weights 0.22–0.25; $\log \hat{Z} = 1.840$ vs. 1.838 (≈ 3 min)
`idem_estimator.py`	MC score/energy targets and their failure at high noise (Listing 8.1)	Table 8.1 (< 5 s)
`adjoint_sampling.py`	Adjoint Sampling with replay buffer (Listing 9.1)	all 4 modes, weights 0.12–0.34 after 100 rounds (≈ 30 s); Figure 9.2 uses 300 rounds
`asbs.py`	ASBS with Gaussian source, adjoint + corrector matching (Listing 10.1)	all 4 modes (≈ 1 min)
`torus_bridge.py`	wrapped Gaussian and exact torus bridges (Listing 11.1)	TV 0.016 vs. brute force (≈ 10 s)
`bayes_logreg.py`	Bayesian logistic regression with AS, exact path log-weights, MALA reference (Listing 12.1)	Table 12.2 (≈ 1 min)
`molecular_utils.py`	zero-CoM projection, Kabsch RMSD, COV/AMR (Listing 13.1)	COV-R 0.6 on a constructed example (< 1 s)

Extending the scripts. To move from the 2-D examples to particle systems: (i) replace the MLP by an $\mathrm{SE}(3)$-equivariant GNN that takes (x, t) and returns a vector per atom; (ii) project all noise and network outputs with `com_project`; (iii) use the dimension $3n - 3$ in $\log p_1^{\text{base}}$ (Proposition 11.7); (iv) clip forces and increase the number of steps near $t = 1$; (v) log the number of energy evaluations alongside every metric. For torsion-angle targets, swap the Euclidean bridge in `adjoint_sampling.py` for `torus_bridge` and the Gaussian $\log p_1^{\text{base}}$ for `log_wrapped_normal` (Proposition 11.9).

A minimal evaluation harness. For any sampler that returns samples and (optionally) log-weights: report the ELBO $\overline{\log w}$, $\log \frac{1}{M} \sum w$, the normalised ESS, mode weights against known values, and, for molecules, Ramachandran KL and energy-histogram W_2 against a reference, together with the number of energy evaluations used in training and sampling (Section 4.4).

Bibliography

Josh Abramson et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630:493–500, 2024.

Tara Akhound-Sadegh, Jarrid Rector-Brooks, Avishek Joey Bose, Sarthak Mittal, Pablo Lemos, Cheng-Hao Liu, Marcin Sendera, Siamak Ravanbakhsh, Gauthier Gidel, Yoshua Bengio, Nikolay Malkin, and Alexander Tong. Iterated denoising energy matching for sampling from Boltzmann densities. In *International Conference on Machine Learning*, 2024.

Tara Akhound-Sadegh et al. Progressive inference-time annealing of diffusion models for sampling from Boltzmann densities. *arXiv preprint*, 2025.

Michael S. Albergo and Eric Vanden-Eijnden. NETS: A non-equilibrium transport sampler. In *International Conference on Machine Learning*, 2025. arXiv:2410.02711.

Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.

Luigi Ambrosio, Nicola Gigli, and Giuseppe Savaré. *Gradient Flows in Metric Spaces and in the Space of Probability Measures*. Birkhäuser, 2 edition, 2008.

Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.

Michael Arbel, Alex Matthews, and Arnaud Doucet. Annealed flow transport Monte Carlo. In *International Conference on Machine Learning*, 2021.

Simon Axelrod and Rafael Gómez-Bombarelli. GEOM, energy-annotated molecular conformations for property prediction and molecular generation. *Scientific Data*, 9(1):185, 2022.

Alessandro Barducci, Giovanni Bussi, and Michele Parrinello. Well-tempered metadynamics: A smoothly converging and tunable free-energy method. *Physical Review Letters*, 100(2):020603, 2008.

Ilyes Batatia, Dávid Péter Kovács, Gregor N. C. Simm, Christoph Ortner, and Gábor Csányi. MACE: Higher order equivariant message passing neural networks for fast and accurate force fields. In *Advances in Neural Information Processing Systems*, 2022.

Simon Batzner et al. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications*, 13(1):2453, 2022.

Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow network based generative models for non-iterative diverse candidate generation. In *Advances in Neural Information Processing Systems*, 2021.

Julius Berner, Lorenz Richter, and Karen Ullrich. An optimal control perspective on diffusion-based generative modeling. *Transactions on Machine Learning Research*, 2024. arXiv:2211.01364.

Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. In *International Conference on Learning Representations*, 2024.

Denis Blessing, Xiaogang Jia, Johannes Esslinger, Francisco Vargas, and Gerhard Neumann. Beyond ELBOs: A large-scale evaluation of variational methods for sampling. In *International Conference on Machine Learning*, 2024.

Denis Blessing, Julius Berner, Lorenz Richter, and Gerhard Neumann. Underdamped diffusion bridges with applications to sampling. In *International Conference on Learning Representations*, 2025.

Avishek Joey Bose, Tara Akhound-Sadegh, Guillaume Huguet, Kilian Fatras, Jarrid Rector-Brooks, Cheng-Hao Liu, Andrei Cristian Nica, Maksym Korablyov, Michael Bronstein, and Alexander Tong. SE(3)-stochastic flow matching for protein backbone generation. In *International Conference on Learning Representations*, 2024.

Anton Bovier, Michael Eckhoff, Véronique Gayrard, and Markus Klein. Metastability in reversible diffusion processes I: Sharp asymptotics for capacities and exit times. *Journal of the European Mathematical Society*, 6(4):399–424, 2004.

Wenhan Cao, Keyu Yan, and Lin Zhao. One-step sampler for Boltzmann distributions via drifting. *arXiv preprint arXiv:2603.17579*, 2026.

Junhua Chen, Lorenz Richter, Julius Berner, Denis Blessing, Gerhard Neumann, and Anima Anandkumar. Sequential controlled Langevin diffusions. In *International Conference on Learning Representations*, 2025.

Ricky T. Q. Chen and Yaron Lipman. Flow matching on general geometries. In *International Conference on Learning Representations*, 2024.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018.

Yongxin Chen, Tryphon T. Georgiou, and Michele Pavon. Stochastic control liaisons: Richard Sinkhorn meets Gaspard Monge on a Schrödinger bridge. *SIAM Review*, 63(2):249–313, 2021.

Mouyang Cheng, Denis Blessing, Botao Yu, Gerhard Neumann, Mingda Li, Carles Domingo-Enrich, and Yuanqi Du. ATLAS: A foundation neural sampler for amorphous materials. *arXiv preprint arXiv:2607.19198*, 2026.

Jaemoo Choi, Yongxin Chen, Molei Tao, and Guan-Horng Liu. Non-equilibrium annealed adjoint sampler. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2506.18165.

Hyungjin Chung, Jeongsol Kim, Michael T. Mccann, Marc L. Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations*, 2023.

Gavin E. Crooks. Path-ensemble averages in systems driven far from equilibrium. *Physical Review E*, 61(3): 2361–2366, 2000.

Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, 2013.

Paolo Dai Pra. A stochastic control approach to reciprocal diffusion processes. *Applied Mathematics and Optimization*, 23(1):313–329, 1991.

Valentin De Bortoli, James Thornton, Jeremy Heng, and Arnaud Doucet. Diffusion Schrödinger bridge with applications to score-based generative modeling. In *Advances in Neural Information Processing Systems*, 2021.

Valentin De Bortoli, Emile Mathieu, Michael Hutchinson, James Thornton, Yee Whye Teh, and Arnaud Doucet. Riemannian score-based generative modelling. In *Advances in Neural Information Processing Systems*, 2022.

Valentin De Bortoli, Michael Hutchinson, Peter Wirnsberger, and Arnaud Doucet. Target score matching. *arXiv preprint arXiv:2402.08667*, 2024.

Pierre Del Moral, Arnaud Doucet, and Ajay Jasra. Sequential Monte Carlo samplers. *Journal of the Royal Statistical Society: Series B*, 68(3):411–436, 2006.

Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems*, 2021.

Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using Real NVP. In *International Conference on Learning Representations*, 2017.

Carles Domingo-Enrich, Michal Drozdzal, Brian Karrer, and Ricky T. Q. Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. In *International Conference on Learning Representations*, 2025. arXiv:2409.08861.

Joseph L. Doob. Conditional Brownian motion and the boundary limits of harmonic functions. *Bulletin de la Société Mathématique de France*, 85:431–458, 1957.

Xiaochen Du, Juno Nam, Jaemoo Choi, Wei Guo, Sathya Edamadaka, Junyi Sha, Elton Pan, Yongxin Chen, Molei Tao, and Rafael Gómez-Bombarelli. MetaDNS: Enhancing exploration in discrete neural samplers via well-tempered metadynamics. *arXiv preprint arXiv:2605.21722*, 2026.

Yilun Du and Igor Mordatch. Implicit generation and modeling with energy based models. In *Advances in Neural Information Processing Systems*, 2019.

Yuanqi Du, Michael Plainer, Rob Brekelmans, Chenru Duan, Frank Noé, Carla P. Gomes, Alán Aspuru-Guzik, and Kirill Neklyudov. Doob's Lagrangian: A sample-efficient variational approach to transition path sampling. In *Advances in Neural Information Processing Systems*, 2024.

Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural spline flows. In *Advances in Neural Information Processing Systems*, 2019.

Alain Durmus, Maxence Noble, and Thibaut Pellerin. Sampling from multi-modal distributions on Riemannian manifolds with training-free stochastic interpolants. *arXiv preprint arXiv:2602.00641*, 2026.

Peter Eastman, Pavan Kumar Behara, David L. Dotson, Raimondas Galvelis, John E. Herr, Josh T. Horton, Yuezhi Mao, John D. Chodera, Benjamin P. Pritchard, Yuanqing Wang, Gianni De Fabritiis, and Thomas E. Markland. SPICE, a dataset of drug-like molecules and peptides for training machine learning potentials. *Scientific Data*, 10(1):11, 2023.

Marshall Fixman. Classical statistical mechanics of constraints: A theorem and application to polymers. *Proceedings of the National Academy of Sciences*, 71(8):3050–3053, 1974.

Hans Föllmer. An entropy approach to the time reversal of diffusion processes. In *Stochastic Differential Systems Filtering and Control*, pages 156–163. Springer, 1985.

Robert Fortet. Résolution d'un système d'équations de M. Schrödinger. *Journal de Mathématiques Pures et Appliquées*, 19:83–105, 1940.

Xiang Fu, Brandon M. Wood, Luis Barroso-Luque, Daniel S. Levine, Meng Gao, Misko Dzamba, and C. Lawrence Zitnick. Learning smooth and expressive interatomic potentials for physical property prediction. *arXiv preprint*, 2025. The eSEN potential.

Mark Girolami and Ben Calderhead. Riemann manifold Langevin and Hamiltonian Monte Carlo methods. *Journal of the Royal Statistical Society: Series B*, 73(2):123–214, 2011.

Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2019.

Louis Grenioux, Maxence Noble, Marylou Gabrié, and Alain Oliviero Durmus. Stochastic localization via iterative posterior sampling. In *International Conference on Machine Learning*, 2024.

Majdi Hassan, Nikhil Shenoy, Jungyoon Lee, Hannes Stärk, Stephan Thaler, and Dominique Beaini. ET-Flow: Equivariant flow-matching for molecular conformer generation. In *Advances in Neural Information Processing Systems*, 2024.

Ulrich G. Haussmann and Etienne Pardoux. Time reversal of diffusions. *The Annals of Probability*, 14(4): 1188–1205, 1986.

Aaron Havens, Benjamin Kurt Miller, Bing Yan, Carles Domingo-Enrich, Anuroop Sriram, Brandon Wood, Daniel Levine, Bin Hu, Brandon Amos, Brian Karrer, Xiang Fu, Guan-Horng Liu, and Ricky T. Q. Chen. Adjoint sampling: Highly scalable diffusion samplers via adjoint matching. In *International Conference on Machine Learning*, 2025. arXiv:2504.11713.

Aaron Havens, Brian Karrer, and Neta Shaul. Flow sampling: Learning to sample from unnormalized densities via denoising conditional processes. In *International Conference on Machine Learning*, 2026. arXiv:2605.03984.

Jiajun He, Yuanqi Du, Francisco Vargas, Dinghuai Zhang, Shreyas Padhy, RuiKang OuYang, Carla Gomes, and José Miguel Hernández-Lobato. No trick, no treat: Pursuits and challenges towards simulation-free training of neural samplers. *arXiv preprint arXiv:2502.06685*, 2025.

Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.

Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021. arXiv:2207.12598.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.

Lars Holdijk, Yuanqi Du, Ferry Hooft, Priyank Jaini, Bernd Ensing, and Max Welling. Stochastic optimal control for collective variable free sampling of molecular transition paths. In *Advances in Neural Information Processing Systems*, 2023.

Elton P. Hsu. *Stochastic Analysis on Manifolds*. American Mathematical Society, 2002.

Chin-Wei Huang, Milad Aghajohari, Joey Bose, Prakash Panangaden, and Aaron Courville. Riemannian diffusion models. In *Advances in Neural Information Processing Systems*, 2022.

Michael F. Hutchinson. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics – Simulation and Computation*, 18(3):1059–1076, 1989.

Christopher Jarzynski. Nonequilibrium equality for free energy differences. *Physical Review Letters*, 78(14): 2690–2693, 1997.

Bowen Jing, Gabriele Corso, Jeffrey Chang, Regina Barzilay, and Tommi Jaakkola. Torsional diffusion for molecular conformer generation. In *Advances in Neural Information Processing Systems*, 2022.

Bowen Jing, Bonnie Berger, and Tommi Jaakkola. AlphaFold meets flow matching for generating protein ensembles. In *International Conference on Machine Learning*, 2024.

Hilbert J. Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of Statistical Mechanics: Theory and Experiment*, 2005(11):P11011, 2005.

Ioannis Karatzas and Steven E. Shreve. *Brownian Motion and Stochastic Calculus*. Springer, 2 edition, 1991.

Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014.

Leon Klein and Frank Noé. Transferable Boltzmann generators. In *Advances in Neural Information Processing Systems*, 2024.

Leon Klein, Andreas Krämer, and Frank Noé. Equivariant flow matching. In *Advances in Neural Information Processing Systems*, 2023.

Jonas Köhler, Leon Klein, and Frank Noé. Equivariant flows: Exact likelihood generative learning for symmetric densities. In *International Conference on Machine Learning*, 2020.

Solomon Kullback. Probability densities with given marginals. *The Annals of Mathematical Statistics*, 39(4): 1236–1243, 1968.

Salem Lahlou, Tristan Deleu, Pablo Lemos, Dinghuai Zhang, Alexandra Volokhova, Alex Hernández-García, Léna Néhale Ezzine, Yoshua Bengio, and Nikolay Malkin. A theory of continuous generative flow networks. In *International Conference on Machine Learning*, 2023.

Alessandro Laio and Michele Parrinello. Escaping free-energy minima. *Proceedings of the National Academy of Sciences*, 99(20):12562–12566, 2002.

Christian Léonard. A survey of the Schrödinger problem and some of its connections with optimal transport. *Discrete and Continuous Dynamical Systems – A*, 34(4):1533–1574, 2014.

Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.

Sarah Lewis et al. Scalable emulation of protein equilibrium ensembles with generative deep learning. *Science*, 2025.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

Guan-Horng Liu, Arash Vahdat, De-An Huang, Evangelos A. Theodorou, Weili Nie, and Anima Anandkumar. I^2SB: Image-to-image Schrödinger bridge. In *International Conference on Machine Learning*, 2023.

Guan-Horng Liu, Jaemoo Choi, Yongxin Chen, Benjamin Kurt Miller, and Ricky T. Q. Chen. Adjoint Schrödinger bridge sampler. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2506.22565.

Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in GFlownets. In *Advances in Neural Information Processing Systems*, 2022.

Alexander G. D. G. Matthews, Michael Arbel, Danilo J. Rezende, and Arnaud Doucet. Continual repeated annealed flow transport Monte Carlo. In *International Conference on Machine Learning*, 2022.

Laurence Illing Midgley, Vincent Stimper, Javier Antorán, Emile Mathieu, Bernhard Schölkopf, and José Miguel Hernández-Lobato. SE(3) equivariant augmented coupling flows. In *Advances in Neural Information Processing Systems*, 2023a.

Laurence Illing Midgley, Vincent Stimper, Gregor N. C. Simm, Bernhard Schölkopf, and José Miguel Hernández-Lobato. Flow annealed importance sampling bootstrap. In *International Conference on Learning Representations*, 2023b.

Mattia Mosso, Jaemoo Choi, and Heng Yang. Hard-constrained sampling on embedded Riemannian manifolds via adjoint Schrödinger bridges. *arXiv preprint arXiv:2608.25838*, 2026.

Juno Nam, Bálint Máté, Artur P. Toshev, Manasa Kaniselvan, Rafael Gómez-Bombarelli, Ricky T. Q. Chen, Brandon Wood, Guan-Horng Liu, and Benjamin Kurt Miller. Enhancing diffusion-based sampling with molecular collective variables. *arXiv preprint arXiv:2510.11923*, 2025.

Radford M. Neal. Annealed importance sampling. *Statistics and Computing*, 11(2):125–139, 2001.

Radford M. Neal. MCMC using Hamiltonian dynamics. In *Handbook of Markov Chain Monte Carlo*. Chapman and Hall/CRC, 2011.

Frank Noé, Simon Olsson, Jonas Köhler, and Hao Wu. Boltzmann generators: Sampling equilibrium states of many-body systems with deep learning. *Science*, 365(6457):eaaw1147, 2019.

Nikolas Nüsken and Lorenz Richter. Solving high-dimensional Hamilton–Jacobi–Bellman PDEs using neural networks: perspectives from the theory of controlled diffusions and measures on path space. *Partial Differential Equations and Applications*, 2(4):48, 2021.

Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Springer, 6 edition, 2003.

Long Ouyang et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

RuiKang OuYang, Bo Qiang, and José Miguel Hernández-Lobato. BNEM: A Boltzmann sampler based on bootstrapped noised energy matching. *Transactions on Machine Learning Research*, 2026. arXiv:2409.09787.

Art B. Owen. *Monte Carlo Theory, Methods and Examples*. 2013. Online book, https://artowen.su.domains/mc/.

George Papamakarios, Eric Nalisnick, Danilo J. Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22 (57):1–64, 2021.

Angus Phillips, Hai-Dang Dau, Michael John Hutchinson, Valentin De Bortoli, George Deligiannidis, and Arnaud Doucet. Particle denoising diffusion sampler. In *International Conference on Machine Learning*, 2024.

Philipp Pracht, Fabian Bohle, and Stefan Grimme. Automated exploration of the low-energy chemical space with fast quantum chemical methods. *Physical Chemistry Chemical Physics*, 22(14):7169–7192, 2020.

Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International Conference on Machine Learning*, 2015.

Danilo Jimenez Rezende, George Papamakarios, Sébastien Racanière, Michael Albergo, Gurtej Kanwar, Phiala Shanahan, and Kyle Cranmer. Normalizing flows on tori and spheres. In *International Conference on Machine Learning*, 2020.

Lorenz Richter and Julius Berner. Improved sampling via learned diffusions. In *International Conference on Learning Representations*, 2024.

Lorenz Richter, Ayman Boustati, Nikolas Nüsken, Francisco Ruiz, and Ömer Deniz Akyildiz. VarGrad: A low-variance gradient estimator for variational inference. In *Advances in Neural Information Processing Systems*, 2020.

Severi Rissanen et al. Progressive tempering sampler with diffusion. In *International Conference on Machine Learning*, 2025.

Gareth O. Roberts and Richard L. Tweedie. Exponential convergence of Langevin distributions and their discrete approximations. *Bernoulli*, 2(4):341–363, 1996.

Ludger Rüschendorf. Convergence of the iterative proportional fitting procedure. *The Annals of Statistics*, 23(4):1160–1174, 1995.

Ludger Rüschendorf and Wolfgang Thomsen. Note on the Schrödinger equation and I-projections. *Statistics & Probability Letters*, 17(5):369–375, 1993.

Víctor Garcia Satorras, Emiel Hoogeboom, and Max Welling. E(n) equivariant graph neural networks. In *International Conference on Machine Learning*, 2021.

Marcin Sendera, Minsu Kim, Sarthak Mittal, Pablo Lemos, Luca Scimeca, Jarrid Rector-Brooks, Alexandre Adam, Yoshua Bengio, and Nikolay Malkin. Improved off-policy training of diffusion samplers. In *Advances in Neural Information Processing Systems*, 2024.

Yuyang Shi, Valentin De Bortoli, Andrew Campbell, and Arnaud Doucet. Diffusion Schrödinger bridge matching. In *Advances in Neural Information Processing Systems*, 2023.

Michael R. Shirts and John D. Chodera. Statistically optimal analysis of samples from multiple equilibrium states. *The Journal of Chemical Physics*, 129(12):124105, 2008.

Marta Skreta, Tara Akhound-Sadegh, Viktor Ohanesian, Roberto Bondesan, Alán Aspuru-Guzik, Arnaud Doucet, Rob Brekelmans, Alexander Tong, and Kirill Neklyudov. Feynman–Kac correctors in diffusion: Annealing, guidance, and product of experts. In *International Conference on Machine Learning*, 2025.

Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, 2015.

Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021a.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021b.

Charlie B. Tan, Avishek Joey Bose, Chen Lin, Leon Klein, Michael M. Bronstein, and Alexander Tong. Scalable equilibrium sampling with sequential Boltzmann generators. In *International Conference on Machine Learning*, 2025a. arXiv:2502.18462.

Charlie B. Tan et al. Amortized sampling with transferable normalizing flows. *arXiv preprint arXiv:2508.18175*, 2025b.

Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. 2024.

Suriyanarayanan Vaikuntanathan and Christopher Jarzynski. Escorted free energy simulations: Improving convergence by reducing dissipation. *Physical Review Letters*, 100(19):190601, 2008.

Francisco Vargas, Will Grathwohl, and Arnaud Doucet. Denoising diffusion samplers. In *International Conference on Learning Representations*, 2023a.

Francisco Vargas, Andrius Ovsianas, David Fernandes, Mark Girolami, Neil D. Lawrence, and Nikolas Nüsken. Bayesian learning via neural Schrödinger–Föllmer flows. *Statistics and Computing*, 33, 2023b.

Francisco Vargas, Shreyas Padhy, Denis Blessing, and Nikolas Nüsken. Transport meets variational inference: Controlled Monte Carlo diffusions. In *International Conference on Learning Representations*, 2024.

Cédric Villani. *Optimal Transport: Old and New*. Springer, 2009.

Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

Yuyang Wang, Ahmed A. Elhag, Navdeep Jaitly, Joshua M. Susskind, and Miguel Ángel Bautista. Swallowing the bitter pill: Simplified scalable conformer generation. In *International Conference on Machine Learning*, 2024.

Max Welling and Yee Whye Teh. Bayesian learning via stochastic gradient Langevin dynamics. In *International Conference on Machine Learning*, 2011.

Peter Wirnsberger, Andrew J. Ballard, George Papamakarios, Stuart Abercrombie, Sébastien Racanière, Alexander Pritzel, Danilo Jimenez Rezende, and Charles Blundell. Targeted free energy estimation via learned mappings. *The Journal of Chemical Physics*, 153(14):144112, 2020.

Dongyeop Woo and Sungsoo Ahn. Iterated energy-based flow matching for sampling from Boltzmann densities. In *ICML Workshop on Structured Probabilistic Inference and Generative Modeling*, 2024.

Luhuan Wu, Brian L. Trippe, Christian A. Naesseth, David M. Blei, and John P. Cunningham. Practical and asymptotically exact conditional sampling in diffusion models. In *Advances in Neural Information Processing Systems*, 2023.

Minkai Xu, Lantao Yu, Yang Song, Chence Shi, Stefano Ermon, and Jian Tang. GeoDiff: A geometric diffusion model for molecular conformation generation. In *International Conference on Learning Representations*, 2022.

Jason Yim, Brian L. Trippe, Valentin De Bortoli, Emile Mathieu, Arnaud Doucet, Regina Barzilay, and Tommi Jaakkola. SE(3) diffusion model with application to protein backbone generation. In *International Conference on Machine Learning*, 2023.

Dinghuai Zhang, Ricky T. Q. Chen, Cheng-Hao Liu, Aaron Courville, and Yoshua Bengio. Diffusion generative flow samplers: Improving learning signals through partial trajectory optimization. In *International Conference on Learning Representations*, 2024.

Qinsheng Zhang and Yongxin Chen. Path integral sampler: A stochastic control approach for sampling. In *International Conference on Learning Representations*, 2022.